

Graph Theory: From First Definitions to Research

*A book that starts at "what is a graph" and ends at graph minors and
spectral expansion,*

36 sections · approximately 49,324 words

Johan De Clercq

Generated from the markdown sources

Contents

Chapter 1 — What a graph is.....	3
Chapter 2 — Representations	8
Chapter 3 — Degree	13
Chapter 4 — Walks, paths, connectivity.....	17
Chapter 5 — Isomorphism.....	21
Chapter 6 — Trees	25
Chapter 7 — Spanning trees and Cayley's formula	29
Chapter 8 — Traversal	33
Chapter 9 — Minimum spanning trees.....	37
Chapter 10 — Shortest paths	41
Chapter 11 — All-pairs distance.....	45
Chapter 12 — Connectivity and Menger's theorem.....	50
Chapter 13 — Max-flow min-cut	54
Chapter 14 — Matching	58
Chapter 15 — Colouring.....	63
Chapter 16 — Bipartite graphs	67
Chapter 17 — Planarity	70
Chapter 18 — The five and four colour theorems	74
Chapter 19 — Perfect and chordal graphs	77
Chapter 20 — Hamiltonicity.....	81
Chapter 21 — Cliques, independent sets, covers.....	84
Chapter 22 — NP-hardness	87
Chapter 23 — Living with hardness	91
Chapter 24 — The probabilistic method.....	95
Chapter 25 — Random graphs.....	99
Chapter 26 — The giant component.....	103
Chapter 27 — Extremal graph theory.....	107
Chapter 28 — Ramsey theory.....	110
Chapter 29 — Spectral graph theory	113
Chapter 30 — The Laplacian.....	117
Chapter 31 — Minors and treewidth	121
Chapter 32 — Expanders, and where to go next	125
Appendix A — Notation	129
Appendix B — Glossary.....	132
Appendix C — Further reading	135
Appendix E — Solutions to the exercises	138

Chapter 1 — What a graph is

A graph is a set of things and a set of pairs of those things. That is the whole definition, and almost everything difficult in this book comes from how little it says.

The definition

A **graph** G is a pair (V, E) where V is a finite set of **vertices** and E is a set of unordered pairs of distinct vertices, the **edges**.

Read that again for what it excludes. E is a *set*, so there is no such thing as two edges between the same pair — you either have the edge or you do not. The pairs are of *distinct* vertices, so no vertex joins itself. The pairs are *unordered*, so an edge from u to v is the same object as an edge from v to u . And V is *finite*, which is a convenience this book keeps throughout and which real research often drops.

A graph satisfying all of those is called **simple**, and unless a chapter says otherwise, every graph here is simple. The three relaxations each have a name:

- allowing repeated edges gives a **multigraph**;
- allowing an edge from a vertex to itself gives a **loop**;
- making the pairs ordered gives a **directed graph**, or **digraph**.

Chapters 10 and 13 need directions and weights, and pick them up there. Everything before that gets by without.

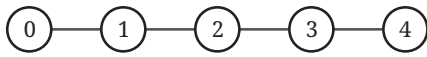
Two numbers name themselves so often that they get single letters: $n = |V|$ and $m = |E|$. If you see $O(n + m)$ in this book — and you will, constantly — it is the size of the graph, not of anything else.

```
from graphs.core import Graph

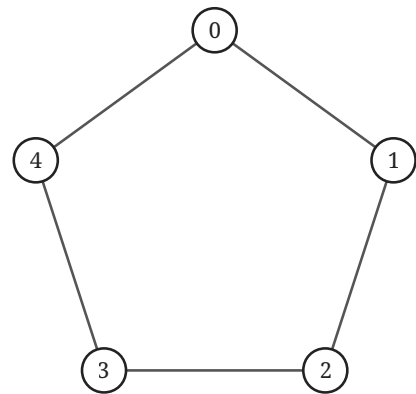
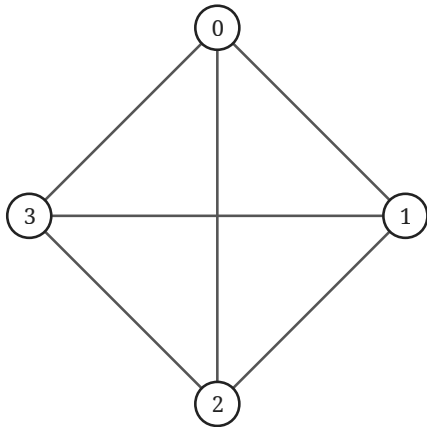
g = Graph(4, [(0, 1), (1, 2), (2, 3), (3, 0)])
print(g.n, g.m)           # 4 4
print(sorted(g.edges()))  # [(0, 1), (0, 3), (1, 2), (2, 3)]
print(g.neighbours(1))    # {0, 2}
```

The vertices here are 0, 1, 2, 3. Nothing forces that; vertices can be cities, people, or web pages. Integers are used throughout this book because they index into arrays, and Chapter 2 is about why that matters more than it sounds.

The drawing is not the graph



P5 - a path: 5 vertices, 4 edges

C5 - odd cycle, $\chi = 3$, not bipartiteK4 - complete, planar, $\chi = 4$

Three of the five named graphs above, drawn. Each picture is *a* choice — the graph supplies no positions, and Chapter 17 is about what does and does not survive that choice.

Draw the graph above and you get a square. Draw it again with the vertices in a different order and you get a bow tie with a crossing in the middle. Both drawings are correct, because **a graph has no geometry**. There are no positions, no lengths, no angles, and no notion of one edge crossing another. A drawing adds all of that, and none of it is data.

This is the single most common beginner error, and it survives well past being a beginner: reasoning about a graph using a property of the picture you drew of it. The picture has a left and a right. The graph does not.

The distinction is not pedantry, because two of it matter later and they pull in opposite directions:

- **Planarity** (Chapter 17) is the question of whether *some* drawing avoids crossings. It is a property of the graph, precisely because it quantifies over all drawings. The drawing you happened to make says nothing.
- **Graph layout**, the problem of producing a drawing a human can read, is not graph theory at all. It is optimisation over a space the graph knows nothing about.

Keep the difference sharp and Chapter 17 is easy. Blur it and Kuratowski's theorem will seem to be about pictures.

What a graph cannot say

The definition is spare, and the things it leaves out are worth naming, because reaching for a graph when you need one of these is how models go wrong.

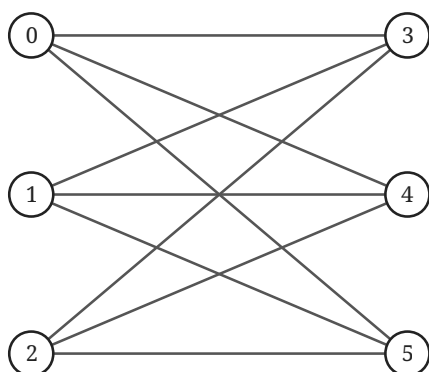
Relations among three or more things at once. An edge joins exactly two vertices. If your relation is genuinely three-way — three authors on one paper, three reagents in one reaction — encoding it as three pairwise edges loses information, and you cannot get it back. The triangle $\{a,b\}, \{b,c\}, \{a,c\}$ is indistinguishable from three separate collaborations that never met. What you want is a **hypergraph**, where an edge is any subset.

Order or multiplicity. Two flights between the same cities are one edge. If the second flight matters, you need a multigraph, and the standard results change: Chapter 3's handshake lemma survives, Chapter 6's characterisation of trees does not.

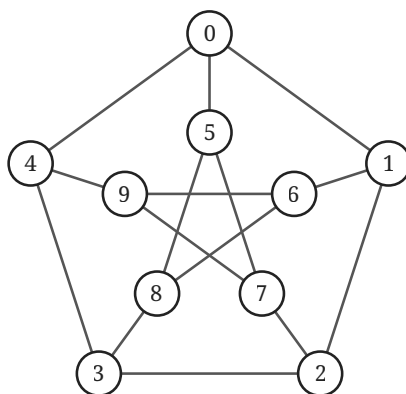
Anything about the vertices themselves. A graph does not know that vertex 3 is a person. Attributes live outside the structure, in a dictionary you carry alongside. This is a feature — every theorem in this book holds regardless of what the vertices mean — but it means a graph alone is rarely a complete model of anything.

Direction, unless you ask for it. "Alice follows Bob" is not symmetric, and modelling it with an undirected edge is a lie that will not announce itself. Roughly half the results in this book have directed analogues; some are harder (Chapter 12), some are easier, and some are false.

A first family of graphs



$K_{3,3}$ - bipartite, and not planar



The Petersen graph - the standard counterexample

$K_{3,3}$ drawn as two columns, which makes its bipartiteness obvious; the Petersen graph in its customary pentagon-and-pentagram layout. Neither drawing is forced by the graph.

Five graphs come up so often they get names, and you should be able to picture all five.

Name	Notation	n	m
Complete graph	K_n	n	$n(n-1)/2$
Empty graph	—	n	0
Path	P_n	n	$n - 1$
Cycle	C_n ($n \geq 3$)	n	n
Complete bipartite	$K_{\{a,b\}}$	$a + b$	ab

```
from graphs.core import complete, cycle, complete_bipartite, petersen

print(complete(5).m)           # 10 = 5*4/2
print(cycle(7).m)              # 7
print(complete_bipartite(3, 3).m) # 9
print(petersen().degree_sequence()) # [3, 3, 3, 3, 3, 3, 3, 3, 3]
```

The last one is the **Petersen graph**, and it earns its keep as this book's standard counterexample. It is 3-regular, it is not planar, it is not Hamiltonian, and its chromatic number is 3. Nearly every plausible-sounding conjecture a reader invents in Chapters 15 to 20 dies on the Petersen graph, which is the most efficient reason to memorise it now.

Try it

Convince yourself the drawing carries no information, by building the same graph two ways and checking they are equal as *labelled* graphs:

```
python -c "
from graphs.core import Graph, cycle
square = Graph(4, [(0,1),(1,2),(2,3),(3,0)])
bowtie = Graph(4, [(3,0),(2,3),(0,1),(1,2)])
print('same graph:', square == bowtie)
print('is it C_4:', square == cycle(4))
"
```

```
same graph: True
is it C_4: True
```

The edge lists were written in different orders and one was described as a square and the other as a bow tie. They are the same object, and `Graph.__eq__` does not care how you drew it.

Now try the harder version, where the *labels* differ:

```
python -c "
from graphs.core import Graph, cycle
relabelled = Graph(4, [(0,2),(2,1),(1,3),(3,0)])
print('equal as labelled graphs:', relabelled == cycle(4))
"
```

```
equal as labelled graphs: False
```

Both are four-cycles. They are not equal, because equality of labelled graphs asks whether the *same pairs* are joined, and here they are not. The relation you actually wanted is isomorphism, and it is hard enough to need Chapter 5 to itself.

Exercises

1. K_n has $n(n-1)/2$ edges. Verify this for $n = 6$ by counting, and check it against `complete(6).m`.
2. The Petersen graph is 3-regular on 10 vertices. How many edges does it have, without counting them one by one?
3. Write two different edge lists that produce the *same labelled graph*, and one that produces a different labelled graph on the same vertices.
4. Three authors write one paper together. Explain what is lost when this is modelled as three pairwise edges, and name the structure that would not lose it.

Solutions in Appendix E.

Takeaways

- A graph is a finite vertex set and a set of unordered pairs. Simple by default: no loops, no repeated edges, no directions.
- n and m are the vertex and edge counts, and are used without introduction from here.
- A graph has no geometry. Any argument that depends on your drawing is not an argument about the graph.
- Pairwise-only, unordered, unlabelled: if your model needs three-way relations, edge multiplicity, or direction, say so explicitly, because the default definition will silently drop them.
- Learn the Petersen graph early. It refutes more guesses than any other graph in the book.

Chapter 2 — Representations

A graph is an abstract object. A graph in memory is a concrete one, and the gap between them is where every algorithm in this book gets its running time.

There are three standard ways to store a graph, and the textbook comparison between them is a table of asymptotics. This chapter gives you that table, and then measures it, and the measurement disagrees with the table in three places. All three disagreements are honest — the asymptotics are correct, and they are also not what happens at the sizes you will actually run.

The three representations

Edge list. Store the pairs and nothing else. $O(m)$ space, and every question except "how many edges" requires a scan. It is the right choice for input, output, and nothing else. Kruskal's algorithm in Chapter 9 is the one place in this book where it is genuinely the natural structure, because that algorithm's first act is to sort the edges anyway.

Adjacency list. For each vertex, the set of its neighbours. $O(n + m)$ space. Scanning one vertex's neighbours costs $O(\deg(v))$ — optimal, because that is the size of the answer. Asking whether a specific edge exists costs $O(\deg(v))$ in a linked list, or $O(1)$ expected in a hash set, which is what `graphs.core.Graph` uses.

Adjacency matrix. An $n \times n$ array of bits, symmetric for an undirected graph. $O(n^2)$ space regardless of how few edges there are. Edge queries are a single array index. Scanning a vertex's neighbours costs $O(n)$, because you must look at every entry in the row including the zeros.

```
from graphs.core import Graph
from graphs.matrix import MatrixGraph

g = Graph(4, [(0, 1), (1, 2), (2, 3)])
mg = MatrixGraph.of(g)
print(g.has_edge(0, 1), mg.has_edge(0, 1))      # True True
print(sorted(g.neighbours(1)), sorted(mg.neighbours(1)))  # [0, 2] [0, 2]
```

`MatrixGraph` stores each row as a single Python integer used as a bitmask. That choice is what makes the measurements below interesting, and it is discussed honestly at the end.

The table everyone writes:

	edge list	adjacency list	adjacency matrix
space	$O(m)$	$O(n + m)$	$O(n^2)$
<code>has_edge(u, v)</code>	$O(m)$	$O(1)$ expected	$O(1)$
scan $N(v)$	$O(m)$	$O(\deg v)$	$O(n)$
add edge	$O(1)$	$O(1)$	$O(1)$

Now measure it.

What actually happens

```
python scripts/bench_representations.py
```

`n = 600`, times in microseconds per operation, best of 3

density	m	edge query		scan neighbours		count triangles	
		list	matrix	list	matrix	list	matrix
0.01	1800	0.088	0.180	0.2	54.6	6	56
0.05	8958	0.089	0.189	0.5	57.1	55	65
0.20	35872	0.095	0.202	1.3	62.7	796	89
0.50	89755	0.104	0.202	2.7	69.3	6619	138

Absolute times are this machine's and move a few percent between runs; the ratios are stable. Three things here are not what the table predicts.

The adjacency list wins the edge query, at every density. The table says both are $O(1)$; the measurement says the list is about twice as fast. A hashed set membership test really is one hash and one probe. The matrix's `rows[u] >> v & 1` is not one machine operation, because `rows[u]` is a Python arbitrary-precision integer, and shifting it right by `v` touches every 30-bit digit below `v`. The matrix's edge query is $O(n)$ in disguise.

That is a claim about scaling, so it needs a scaling measurement rather than an assertion:

edge query as `n` grows, `p = 0.05` fixed:

n	bits/row	list	matrix	ratio
64	64	0.084	0.144	1.72x
256	256	0.093	0.163	1.76x
1024	1024	0.098	0.199	2.02x
4096	4096	0.106	0.267	2.52x
16384	16384	0.118	0.674	5.70x

The list's cost is flat — 0.084 to 0.118 microseconds across a 256-fold increase in `n`. The matrix's grows steadily, and by `n = 16384` the "constant-time" structure is **about 6× slower than the "linear-time" one** at the operation it was supposed to win.

Hold the density fixed when you try this. The first version of this measurement varied `p` with `n` by accident, which made the large graphs nearly empty; the rows were then small integers, the shift was cheap, and the timings came out flat. The effect vanished entirely and the wrong conclusion looked well-supported. What makes the rows expensive is the position of the *highest* set bit, not how many bits are set.

The matrix wins triangle counting by 48×, and by more as density rises. This is the operation a matrix exists for. Counting triangles through a vertex means intersecting neighbourhoods, and intersecting two bitmasks is `&` — 64 vertices per machine word, with no per-element interpreter overhead at all:

```
def common_neighbours(self, u: int, v: int) -> int:
    return (self.rows[u] & self.rows[v]).bit_count()
```

At `p = 0.5` that is 138 microseconds against 6619 for the list. The arbitrary-precision integer that made edge queries slow is exactly what makes this fast: the work happens inside one C-level operation instead of inside a

Python loop. Chapter 21 uses this, and it is the difference between a clique search that finishes and one that does not.

The matrix uses 22× less memory than the list, on a sparse graph.

```
memory, n = 600, p = 0.05:
  adjacency list 1,344,576 bytes
  bitset matrix   61,704 bytes
```

$O(n^2)$ beating $O(n + m)$ by a factor of twenty looks impossible, and the asymptotics are not wrong — they are just not in charge at $n = 600$. A Python `set` carries roughly two kilobytes of overhead before it holds anything, and there are 600 of them. A 600-bit integer is about 100 bytes. The crossover where $O(n^2)$ genuinely loses is far to the right of most graphs you will meet, and if you have ever chosen an adjacency list "to save memory" on a graph of a few thousand vertices, it is worth measuring what you saved.

Reading the disagreement correctly

None of this makes the asymptotic table wrong. It makes it incomplete in a specific way, and the specifics matter:

- The matrix's slow edge query is a fact about **encoding a row as one big integer**. A real bitset — numpy, C, or `bitarray` — indexes the containing word directly and is genuinely $O(1)$. If you take one implementation lesson from this chapter, it is that "bitset" and "Python int used as a bitset" have different complexity, and only one of them matches the textbook.
- The list's memory cost is a fact about **Python's set overhead**, not about adjacency lists. In C, an adjacency list at $p = 0.05$ really would be smaller.
- The matrix's triangle-counting win is **not** an artefact. It is the real asymptotic advantage — $n/64$ words instead of $\deg(v)$ interpreted operations — and it survives in any language.

The general lesson is the one this book will repeat: an asymptotic bound tells you the shape of a curve, not its position. Two $O(1)$ operations can differ by sixfold, and which one wins can reverse as n grows.

Choosing

For everything in Parts I through III, use the adjacency list. Traversal, shortest paths, flows and matchings are all $O(n + m)$ or worse in m , and they all scan neighbourhoods, which is the list's best operation and the matrix's worst by two orders of magnitude.

Reach for a matrix when you are doing **set operations on neighbourhoods** — triangle counting, clique search (Chapter 21), common-neighbour similarity — or when the graph is genuinely dense, meaning m is a constant fraction of n^2 rather than merely large.

Reach for an edge list when the edges are the subject: sorting them (Chapter 9), streaming them, or writing them to disk.

Try it

Watch the two representations disagree about which is faster, on the same graph, depending only on the question asked:

```
python -c "
import sys, time, random
sys.path.insert(0, '.')
from graphs.generate import random_graph
from graphs.matrix import MatrixGraph
g = random_graph(800, 0.3, random.Random(1))
mg = MatrixGraph.of(g)

t = time.perf_counter(); sum(1 for _ in g.neighbours(5)); scan_list =
time.perf_counter() - t
t = time.perf_counter(); sum(1 for _ in mg.neighbours(5)); scan_mat =
time.perf_counter() - t
print(f'scan one neighbourhood: list {scan_list*1e6:.0f}us, matrix
{scan_mat*1e6:.0f}us')

t = time.perf_counter(); mg.common_neighbours(5, 6); tri_mat = time.perf_counter()
- t
t = time.perf_counter(); len(g.neighbours(5) & g.neighbours(6)); tri_list =
time.perf_counter() - t
print(f'intersect two:          list {tri_list*1e6:.0f}us, matrix {tri_mat*1e6:.0f}
us')
"
```

The first line favours the list by a wide margin and the second favours the matrix. Neither representation is better; they answer different questions well, and the only way to know which one you need is to know which question you are asking.

Exercises

1. For $n = 1000$ and $p = 0.01$, how many edges are there on average, and how many entries does the adjacency matrix have?
2. Which representation would you choose to count triangles through a given vertex, and why?
3. `MatrixGraph.has_edge` looks like a constant-time bit test. Explain why it is not, in this implementation.
4. In this book's code the bitset matrix used *less* memory than the adjacency list at $n = 600$. Give the reason, and say whether it would still hold in C.

Solutions in Appendix E.

Takeaways

- Edge list for input and output, adjacency list for traversal, adjacency matrix for neighbourhood set operations and dense graphs.
- n and m bound everything, but constants decide real programs. The $O(1)$ matrix edge query measured about $6\times$ slower than the $O(1)$ list query at $n = 16384$, because a Python int is not a machine word.

- The matrix beat the list on memory by $22\times$ at $n = 600$. $O(n^2)$ does not mean "large" at the sizes most graphs actually are.
- Bitmask neighbourhood intersection is the one genuine, language-independent asymptotic win, and Chapter 21 depends on it.
- When a measurement contradicts a bound, hold the other variables fixed and measure again before believing either. Varying density with n hid this chapter's main effect completely.

Chapter 3 — Degree

The degree of a vertex is how many edges touch it. It is the first number you can attach to a graph, it is the cheapest to compute, and a surprising amount follows from it alone.

The handshake lemma

Write $\deg(v)$ for the number of neighbours of v , and $\Delta(G)$ and $\delta(G)$ for the largest and smallest degrees in G .

Theorem (handshake lemma). For every graph, $\sum_v \deg(v) = 2m$.

Proof. Count the pairs (v, e) where v is a vertex, e is an edge, and v is an endpoint of e . Counting by vertex, each v appears in $\deg(v)$ such pairs, giving $\sum_v \deg(v)$. Counting by edge, each edge has exactly two endpoints, giving $2m$. The two counts are of the same set, so they are equal. ■

That is a **double count**: one set, two ways of tallying it. It is the single most reusable proof technique in this book, and it will reappear in Chapter 17 for Euler's formula and in Chapter 27 for Mantel's theorem.

```
from graphs.core import petersen

g = petersen()
print(sum(g.degree(v) for v in g.vertices()), 2 * g.m)    # 30 30
```

The lemma's real content is a parity statement, and this is the form you will actually use:

Corollary. In every graph, the number of vertices of odd degree is even.

Proof. Split the sum in the handshake lemma into even-degree and odd-degree terms. The even terms sum to an even number, and the total $2m$ is even, so the odd terms must sum to an even number too. A sum of odd numbers is even exactly when there is an even count of them. ■

This is why you cannot have a party where every one of nine people shakes hands with exactly three others, and why a 3-regular graph must have an even number of vertices. Both statements sound like they need work; both are one line.

Two vertices always share a degree

Theorem. Every graph with $n \geq 2$ has two vertices of the same degree.

Proof. Degrees lie in $\{0, 1, \dots, n-1\}$, which has n values for n vertices — not yet a contradiction. But 0 and $n-1$ cannot both occur: a vertex of degree $n-1$ is adjacent to everything, so nothing has degree 0 . So the degrees actually lie in a set of size $n-1$, and pigeonhole finishes it. ■

The hypothesis $n \geq 2$ is doing real work, and it is the kind of thing that gets dropped in transcription. On one vertex the claim is false — there is only one vertex, so no pair to compare. The harness registers the guard explicitly rather than trusting the reader:

```
@theorem("Any graph with  $n \geq 2$  has two vertices of equal degree", chapter=3)
def two_vertices_share_a_degree(g: Graph) -> bool | None:
    if g.n < 2:
        return None          # not a pass -- the hypothesis fails, so this graph
    says nothing
    degrees = [g.degree(v) for v in g.vertices()]
    return len(set(degrees)) < len(degrees)
```

Returning `None` rather than `True` matters. `True` would mean "the theorem held here", which is a lie about a graph the theorem does not address.

Degree sequences

The **degree sequence** is the list of degrees, conventionally non-increasing. It is an invariant: relabelling the vertices cannot change it. So it is a fast way to prove two graphs are *not* isomorphic — and, as Chapter 5 shows, a badly incomplete way to prove they are.

The interesting question runs the other way. Given a list of numbers, is there a graph with those degrees? Such a list is called **graphical**.

Some fail for easy reasons. $[1, 1, 1]$ has an odd sum, so the handshake lemma kills it. $[5, 1, 1, 1]$ has a vertex wanting five neighbours in a graph with four vertices. But $[3, 3, 3, 1]$ has an even sum and no entry too large, and is still not graphical — and seeing why requires an argument rather than an observation.

Havel–Hakimi

The algorithm is greedy and the proof is the algorithm.

Theorem (Havel–Hakimi). Let $d_1 \geq d_2 \geq \dots \geq d_n$ with $d_1 \geq 1$. The sequence is graphical if and only if the sequence obtained by deleting d_1 and subtracting one from each of the next d_1 entries is graphical.

Proof. One direction is easy: given a graph for the reduced sequence, add a new vertex joined to the d_1 vertices whose degrees were reduced, and you have a graph for the original.

The other direction is the content. Suppose G realises the original sequence, with v the vertex of degree d_1 . If v is adjacent to exactly the d_1 vertices of next-largest degree, delete it and you are done. If not, there are vertices x and y with $\deg(x) \geq \deg(y)$, where v is adjacent to y but not to x . Since $\deg(x) \geq \deg(y)$ and x is joined to v 's non-neighbour set more than y is, there must be a vertex z adjacent to x but not to y . Now delete the edges vy and xz , and add vx and yz . Every degree is unchanged, and v is now adjacent to x instead of y . Repeat; each swap strictly increases the number of high-degree vertices adjacent to v , so the process terminates in a realisation of the required shape. ■

That **two-swap** argument — exchange a pair of edges for a pair that preserves all degrees — is worth remembering. It is the same move that proves Chapter 14's augmenting-path theorem and Chapter 20's Ore condition.

The algorithm is the theorem applied repeatedly:

```

from graphs.degree import is_graphical_havel_hakimi, realise

print(is_graphical_havel_hakimi([3, 3, 3, 1]))    # False
print(is_graphical_havel_hakimi([3, 3, 2, 2, 1, 1])) # True
print(sorted(realise([3, 3, 2, 2, 1, 1]).edges()))
# [(0, 1), (0, 4), (0, 5), (1, 2), (1, 3), (2, 3)]

```

Because the proof is constructive, `realise` gives you an actual graph rather than a yes.

Erdős–Gallai

There is also a non-constructive characterisation, and it is the one to use when you want a criterion rather than a graph.

Theorem (Erdős–Gallai). A non-increasing sequence of non-negative integers with even sum is graphical if and only if for every k from 1 to n ,

$$\sum_{i \leq k} d_i \leq k(k-1) + \sum_{i > k} \min(d_i, k).$$

The inequality says something you can read: the k largest-degree vertices must absorb their degree either among themselves — at most $k(k-1)$ edge-endpoints — or by reaching out to the rest, where each remaining vertex i can accept at most $\min(d_i, k)$ of them. If they cannot, the sequence is impossible.

Two theorems, two algorithms, and no reason to assume they agree. So the harness checks both against exhaustive search rather than against each other:

```

held      ch 3  Havel-Hakimi agrees with brute-force realisability  (52 graphs)
held      ch 3  Erdos-Gallai agrees with Havel-Hakimi  (52 graphs)
held      ch 3  Havel-Hakimi's construction really has the requested degrees  (52
graphs)

```

The first line is the one that matters. Checking Havel–Hakimi against Erdős–Gallai alone would only establish that two implementations of mine agree, which they might do while both being wrong. Checking against "try every graph on n vertices and look" cannot fail that way.

Try it

```

python -c "
import sys; sys.path.insert(0, '.')
from graphs.degree import is_graphical_havel_hakimi, is_graphical_erdos_gallai,
is_graphical_bruteforce
for s in ([3,3,3,3], [3,3,3,1], [1,1,1], [5,1,1,1,1,1], [2,2,2]):
    hh = is_graphical_havel_hakimi(list(s))
    eg = is_graphical_erdos_gallai(list(s))
    bf = is_graphical_bruteforce(list(s))
    print(f'{str(s):<18} Havel-Hakimi={hh!s:<6} Erdos-Gallai={eg!s:<6}
exhaustive={bf!s:<6}')
"

```

[3, 3, 3, 3]	Havel-Hakimi=True	Erdos-Gallai=True	exhaustive=True
[3, 3, 3, 1]	Havel-Hakimi=False	Erdos-Gallai=False	exhaustive=False
[1, 1, 1]	Havel-Hakimi=False	Erdos-Gallai=False	exhaustive=False
[5, 1, 1, 1, 1, 1]	Havel-Hakimi=True	Erdos-Gallai=True	exhaustive=True
[2, 2, 2]	Havel-Hakimi=True	Erdos-Gallai=True	exhaustive=True

Note the fourth: [5, 1, 1, 1, 1, 1] is graphical on **six** vertices — it is the star $K_{1,5}$ — even though it would be impossible on five. Degree sequences carry n with them, and forgetting that is the most common way to misread this section.

Exercises

1. Can nine people each shake hands with exactly three others? Justify your answer with the handshake lemma.
2. Is [4, 3, 2, 1, 0] graphical? Run Havel–Hakimi by hand.
3. What is the sum of the degrees of the Petersen graph, and what does that tell you about its edge count?
4. The theorem "two vertices share a degree" requires $n \geq 2$. Give the one-vertex graph and explain precisely which step of the proof fails.

Solutions in Appendix E.

Takeaways

- The handshake lemma is a double count, and double counting is the workhorse technique of this book.
- The number of odd-degree vertices is always even. Most parity arguments about graphs reduce to this.
- The degree sequence is an invariant, so it can disprove isomorphism but never prove it.
- Havel–Hakimi decides realisability by construction; Erdős–Gallai decides it by inequality. Both are proved here, and both are checked against exhaustive search rather than against each other.
- When a theorem has a hypothesis like $n \geq 2$, encode it as "this case says nothing" rather than "this case passed".

Chapter 4 — Walks, paths, connectivity

This chapter is vocabulary, and it is worth being fussy about, because three words that beginners use interchangeably are not interchangeable, and half the confusion in later proofs comes from blurring them.

Three words

A **walk** is a sequence of vertices $v_0, v_1, \dots, v_k$ where consecutive vertices are adjacent. Repeats are allowed, in both vertices and edges. Its **length** is k , the number of edges traversed — not the number of vertices, which is $k + 1$.

A **path** is a walk with no repeated vertex.

A **cycle** is a walk of length at least 3 that starts and ends at the same vertex, with no other repeats.

The relationship between the first two is the lemma everything else rests on:

Lemma. If there is a walk from u to v , there is a path from u to v .

Proof. Take a walk from u to v of minimum length. If it repeated a vertex w , the section between the two visits to w could be deleted, leaving a shorter walk from u to v — contradicting minimality. So it repeats no vertex, and is a path. ■

This is why you may always assume "path" when an argument hands you a "walk", and it is used without comment from Chapter 9 onwards. It is also why the two notions genuinely differ: walks are easy to count and paths are not, which is the subject of the next section.

Counting walks

Walks have a clean algebraic description that paths conspicuously lack.

Theorem. Let A be the adjacency matrix of G . The (u, v) entry of A^k is the number of walks of length exactly k from u to v .

Proof. Induction on k . For $k = 1$ the claim is the definition of A . Suppose it holds for k . A walk of length $k+1$ from u to v is a walk of length k from u to some t , followed by an edge tv . Summing over t gives $(A^{k+1})_{uv} = \sum_t (A^k)_{ut} \cdot A_{tv}$, which is exactly the definition of matrix multiplication. ■

```
from graphs.core import cycle, complete
from graphs.algorithms import walk_counts

for row in walk_counts(cycle(4), 2):
    print(row)
```

```
[2, 0, 2, 0]
[0, 2, 0, 2]
[2, 0, 2, 0]
[0, 2, 0, 2]
```

Read it. From vertex 0 there are two walks of length 2 back to vertex 0 (out to either neighbour and back), two to vertex 2 (round either side), and none to vertices 1 and 3 — because C_4 is bipartite, and a walk of even length cannot cross between the two sides. The zeros in that matrix are Chapter 16's theorem, visible three chapters early.

Now try to write the same theorem for paths. You cannot: there is no matrix operation that counts paths, and counting them is $\#P$ -complete. The gap between "walks are linear algebra" and "paths are intractable" is one of the sharpest divides in the subject, and Chapter 29 is what happens when you take the linear-algebra side seriously.

Connectivity

u and v are **connected** if some path joins them. This relation is reflexive (the length-zero walk), symmetric (reverse the path), and transitive (concatenate two walks, then apply the lemma above to get a path) — so it is an equivalence relation, and its classes are the **connected components**.

That "so" is the whole reason the lemma was worth proving. Concatenating two paths does not give a path; it gives a walk. Without the walk-to-path lemma, transitivity fails and components are not well defined.

```
from graphs.core import Graph
from graphs.algorithms import components, is_connected

g = Graph(7, [(0, 1), (1, 2), (2, 0), (3, 4), (5, 6)])
print(components(g))      # [{0, 1, 2}, {3, 4}, {5, 6}]
print(is_connected(g))    # False
```

A graph is **connected** if it has exactly one component. Two conventions need stating rather than assuming, because texts differ and proofs quietly depend on the choice:

- The one-vertex graph **is** connected. Nothing is disputed here.
- The empty graph — no vertices at all — is treated as connected in this book. It has zero components, not one, so this is a convention rather than a consequence. It is chosen because it makes "every graph is the disjoint union of its components" true without a special case, and Chapter 6's edge-count identity works out. Some texts call the empty graph disconnected; when you read one, check which theorems acquire a hypothesis.

The edge budget

Theorem. A connected graph on $n \geq 1$ vertices has $m \geq n - 1$.

Proof. Induction on n . For $n = 1$ we need $m \geq 0$. For the step, take a connected G on n vertices. If every vertex has degree at least 2 then $2m = \sum \deg(v) \geq 2n$, so $m \geq n > n - 1$ and we are done. Otherwise some vertex v has degree at most 1; since G is connected and $n \geq 2$, its degree is exactly 1. Delete v and its edge. The result is connected on $n - 1$ vertices, so has at least $n - 2$ edges by induction, and G has at least $n - 1$. ■

The complementary fact — a graph with $m \geq n$ must contain a cycle — is Chapter 6's, and together they pin trees down exactly.

A second small result that gets used constantly:

Lemma. Removing one edge increases the number of components by at most one.

Proof. Removing uv can only separate vertices whose every path used uv . Any two vertices still joined by a path in $G - uv$ remain in the same component. Any vertex still reaches either u or v in $G - uv$, since a path to it in G that used uv can be truncated at the first of u, v it meets. So the component containing uv splits into at most two pieces, and no other component changes. ■

An edge whose removal *does* increase the count is a **bridge**, and Chapter 12 characterises them: an edge is a bridge exactly when it lies on no cycle.

Distance

For u, v in the same component, $d(u, v)$ is the length of a shortest path between them. It is a metric: non-negative, zero only on the diagonal, symmetric, and satisfying the triangle inequality $d(u, w) \leq d(u, v) + d(v, w)$ — concatenate and shorten.

```
from graphs.core import path
from graphs.algorithms import distances

print(distances(path(5), 0))    # {0: 0, 1: 1, 2: 2, 3: 3, 4: 4}
```

Breadth-first search computes all of these in $O(n + m)$, and Chapter 8 explains why it is correct. The **diameter** is the largest finite distance; the **eccentricity** of v is the largest distance from v ; the **radius** is the smallest eccentricity. Chapter 11 computes all of them, and Chapter 30 relates the diameter to the Laplacian spectrum, which is a much less obvious connection than it sounds.

Try it

Confirm the matrix-power theorem against a direct count, on a graph small enough to check by hand:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete
from graphs.algorithms import walk_counts
w = walk_counts(complete(4), 3)
print('closed walks of length 3 at each vertex:', [w[i][i] for i in range(4)])
"
```

```
closed walks of length 3 at each vertex: [6, 6, 6, 6]
```

Six is right, and you can see why: a closed walk of length 3 from v in K_4 must visit two distinct other vertices, and there are $3 \times 2 = 6$ ordered ways to choose them. Note that this counts each triangle **twice** — once in each direction — which is why the standard formula for the number of triangles is $\text{trace}(A^3)/6$: three starting points, two directions.

Exercises

1. Give a walk that is not a path, and a path that is not a cycle, in C_5 .
2. What does $(A^2)_{vv}$ equal, and why?
3. What is the minimum number of edges in a connected graph on n vertices, and which graphs achieve it?
4. Concatenating two paths need not give a path. Explain why connectivity is still an equivalence relation.

Solutions in Appendix E.

Takeaways

- Walk, path, cycle are three different things. A walk repeats; a path does not; a cycle closes.
- Every walk contains a path with the same endpoints. This is what makes connectivity an equivalence relation, and it is used silently everywhere after this chapter.
- $(A^k)_{uv}$ counts walks of length k . Nothing comparable exists for paths, and that gap is not a gap in our knowledge — counting paths is $\#P$ -complete.
- Connected graphs have $m \geq n - 1$; removing an edge splits at most one component in two.
- State your convention for the empty graph once, then hold to it. This book calls it connected.

Chapter 5 — Isomorphism

Chapter 1 showed two four-cycles that were not equal. They were obviously the same graph. Reconciling those two sentences is what this chapter is for, and the reconciliation turns out to be one of the strangest problems in computer science.

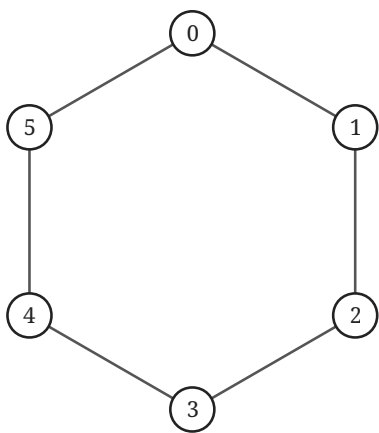
The definition

An **isomorphism** from G to H is a bijection $\phi : V(G) \rightarrow V(H)$ such that $uv \in E(G)$ if and only if $\phi(u)\phi(v) \in E(H)$. If one exists, $G \cong H$.

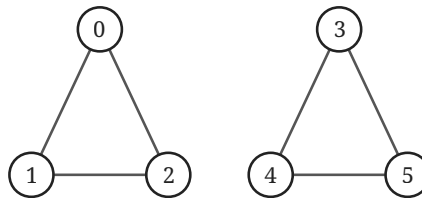
The definition is unremarkable. What is remarkable is that nobody knows how hard it is to decide.

Almost every natural problem in this book is either in P — shortest paths, matching, planarity — or NP -complete — colouring, Hamiltonicity, clique. Graph isomorphism is in NP , is not known to be in P , and is *not believed to be* NP -complete either, because if it were, the polynomial hierarchy would collapse. It sits in a small class of problems suspected of being genuinely intermediate. Babai's 2016 algorithm runs in quasipolynomial time, $\exp((\log n)^{O(1)})$, which is faster than any exponential and slower than any polynomial. That is the current state of the art, and it is a strange place for a problem this easy to state.

Invariants, and what they cannot do



C_6 - even cycle, bipartite



Two triangles - same degrees as C_6 , not isomorphic

The two graphs this chapter turns on: both 2-regular on six vertices, identical degree sequences, and not isomorphic. One is connected and the other is not.

An **invariant** is anything preserved by isomorphism: n , m , the degree sequence, the number of triangles, the multiset of component sizes. Every invariant gives a **one-sided test**. If two graphs differ on any invariant, they are certainly not isomorphic. If they agree on all the invariants you checked, you have learned nothing.

That asymmetry is the whole practical story, and it is worth stating as a claim the harness can check, in both directions:

held	ch 5	Isomorphic graphs have equal degree sequences (52 graphs)
refuted	ch 5	Equal degree sequences imply isomorphic (1 graphs)

The second line is a *success*. It is registered as a theorem expected to fail, and the harness turns red if a counterexample ever stops being found. The counterexample is the smallest one there is:

```
from graphs.iso import cospectral_mates, canonical

c6, two_triangles = cospectral_mates()
print(c6.degree_sequence())          # [2, 2, 2, 2, 2, 2]
print(two_triangles.degree_sequence()) # [2, 2, 2, 2, 2, 2]
print(canonical(c6) == canonical(two_triangles)) # False
```

The six-cycle and two disjoint triangles are both 2-regular on six vertices. Identical degree sequences, same n , same m , and not isomorphic — one is connected and the other is not.

Colour refinement, and its famous blind spot

The standard fast heuristic is **colour refinement**, also called one-dimensional Weisfeiler–Leman. Give every vertex the same colour. Repeatedly recolour each vertex by the pair (its current colour, the sorted multiset of its neighbours' colours). Stop when the partition stops changing.

```
def wl_colours(g, rounds=None):
    colour = [0] * g.n
    for _ in range(rounds if rounds is not None else g.n):
        signature = [
            (colour[v], tuple(sorted(colour[w] for w in g.neighbours(v))))
            for v in g.vertices()
        ]
        relabel = {s: i for i, s in enumerate(sorted(set(signature)))}
        new = [relabel[s] for s in signature]
        if new == colour:
            break
        colour = new
    return colour
```

It runs in near-linear time and it separates almost every pair of graphs you will meet. It is also **sound but not complete**: isomorphic graphs always get the same colour multiset, so a difference proves non-isomorphism, but agreement proves nothing.

Its blind spot is exactly the pair above:

```
from graphs.iso import wl_signature, wl_distinguishes

print(wl_signature(c6))          # ((0, 6),)
print(wl_signature(two_triangles)) # ((0, 6),)
print(wl_distinguishes(c6, two_triangles)) # False
```

Both signatures say "six vertices, all of colour 0". Refinement cannot start, because in a regular graph every vertex looks identical to every other: same colour, same multiset of neighbour colours, forever. The partition is stable at round zero.

This is not a defect of the implementation. **One-dimensional Weisfeiler–Leman cannot distinguish any two regular graphs of the same size and degree**, and there are a great many of those. The fix is k -dimensional WL, which colours k -tuples of vertices instead of vertices; 2-WL separates C_6 from two triangles. But for

every k there are graphs that k -WL fails on, a result of Cai, Fürer and Immerman that closed off the whole approach as a route to a polynomial algorithm.

What actually works

Combine them: refinement first as a cheap filter, brute force only when refinement is undecided.

```
def is_isomorphic(g, h):
    if wl_distinguishes(g, h):
        return False          # cheap, and decisive when it fires
    return canonical(g) == canonical(h)  # expensive, and always right
```

`canonical` takes the lexicographically least edge set over all $n!$ relabellings. It is correct by construction and it is not a tool — it is a definition you can execute. Here is what that costs, enumerating every graph on n vertices up to isomorphism:

```
n=1:    1 graphs up to isomorphism    (0.00s)
n=2:    2 graphs up to isomorphism    (0.00s)
n=3:    4 graphs up to isomorphism    (0.00s)
n=4:   11 graphs up to isomorphism    (0.01s)
n=5:   34 graphs up to isomorphism    (0.53s)
n=6:  156 graphs up to isomorphism    (71.82s)
```

One more vertex costs **135× more time**. Two factors compound: the number of labelled graphs is $2^{\binom{n}{2}}$, which multiplies by 2^n each step, and the canonical form costs $n!$. This is why the verification harness runs exhaustively to $n = 5$ by default and needs an explicit `--exhaustive` flag for $n = 6$, and why $n = 7$ — 1044 isomorphism classes, a number that looks small — is out of reach for this method entirely.

Real isomorphism software (`nauty`, `bliss`, `traces`) does refinement with individualisation: refine, and when the partition stabilises without separating everything, pick one vertex, give it a unique colour, refine again, and backtrack over that choice. In practice it is fast on essentially everything, including graphs specifically constructed to defeat it. The worst case is still exponential.

The counts themselves — 1, 2, 4, 11, 34, 156, 1044 — are worth recognising. They are OEIS A000088, and the fact that this book's brute-force enumerator reproduces them is a check on `canonical` that no theorem of mine could provide.

Try it

Watch the cheap test fail and the expensive one succeed on the same pair:

```
python -c "
import sys, time; sys.path.insert(0, '.')
from graphs.iso import cospectral_mates, wl_distinguishes, canonical
a, b = cospectral_mates()
print('same degree sequence: ', a.degree_sequence() == b.degree_sequence())
print('refinement separates: ', wl_distinguishes(a, b))
print('actually isomorphic: ', canonical(a) == canonical(b))
"
```

```

same degree sequence:  True
refinement separates:  False
actually isomorphic:   False

```

Three lines, and the middle one is the subject of this chapter: a test that says "I cannot tell" is not a test that says "yes".

Exercises

1. Are C_6 and two disjoint triangles isomorphic? Give a reason that does not require any computation.
2. The degree sequence is an invariant. Explain in one sentence why that makes it useful for disproving isomorphism and useless for proving it.
3. Why does colour refinement fail to separate any two regular graphs of the same degree and size?
4. How many graphs are there on 4 vertices up to isomorphism? Check your answer against `all_graphs_up_to_iso(4)`.

Solutions in Appendix E.

Takeaways

- Isomorphism is relabelling. Equality of labelled graphs is a different and much stronger relation.
- The problem is in NP , not known to be in P , and believed not to be NP -complete — a genuinely unusual status. Babai's quasipolynomial algorithm is the best known.
- Invariants are one-sided. A difference disproves isomorphism; agreement proves nothing.
- Colour refinement is fast, sound, and blind to regular graphs of equal degree. C_6 and two triangles is the smallest witness, and no amount of k fixes the approach in general.
- Brute-force canonical forms cost $n!$ and stop being usable at about $n = 6$: 0.53 seconds at $n = 5$, 71.8 at $n = 6$.

Chapter 6 — Trees

A tree is the smallest connected graph on its vertices, the largest acyclic one, and the only one in which every pair of vertices is joined in exactly one way. Those are three different sentences describing the same object, and this chapter proves they are.

Five definitions

Theorem. For a graph G on $n \geq 1$ vertices, the following are equivalent:

1. G is connected and acyclic.
2. G is connected and $m = n - 1$.
3. G is acyclic and $m = n - 1$.
4. Any two vertices are joined by exactly one path.
5. G is connected, and removing any edge disconnects it.

A graph satisfying any of these is a **tree**. An acyclic graph — a disjoint union of trees — is a **forest**.

Proof. We show $1 \implies 4 \implies 5 \implies 2 \implies 3 \implies 1$.

1 $\implies$ 4. Connectivity gives at least one path between any u and v . If there were two distinct paths P and Q , let x be the first vertex after which they diverge and y the next vertex they share. The section of P from x to y and the reverse of Q from y to x form a closed walk with no repeated vertex apart from its endpoints — a cycle, contradicting acyclicity.

4 $\implies$ 5. Unique paths give connectivity outright. If removing uv left G connected, there would be a path from u to v avoiding uv , which together with the edge uv gives two distinct $u-v$ paths.

5 $\implies$ 2. Induct on n . For $n = 1$, $m = 0$. For $n \geq 2$, pick any edge uv . Removing it leaves exactly two components (Chapter 4's lemma bounds the split at two, and it is at least two by hypothesis), say G_1 and G_2 with $n_1 + n_2 = n$. Each inherits property 5, so by induction has $n_i - 1$ edges. Then $m = (n_1 - 1) + (n_2 - 1) + 1 = n - 1$.

2 $\implies$ 3. Suppose G is connected with $m = n - 1$ and contains a cycle. Removing an edge of that cycle keeps the graph connected — the two endpoints are still joined the long way round — and leaves $n - 2$ edges on n vertices, contradicting Chapter 4's bound $m \geq n - 1$ for connected graphs.

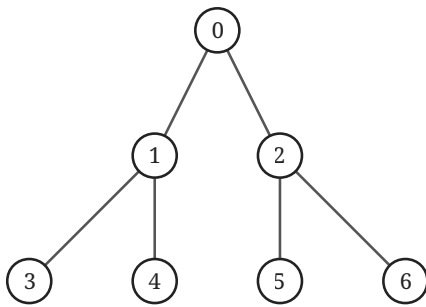
3 $\implies$ 1. Let G be acyclic with components $C_1, \dots, C_k$, each of size n_i . Each component is connected and acyclic, so by 1 $\implies$ 2 (already proved via 4 and 5) has $n_i - 1$ edges. Summing, $m = n - k$. Since $m = n - 1$, we get $k = 1$, so G is connected. ■

Five properties, one object. In practice you use whichever is cheapest for the argument at hand — usually 2 for counting, 4 for constructions, and 5 when you need to delete something.

The harness checks the equivalence of 1 and 2 without using either as the definition: acyclicity is tested by enumerating cycles, not by the $m = n - 1$ shortcut, since using the shortcut would make the claim circular.

held graphs)	ch 6	Tree iff connected and acyclic iff connected with $m = n - 1$ (52
held	ch 6	A tree has a unique path between any two vertices (7 graphs)
held	ch 6	Removing any edge of a tree disconnects it (7 graphs)

Leaves



A tree: 7 vertices, 6 edges, 4 leaves

Seven vertices, six edges, four leaves — and drawn rooted, though a tree has no root until you choose one.

A **leaf** is a vertex of degree 1.

Theorem. Every tree on $n \geq 2$ vertices has at least two leaves.

Proof. Take a longest path $P = v_0 v_1 \dots v_k$ in G ; since $n \geq 2$ and G is connected, $k \geq 1$. Every neighbour of v_0 lies on P — otherwise P could be extended, contradicting maximality. And v_0 has no neighbour on P other than v_1 , since $v_0 v_i$ for $i \geq 2$ would close a cycle. So $\deg(v_0) = 1$, and the same argument applies at v_k . ■

The **longest-path trick** is worth extracting from this proof. Taking a maximal object and observing that its maximality constrains the ends is one of the few genuinely general moves in graph theory; it reappears in Chapter 20 for Hamiltonicity and Chapter 15 for degeneracy orderings.

```
from graphs.core import Graph
from graphs.algorithms import is_tree

t = Graph(7, [(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)])
print(is_tree(t), t.n, t.m)                # True 7 6
print([v for v in t.vertices() if t.degree(v) == 1])  # [3, 4, 5, 6]
```

Leaves are what make induction on trees work. Nearly every proof about trees in this book proceeds by deleting a leaf, applying the inductive hypothesis to the smaller tree, and putting the leaf back — and the theorem above is what guarantees there is always a leaf to delete.

Adding and removing edges

Two complementary facts, both immediate from the equivalences, and both used constantly in Chapter 9:

Lemma (the exchange facts). Let T be a tree.

- Adding any edge not already present creates **exactly one** cycle.
- Removing any edge disconnects T into **exactly two** components.

Proof. For the first: T has a unique path P from u to v by property 4, so adding uv creates the cycle $P + uv$. Any cycle through the new edge must use a $u-v$ path in T , and there is only one, so this cycle is unique. Any cycle not through the new edge would be a cycle in T . For the second: property 5 gives at least two components, and Chapter 4's lemma gives at most two. ■

Put them together and you get the **exchange property**: if you add an edge to a tree and then remove any edge of the resulting cycle, you have a tree again. That single observation is the engine of both minimum-spanning-tree algorithms in Chapter 9, and of the two-swap argument you already saw in Chapter 3.

Trees are bipartite

Corollary. Every tree is bipartite.

Proof. A tree has no cycles, so in particular no odd cycles, and Chapter 16's theorem gives bipartiteness. Or directly: root the tree anywhere and colour each vertex by the parity of its depth. Adjacent vertices differ in depth by exactly one, so they differ in parity. ■

The direct argument is the useful one, because it also tells you the two-colouring explicitly: it is the parity of the distance from the root.

Try it

Confirm the exchange property by hand — add an edge and watch exactly one cycle appear:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.algorithms import is_tree, is_connected

t = Graph(5, [(0,1),(1,2),(2,3),(3,4)])
print('a path is a tree:', is_tree(t), 'm =', t.m, 'n-1 =', t.n - 1)
t.add_edge(0, 4)
print('after adding 0-4: is_tree =', is_tree(t), 'm =', t.m)
t.remove_edge(2, 3)
print('after removing 2-3 from the cycle: is_tree =', is_tree(t), 'connected =',
is_connected(t))
"
```

```

a path is a tree: True m = 4 n-1 = 4
after adding 0-4: is_tree = False m = 5
after removing 2-3 from the cycle: is_tree = True connected = True

```

Adding an edge broke the tree; removing a *different* edge of the cycle it created restored one. That is the exchange property, and Chapter 9 turns it into an algorithm.

Exercises

1. A tree has 12 vertices. How many edges does it have?
2. What is the smallest number of leaves a tree on $n \geq 2$ vertices can have, and which tree achieves it?
3. You add one edge to a tree. How many cycles does the result contain?
4. Every tree is bipartite. Is every bipartite graph a tree? Give a witness.

Solutions in Appendix E.

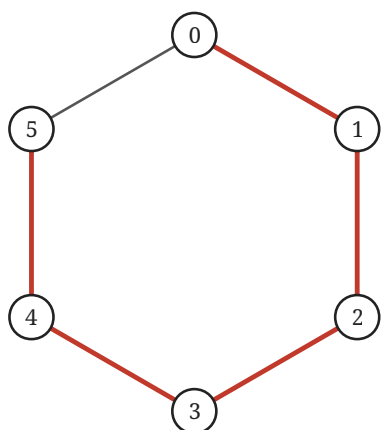
Takeaways

- Five definitions of a tree, all equivalent. Use whichever makes the proof shortest: $m = n - 1$ for counting, unique paths for construction, "every edge is a bridge" for deletion arguments.
- Every tree with at least two vertices has at least two leaves, proved by the longest-path trick — a move that recurs throughout the book.
- Adding an edge creates exactly one cycle; removing an edge creates exactly two components. Together they give the exchange property, which powers Chapter 9.
- Induction on trees means deleting a leaf. The two-leaf theorem is what makes that always possible.

Chapter 7 — Spanning trees and Cayley's formula

A **spanning tree** of G is a subgraph that is a tree and includes every vertex. It is the cheapest way to keep a graph connected, and counting how many a graph has turns out to be a much richer question than it looks.

Existence



A spanning tree of C_6 (red): 5 of the 6 edges

A spanning tree of C_6 : delete any one edge of the cycle and what remains is a tree on all six vertices. C_6 has exactly six spanning trees, one per deleted edge.

Theorem. Every connected graph has a spanning tree.

Proof. Among all connected spanning subgraphs of G , take one with the fewest edges; call it T . If T contained a cycle, removing an edge of that cycle would leave it connected (Chapter 6) and use fewer edges, contradicting minimality. So T is connected and acyclic: a tree. ■

That is the **extremal argument**, and it is the second general move worth naming after the longest-path trick. Take the extreme object, then show that any defect would let you go further. Chapters 13 and 27 use it again.

The proof is not an algorithm — "take a minimal one" is not a procedure — but two algorithms fall out immediately, and Chapter 8 gives both: the BFS tree and the DFS tree of a connected graph are spanning trees, each computed in $O(n + m)$.

Counting them

How many spanning trees does a graph have? For K_5 , brute force says 125. For K_6 , 1296. Those are 5^3 and 6^4 .

Theorem (Cayley, 1889). The complete graph K_n has exactly n^{n-2} labelled spanning trees — equivalently, there are n^{n-2} distinct trees on a fixed vertex set of size n .

The exhaustive count agrees, which is worth doing before trusting any proof:

K_2:	cayley	$n^{(n-2)}$	=	1	enumerated	=	1
K_3:	cayley	$n^{(n-2)}$	=	3	enumerated	=	3
K_4:	cayley	$n^{(n-2)}$	=	16	enumerated	=	16
K_5:	cayley	$n^{(n-2)}$	=	125	enumerated	=	125
K_6:	cayley	$n^{(n-2)}$	=	1296	enumerated	=	1296

The word **labelled** is doing critical work. Up to isomorphism there are only 6 trees on 6 vertices; there are 1296 trees *on the vertex set* $\{0, \dots, 5\}$. Chapter 5's distinction between equality and isomorphism is exactly the difference, and confusing the two makes Cayley's formula look absurd.

The Prüfer bijection

There are several proofs. This one is the best, because it does not merely count the trees — it *names* them.

Theorem (Prüfer). For $n \geq 2$ there is a bijection between labelled trees on $\{0, \dots, n-1\}$ and sequences of length $n - 2$ over $\{0, \dots, n-1\}$.

Since there are $n^{(n-2)}$ such sequences, Cayley's formula follows immediately.

Encoding. While more than two vertices remain, find the leaf with the smallest label, record its unique neighbour, and delete the leaf. Stop when two vertices remain.

```
def to_prufer(t):
    degree = [t.degree(v) for v in t.vertices()]
    neighbours = [set(t.neighbours(v)) for v in t.vertices()]
    seq = []
    for _ in range(t.n - 2):
        leaf = min(v for v in t.vertices() if degree[v] == 1)
        parent = next(iter(neighbours[leaf]))
        seq.append(parent)
        degree[leaf] = 0
        neighbours[parent].discard(leaf)
        degree[parent] -= 1
    return seq
```

Decoding. Given a sequence, compute each vertex's degree as one plus the number of times it appears. Then repeatedly take the smallest-labelled vertex of degree 1 that has not yet been used, join it to the next sequence entry, and decrement both degrees. Finally join the two remaining degree-1 vertices.

Proof that these are inverse. The key observation is that **a vertex appears in the sequence exactly $\deg(v) - 1$ times**. A leaf therefore never appears, and an internal vertex appears at least once. So the decoder can identify the leaf removed at each step — it is the smallest label not appearing in the remaining sequence and not yet consumed — which is exactly the leaf the encoder chose. By induction on n , each step of the decoder undoes the corresponding step of the encoder. Both maps are therefore well defined and mutually inverse, so each is a bijection. ■

That degree observation is the whole proof, and it is also the fastest way to read information out of a Prüfer sequence without decoding it:

```

from graphs.core import Graph, path
from graphs.generate import to_prufer, from_prufer

star = Graph(5, [(0, 1), (0, 2), (0, 3), (0, 4)])
print(to_prufer(star))           # [0, 0, 0]
print(to_prufer(path(5)))       # [1, 2, 3]
print(sorted(from_prufer([0, 0, 0]).edges()))
# [(0, 1), (0, 2), (0, 3), (0, 4)]

```

The star's centre has degree 4 and appears $4 - 1 = 3$ times. The path's two ends have degree 1 and never appear. You can read the degree sequence straight off.

A corollary that would be fiddly to prove directly falls out for free: **the number of labelled trees on n vertices with prescribed degrees $d_1, \dots, d_n$ is the multinomial coefficient $(n-2)! / \prod (d_i - 1)!$** — because that is exactly how many sequences have each vertex i appearing $d_i - 1$ times.

Counting for general graphs

Cayley's formula handles K_n . For an arbitrary graph, the answer is the **matrix–tree theorem**: the number of spanning trees equals any cofactor of the Laplacian matrix. It computes in $O(n^3)$ by a determinant, without enumerating anything.

That theorem needs the Laplacian, so it waits for Chapter 30. It is worth flagging now as the single largest gap between what this chapter can do and what is possible: `spanning_trees` in this book's library enumerates $C(m, n-1)$ subsets and tests each, which is why the Cayley check above stops at K_6 . The matrix–tree theorem gets K_6 's 1296 from a 5×5 determinant.

Try it

Watch the bijection round-trip on a random tree, and check the degree observation:

```

python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.generate import random_tree, to_prufer, from_prufer
rng = random.Random(11)
t = random_tree(8, rng)
seq = to_prufer(t)
print('prufer sequence:', seq)
print('degrees:         ', [t.degree(v) for v in t.vertices()])
print('appearances+1:   ', [seq.count(v) + 1 for v in t.vertices()])
print('round trips:     ', sorted(from_prufer(seq).edges()) == sorted(t.edges()))
"

```

```

prufer sequence: [7, 7, 7, 3, 2, 7]
degrees:         [1, 1, 2, 2, 1, 1, 1, 5]
appearances+1:   [1, 1, 2, 2, 1, 1, 1, 5]
round trips:     True

```

The second and third lines are identical, which is the lemma the whole proof turns on.

Exercises

1. How many labelled spanning trees does K_4 have? Check against Cayley's formula.
2. Compute the Prüfer sequence of the path $0-1-2-3$ by hand.
3. A vertex appears $\deg(v) - 1$ times in a Prüfer sequence. What does that say about the labels that never appear?
4. There are 1296 labelled trees on 6 vertices but only 6 up to isomorphism. Explain the gap in one sentence.

Solutions in Appendix E.

Takeaways

- Every connected graph has a spanning tree, by the extremal argument: take a minimal connected spanning subgraph and show it cannot contain a cycle.
- K_n has n^{n-2} **labelled** spanning trees. Up to isomorphism the count is far smaller, and conflating the two makes the formula look wrong.
- The Prüfer bijection proves Cayley's formula by naming each tree with a sequence. The lemma that carries it: vertex v appears exactly $\deg(v) - 1$ times.
- Prescribed-degree tree counts fall out of the same bijection as a multinomial coefficient, with no extra work.
- Counting spanning trees of a general graph is the matrix–tree theorem, $O(n^3)$ via a determinant. This chapter's enumerator is exponential and stops at K_6 .

Chapter 8 — Traversal

Breadth-first and depth-first search visit the same vertices in $O(n + m)$ time. They differ only in which vertex they take next — a queue versus a stack — and that single difference gives them entirely different uses.

The two orders

```
from graphs.core import Graph
from graphs.algorithms import bfs_order, dfs_order, distances

g = Graph(6, [(0, 1), (0, 2), (1, 3), (2, 3), (3, 4), (4, 5)])
print(bfs_order(g, 0))    # [0, 1, 2, 3, 4, 5]
print(dfs_order(g, 0))    # [0, 1, 3, 2, 4, 5]
print(distances(g, 0))    # {0: 0, 1: 1, 2: 1, 3: 2, 4: 3, 5: 4}
```

BFS visits 2 before 3, because 2 is closer to the source. DFS visits 3 before 2, because it follows the first path as far as it goes before backing up. Both take $O(n + m)$: every vertex enters the container once, and every edge is examined twice — once from each endpoint.

The only structural difference in the code is the container:

```
queue.popleft()    # BFS: first in, first out
stack.pop()        # DFS: last in, first out
```

Note that `dfs_order` here is **iterative**. The recursive version is shorter and is what most texts print, but Python's default recursion limit is 1000, so a path graph on ten thousand vertices crashes it. The iterative version has no such limit, and this book uses it throughout.

Why BFS finds shortest paths

BFS's correctness is the one thing in this chapter that genuinely needs proving.

Theorem. BFS from s assigns each reachable v the value $d(s, v)$, the length of a shortest path.

Proof. Let $\text{dist}[v]$ be the value BFS assigns. We show $\text{dist}[v] = d(s, v)$ by induction on $d(s, v)$.

First, $\text{dist}[v] \geq d(s, v)$ always, because BFS only ever sets $\text{dist}[w] = \text{dist}[v] + 1$ when vw is an edge, so the assigned values trace out an actual walk from s .

For the other direction, suppose $d(s, v) = k$ and every vertex at true distance $< k$ has been correctly labelled. Take a shortest $s-v$ path and let u be the vertex before v on it, so $d(s, u) = k-1$ and by induction $\text{dist}[u] = k-1$. The queue is processed in non-decreasing order of dist — this is the invariant that makes BFS work, and it holds because we only ever append values one greater than the value being processed. So u is dequeued before any vertex of distance k or more is processed, and at that moment v is either already labelled with something $\leq k$, or gets labelled k . Either way $\text{dist}[v] \leq k$. ■

The invariant — **the queue holds at most two distinct distance values at any time, and they are consecutive** — is the thing to remember. It fails the moment edges have different weights, which is precisely why Chapter 10 needs Dijkstra and a priority queue instead of a plain queue.

The harness checks the theorem against paths enumerated exhaustively rather than against a second shortest-path routine:

```
held      ch 8  BFS distances equal true shortest-path lengths  (52 graphs)
held      ch 8  DFS and BFS reach exactly the same vertices  (52 graphs)
```

Search trees, and the edges they leave behind

Both searches build a **search tree**: for each vertex other than the source, remember the edge along which it was first discovered. Since each vertex is discovered exactly once, this is $n - 1$ edges on a connected graph, and it is connected, so it is a spanning tree — which is Chapter 7's existence theorem made constructive.

Every edge of the graph is then either a **tree edge** or not. Classifying the ones that are not is where the two searches diverge most sharply, and it is the foundation of Chapter 12.

For **DFS on an undirected graph**, every non-tree edge is a **back edge**: it joins a vertex to one of its own ancestors in the tree.

Proof. Consider an edge uv and suppose u is discovered first. DFS does not return from u until every vertex reachable through unvisited vertices from u has been finished — in particular, v is discovered during the exploration of u , so v is a descendant of u . Hence uv joins a vertex to an ancestor. ■

There are no cross edges in undirected DFS. That is a strong statement, and it is what makes DFS the right tool for finding bridges, cut vertices, and biconnected components: the only way to get back "above" your current position is a back edge, so tracking the highest ancestor reachable from each subtree tells you exactly which tree edges are bridges. Chapter 12 builds that algorithm.

For **BFS on an undirected graph**, the corresponding fact is about levels: every edge joins vertices whose distances from the source differ by **at most one**. An edge to a vertex two levels down would have shortened the path. So each non-tree edge is either within a level or between consecutive levels — and an edge *within* a level is exactly an odd cycle waiting to be found, which is Chapter 16's two-colouring algorithm.

Which one to use

Problem	Search	Why
Shortest paths, unweighted	BFS	The level invariant is exactly the distance
Connected components	either	Both reach the same set
Bipartiteness / odd cycle	BFS	An intra-level edge names the odd cycle
Bridges, cut vertices	DFS	No cross edges, so back edges tell the whole story
Topological order (directed)	DFS	Finishing order reversed
Cycle detection	either	A back edge in DFS; an intra- or back-level edge in BFS

The one case where the choice is a mistake rather than a preference is shortest paths. DFS finds *a* path, and people do use it for that, but the path it finds can be arbitrarily longer than the shortest — on a graph with a long detour available, DFS will happily take it.

Try it

Watch DFS return a path far longer than the shortest one, on a graph designed to mislead it:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.algorithms import bfs_order, dfs_order, distances
# 0 and 7 are adjacent; there is also a long way round through 1..6
g = Graph(8, [(0,7)] + [(i, i+1) for i in range(7)])
print('bfs order:', bfs_order(g, 0))
print('dfs order:', dfs_order(g, 0))
print('true distance 0 to 7:', distances(g, 0)[7])
print('position of 7 in dfs:', dfs_order(g, 0).index(7))
"
```

```
bfs order: [0, 1, 7, 2, 6, 3, 5, 4]
dfs order: [0, 1, 2, 3, 4, 5, 6, 7]
true distance 0 to 7: 1
position of 7 in dfs: 7
```

BFS reaches vertex 7 second, because it is one step away. DFS reaches it last, having walked the entire seven-edge detour first. Both are correct traversals; only one of them knows anything about distance.

Exercises

1. You need shortest paths in an unweighted graph. BFS or DFS, and why is the other one wrong rather than merely slower?
2. What is the running time of both searches, and why is it the same?
3. In a BFS, you find an edge joining two vertices in the same level. What have you found?
4. Why does this book write DFS iteratively rather than recursively?

Solutions in Appendix E.

Takeaways

- BFS and DFS differ only by queue versus stack, and both run in $O(n + m)$.
- BFS computes exact shortest-path distances on unweighted graphs. The invariant is that the queue holds at most two consecutive distance values — and that is exactly what unequal edge weights destroy, which is why Chapter 10 exists.
- Both build spanning trees, making Chapter 7's existence proof constructive.
- Undirected DFS has **no cross edges**: every non-tree edge is a back edge. That is what makes it the tool for bridges and cut vertices.

- Undirected BFS puts every edge within a level or between consecutive levels. An intra-level edge is an odd cycle.
- Write DFS iteratively. The recursive form dies at Python's 1000-frame limit on a graph that is merely long.

Chapter 9 — Minimum spanning trees

Give the edges weights and ask for the cheapest spanning tree. Two greedy algorithms solve it, and greedy algorithms almost never work — so the interesting content is the single property that makes both correct.

The cut property

A **cut** is a partition of the vertices into two non-empty parts $(S, V \setminus S)$. An edge **crosses** the cut if it has one endpoint in each part.

Theorem (cut property). Let $(S, V \setminus S)$ be any cut, and let e be a crossing edge of strictly minimum weight among crossing edges. Then e belongs to every minimum spanning tree.

Proof. Let T be a spanning tree not containing $e = uv$. Adding e to T creates exactly one cycle (Chapter 6's exchange fact). That cycle crosses the cut at e , and a cycle crosses any cut an even number of times, so it crosses at some other edge $f \neq e$. Now $T + e - f$ is again a spanning tree, and since $w(e) < w(f)$ by the strict minimality of e , it is strictly lighter than T . So T was not minimum. ■

Two things in that proof carry their weight. The **even-crossing** fact — walk around a cycle and every time you leave S you must come back — is used again in Chapters 13 and 17. And the exchange fact from Chapter 6 is what makes $T + e - f$ a tree, which is the whole reason Chapter 6 bothered to prove it.

The word **strictly** is load-bearing, and dropping it is the standard error. If two crossing edges tie for minimum, neither need be in every MST; each is in *some* MST. The theorem as stated is exactly right, and the version without "strictly" is false.

There is a companion, proved the same way:

Theorem (cycle property). If e is the unique heaviest edge of some cycle, then e belongs to no minimum spanning tree.

Kruskal

Sort the edges by weight and add each one unless it would close a cycle.

```
def kruskal(g):
    uf = UnionFind(g.n)
    chosen = []
    for u, v, _ in sorted(g.edges(), key=lambda e: e[2]):
        if uf.union(u, v):
            chosen.append((u, v))
    return chosen
```

Correctness. When Kruskal accepts an edge $e = uv$, let S be the set of vertices already connected to u by chosen edges. Every edge crossing $(S, V \setminus S)$ that was examined earlier was rejected, and edges are examined in weight order, so e is a minimum-weight crossing edge. The cut property applies. ■

The running time is $O(m \log m)$ and is entirely the sort; the union–find operations are effectively constant. Note that on a **disconnected** graph, Kruskal returns a minimum spanning *forest* — one tree per component — rather than failing. That is usually what you want, and it is worth knowing that is what you get.

Union–find is the supporting structure: `find` returns a set's representative, `union` merges two sets and reports whether they were already the same. With path compression and union by size, m operations cost $O(m \alpha(n))$, where α is the inverse Ackermann function — at most 4 for any n that fits in the observable universe. This book treats it as constant while saying plainly that it is not.

Prim

Grow a single tree from a starting vertex, always taking the cheapest edge that leaves it.

```
def prim(g, source=0):
    seen = {source}
    frontier = [(g.weight(source, x), source, x) for x in g.neighbours(source)]
    heapq.heapify(frontier)
    chosen = []
    while frontier:
        _, u, v = heapq.heappop(frontier)
        if v in seen:
            continue
        seen.add(v)
        chosen.append((min(u, v), max(u, v)))
        for x in g.neighbours(v):
            if x not in seen:
                heapq.heappush(frontier, (g.weight(v, x), v, x))
    return chosen
```

Correctness. At every step, `seen` is one side of a cut and the algorithm takes a minimum-weight crossing edge. The cut property applies directly. ■

$O(m \log n)$ with a binary heap. Unlike Kruskal, Prim only spans the source's component — it has no way to jump to a different one.

The two algorithms make genuinely different choices about what to be greedy over: Kruskal is greedy over *all* edges globally, Prim over the edges leaving one growing tree. That they both work is the cut property being applied to two different families of cuts.

The MST is not unique

Here is the misconception the chapter exists to kill. "The" minimum spanning tree suggests there is one. There is exactly one minimum **weight**; there can be many trees achieving it.

The harness registers this as a theorem expected to be refuted:

refuted	ch 9	Kruskal and Prim always choose the same edges (5 graphs)
held	ch 9	Kruskal and Prim both achieve the true minimum weight (91 graphs)

The second line is the theorem; the first is the misconception, and it is refuted rather than proved.

Getting that check to fire took a correction worth recording. On the first attempt the weights were drawn from 1..20, and the two algorithms agreed on **all 79 graphs** — the claim held, and looked like a theorem. With weights drawn from 1..2, so that ties are common, they disagreed on **66 of 3000**. Ties are the entire mechanism, and a random family without them cannot see the phenomenon at all. If the weights are all distinct, the MST genuinely *is* unique — a consequence of the cut property with its strictness intact.

```
from graphs.weighted import WeightedGraph
from graphs.mst import kruskal, prim, brute_force_mst, spanning_trees

wg = WeightedGraph(5, [(0,1,1), (0,2,3), (1,2,2), (1,3,6), (2,3,4), (3,4,5), (2,4,7)])
print(sorted(kruskal(wg)))           # [(0, 1), (1, 2), (2, 3), (3, 4)]
print(brute_force_mst(wg)[1])        # 12
print(len(spanning_trees(wg.graph))) # 21
```

Twenty-one spanning trees, one of weight 12, and both algorithms find it.

Verification without circularity

Checking Kruskal against Prim would establish only that two of my greedy implementations agree — which they might do while both being wrong, since they share the same cut-property reasoning and the same author. So the harness enumerates **every** spanning tree and takes the cheapest:

```
def brute_force_mst(g):
    best = None
    for tree in spanning_trees(g.graph):
        w = g.subgraph_weight(tree)
        if best is None or w < best[1]:
            best = (tree, w)
    return best
```

$C(m, n-1)$ subsets, so it is capped at seven vertices. That is enough: an error in a greedy algorithm shows up on small graphs, because the greedy choice is wrong locally or not at all.

Try it

Watch the two algorithms disagree about the tree while agreeing about the weight:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.weighted import WeightedGraph
from graphs.mst import kruskal, prim
# C_4 with every weight 1, so every spanning tree is minimum
g = WeightedGraph(4, [(0,2,1),(0,3,1),(1,2,1),(1,3,1)])
k = sorted(kruskal(g))
p = sorted((min(u,v), max(u,v)) for u,v in prim(g, source=1))
print('kruskal:', k, 'weight', g.subgraph_weight(k))
print('prim:    ', p, 'weight', g.subgraph_weight(p))
print('same weight:', g.subgraph_weight(k) == g.subgraph_weight(p), ' same edges:', k == p)
"
```

```
kruskal: [(0, 2), (0, 3), (1, 2)] weight 3
prim:    [(0, 2), (1, 2), (1, 3)] weight 3
same weight: True    same edges: False
```

Different trees, identical cost. Both are minimum spanning trees; neither is *the* minimum spanning tree.

Four vertices is the smallest this can happen on, found by exhaustive search rather than by guessing — and the first two graphs I guessed both had the two algorithms agreeing.

Exercises

1. State the cut property, and say which word in it is load-bearing.
2. Give a graph with two different minimum spanning trees of equal weight.
3. What does Kruskal return when the input graph is disconnected?
4. Under what condition is the minimum spanning tree unique?

Solutions in Appendix E.

Takeaways

- The cut property is the whole subject: the strictly-lightest edge across any cut is in every MST. Both algorithms are corollaries.
- "Strictly" is not decoration. Without it the theorem is false, and with ties the MST stops being unique.
- Kruskal is greedy globally and returns a forest on disconnected input; Prim is greedy from one growing tree and spans only the source's component.
- Union–find is $O(\alpha(n))$ per operation, which is not constant and is at most 4.
- Distinct weights $\implies$ unique MST. Tied weights $\implies$ many. A random test family with widely spread weights will never show you this: 0 disagreements in 79 graphs at weights `1..20`, 66 in 3000 at weights `1..2`.

Chapter 10 — Shortest paths

Give the edges weights and BFS stops working. This chapter is about why, what replaces it, and what a negative weight does to the whole question.

Directions, and why they arrive now

Everything so far has been undirected. From here the book needs **arcs**, and the reason is specific: a negative weight on an undirected edge is meaningless. You could cross it, cross back, and cross again, dropping the total without bound. Negative weights only make sense when the edge goes one way, so the moment the chapter admits them it must admit direction.

```
from graphs.digraph import Digraph

d = Digraph(5, [(0,1,4), (0,2,1), (2,1,2), (1,3,1), (2,3,5), (3,4,3)])
```

An undirected weighted graph is the special case where every arc has a twin. `Digraph.of(g)` builds it, and every result below then applies unchanged.

Dijkstra

Keep a set of **settled** vertices whose distance is final. Repeatedly take the unsettled vertex of smallest tentative distance, settle it, and relax its outgoing arcs.

```
def dijkstra(d, source):
    dist = {source: 0.0}
    heap = [(0.0, source)]
    settled = set()
    while heap:
        du, u = heapq.heappop(heap)
        if u in settled:
            continue
        settled.add(u)
        for v in d.successors(u):
            alt = du + d.weight(u, v)
            if alt < dist.get(v, INF):
                dist[v] = alt
                heapq.heappush(heap, (alt, v))
    return dist
```

```
print(dijkstra(d, 0))    # {0: 0.0, 1: 3.0, 2: 1.0, 3: 4.0, 4: 7.0}
```

The route to **1** costs 3, not the 4 of the direct arc, because going via **2** costs **1 + 2**.

Theorem. With non-negative weights, Dijkstra settles every reachable vertex with its true shortest distance.

Proof. By induction on the order of settling. Suppose every previously settled vertex has its true distance, and let **u** be the vertex now being settled with tentative value **du**. Any path from **s** to **u** must at some point leave

the settled set: let xy be the first arc doing so, with x settled. Then that path costs at least $\text{dist}[x] + w(x, y)$, which is at least the tentative value of y , which is at least du since u was chosen as the smallest. **The final inequality needs every remaining arc weight to be non-negative** — the rest of the path from y to u can only add. So no path beats du . ■

The hypothesis is used in exactly one place, and that is the place it fails.

What a negative arc actually does

The usual statement is "Dijkstra fails on negative weights", which is true but not informative — and the naive explanation, "once a vertex is settled it is never improved", is not quite what happens in this implementation. Watch:

```
d = Digraph(4, [(0,1,-1), (0,2,-1), (1,3,-1), (2,1,-1)])
print(dijkstra(d, 0))      # {0: 0.0, 1: -2.0, 2: -1.0, 3: -2.0}
print(bellman_ford(d, 0)[0]) # {0: 0.0, 1: -2.0, 2: -1.0, 3: -3.0}
```

Look at vertex 1. Dijkstra reports -2, which is **correct** — the improvement via 2 was found and recorded. It is vertex 3 that is wrong: -2 instead of -3.

That is the real failure mode. Vertex 1 was settled at -1, and its arcs were relaxed then. When the better value -2 arrived later, $\text{dist}[1]$ was updated, but 1 was already in `settled`, so popping it again did nothing and **the improvement never propagated onward to 3**. The bug is not that settled vertices cannot improve; it is that their improvement cannot travel.

This is the smallest graph on which it happens — four vertices and four arcs, all of weight -1, found by exhaustive search rather than by construction. The harness registers it as a theorem expected to be refuted:

held	ch10	Dijkstra is correct when weights are non-negative	(120 graphs)
refuted	ch10	Dijkstra is correct when weights may be negative	(3 graphs)

Note that graphs *containing* a negative cycle are excluded from that second family. The claim is refuted on graphs where a correct answer exists and Dijkstra does not find it, which is a sharper statement than "it breaks when the problem is ill-posed".

Bellman–Ford

Give up on settling anything. Just relax every arc, $n - 1$ times.

```
def bellman_ford(d, source):
    dist = {v: INF for v in d.vertices()}
    dist[source] = 0.0
    for _ in range(d.n - 1):
        for u, v, w in d.arcs():
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in d.arcs():
        if dist[u] + w < dist[v]:
            return dist, True          # a negative cycle is reachable
    return dist, False
```

Correctness. After k rounds, $\text{dist}[v]$ is at most the weight of the lightest path from s to v using at most k arcs. This is immediate by induction: round k relaxes the last arc of any such path, whose prefix was handled by round $k-1$. A shortest path with no negative cycle is simple, so uses at most $n-1$ arcs, and $n-1$ rounds suffice. ■

$O(nm)$, which is much worse than Dijkstra's $O(m \log n)$ — the price of handling negative arcs.

Negative cycles

If a negative cycle is reachable, there is no shortest path: go round the cycle again and the total drops. The question becomes ill-posed, not merely hard.

The final loop of Bellman–Ford is exactly the test. After $n-1$ rounds no further improvement is possible unless a path can use more than $n-1$ arcs profitably, which requires a negative cycle.

```
neg = Digraph(3, [(0,1,1), (1,2,-3), (2,0,1)])
print(bellman_ford(neg, 0)[1])    # True
```

The harness checks this against an independent enumeration of every simple cycle, rather than against Floyd–Warshall, which shares this book's arithmetic:

```
held      ch10  Bellman-Ford flags exactly the reachable negative cycles (300
graphs)
```

Reachable is part of the statement. A negative cycle in a far corner of the graph does not make the distances from s ill-defined, and Bellman–Ford from s correctly ignores it. Chapter 11's Floyd–Warshall sees the whole graph and so answers a slightly different question.

One consequence worth stating, because it explains why people care: with negative arcs allowed, finding the *shortest simple path* is NP-hard — a Hamiltonian path is a shortest simple path on the right weights (Chapter 20). Bellman–Ford does not solve that. It solves shortest *walk*, which coincides with shortest simple path precisely when no negative cycle is reachable.

Choosing

Situation	Use	Cost
Unweighted	BFS	$O(n + m)$
Non-negative weights	Dijkstra	$O(m \log n)$
Any weights, one source	Bellman–Ford	$O(nm)$
Any weights, all pairs	Floyd–Warshall (Ch 11)	$O(n^3)$
Negative cycle reachable	none — the question is ill-posed	—

BFS is Dijkstra with all weights equal to 1, and the priority queue collapses to a plain queue because the invariant from Chapter 8 — at most two consecutive distance values in flight — holds again.

Try it

Watch the improvement fail to propagate, one vertex at a time:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.digraph import Digraph
from graphs.paths import dijkstra, bellman_ford, brute_force_shortest
d = Digraph(4, [(0,1,-1), (0,2,-1), (1,3,-1), (2,1,-1)])
dj = dijkstra(d, 0)
bf = bellman_ford(d, 0)[0]
for t in range(4):
    print(f' to {t}: dijkstra={dj[t]:>5} bellman={bf[t]:>5}
true={brute_force_shortest(d,0,t):>5}')
"
```

```
to 0: dijkstra=  0.0  bellman=  0.0  true=  0.0
to 1: dijkstra= -2.0  bellman= -2.0  true=  -2
to 2: dijkstra= -1.0  bellman= -1.0  true=  -1
to 3: dijkstra= -2.0  bellman= -3.0  true=  -3
```

Three of the four are right. Only the vertex *downstream* of the late improvement is wrong, which is why this failure is easy to miss on a graph you have not checked exhaustively.

Exercises

1. Dijkstra's proof uses non-negativity in exactly one inequality. Which one?
2. What is Bellman–Ford's running time, and why is it worse than Dijkstra's?
3. What does the n -th relaxation round of Bellman–Ford detect?
4. A negative cycle is reachable from s . What is the shortest walk from s to a vertex on that cycle?

Solutions in Appendix E.

Takeaways

- Negative weights force direction, which is why the book turns to digraphs here.
- Dijkstra's proof uses non-negativity in exactly one inequality, and that is exactly where it breaks.
- Dijkstra's real failure is not that settled vertices never improve — they can — but that the improvement cannot propagate past them. The minimal witness has four vertices.
- Bellman–Ford relaxes everything $n - 1$ times, handles negative arcs in $O(nm)$, and its n -th round is precisely a reachable-negative-cycle test.
- With a reachable negative cycle there is no shortest walk. Shortest *simple* path with negative weights is NP-hard, and no algorithm here attempts it.

Chapter 11 — All-pairs distance

Running Dijkstra from every vertex costs $O(nm \log n)$. On a dense graph there is something simpler and, remarkably, faster — and it is four lines.

Floyd–Warshall

The idea is a different induction from every algorithm so far. Instead of growing outward from a source, restrict which vertices a path may pass **through**.

Let $d_k(u, v)$ be the length of the shortest $u-v$ path whose interior vertices all lie in $\{0, \dots, k-1\}$. Then d_0 is just the arc weights, and

$$d_{k+1}(u, v) = \min(d_k(u, v), d_k(u, k) + d_k(k, v))$$

because a path allowed to use k either does not — the first term — or does, in which case it passes through k exactly once and splits into two paths that do not.

That "exactly once" needs justification: a shortest path visits no vertex twice, provided there is no negative cycle. So the recurrence is valid under the same hypothesis Bellman–Ford needs, and fails in the same way without it.

```
def floyd_warshall(d):
    n = d.n
    dist = [[INF] * n for _ in range(n)]
    for v in range(n):
        dist[v][v] = 0.0
    for u, v, w in d.arcs():
        dist[u][v] = min(dist[u][v], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

$O(n^3)$, three nested loops, no priority queue and no data structure at all. **The k loop must be outermost.** That is the single thing to get right: k indexes the induction, and i, j iterate within a fixed stage. Swap the loops and you get an algorithm that sometimes returns correct answers, which is the worst kind of wrong.

```
d = Digraph(5, [(0,1,4), (0,2,1), (2,1,2), (1,3,1), (2,3,5), (3,4,3)])
for row in floyd_warshall(d):
    print(row)
```

```
0 [0, 3, 1, 4, 7]
1 [inf, 0, inf, 1, 4]
2 [inf, 2, 0, 3, 6]
3 [inf, inf, inf, 0, 3]
4 [inf, inf, inf, inf, 0]
```

Row 0 matches Chapter 10's Dijkstra run exactly. The infinities are honest: this digraph has no arcs back towards 0, so nothing reaches it.

Negative weights, and a different question

Floyd–Warshall handles negative arcs with no modification, which Dijkstra cannot. And it detects negative cycles differently from Bellman–Ford, in a way worth being precise about:

```
neg = Digraph(3, [(0,1,1), (1,2,-3), (2,0,1)])
print([floyd_warshall(neg)[v][v] for v in range(3)]) # [-1, -1, -2]
```

A **negative diagonal entry** means v lies on a negative cycle. Bellman–Ford from a source s reports whether a negative cycle is *reachable from* s ; Floyd–Warshall reports which vertices are *on* one. Those are different questions, and both are useful:

- Bellman–Ford's version tells you your distances from s are meaningless.
- Floyd–Warshall's version tells you which part of the graph is the problem.

The -2 at vertex 2 is not a distance. Once a negative cycle exists, the numbers in the matrix are whatever the fixed number of relaxation rounds happened to produce, and only their sign is meaningful.

The metric

For an undirected graph with non-negative weights, d is a genuine **metric**: non-negative, zero exactly on the diagonal, symmetric, and satisfying the triangle inequality. The harness checks the last of these directly:

```
held      ch11 Graph distance satisfies the triangle inequality (120 graphs)
held      ch11 Floyd-Warshall agrees with enumerating every simple path (120
graphs)
```

Symmetry is the one that fails for digraphs, and it fails badly — $d(u,v)$ can be finite while $d(v,u)$ is infinite, as row 1 of the matrix above shows. A directed graph gives a **quasimetric**, and any geometric intuition you carry over from metric spaces needs checking against that.

From the all-pairs matrix, three standard quantities:

- the **eccentricity** of v is $\max_u d(v,u)$;
- the **diameter** is the maximum eccentricity;
- the **radius** is the minimum eccentricity, and a vertex achieving it is a **centre**.

$\text{radius} \leq \text{diameter} \leq 2 \cdot \text{radius}$. The right inequality is the triangle inequality applied through a centre c : $d(u,v) \leq d(u,c) + d(c,v) \leq 2 \cdot \text{radius}$. Both bounds are achieved — by a cycle and by a path respectively — so neither can be improved.

Which algorithm

	time	negative arcs	best when
BFS from each vertex	$O(nm)$	no (unweighted)	unweighted
Dijkstra from each vertex	$O(nm \log n)$	no	sparse, $m \ll n^2$
Floyd–Warshall	$O(n^3)$	yes	dense, or negative arcs
Johnson's	$O(nm \log n)$	yes	sparse and negative arcs

The crossover is at $m \approx n^2 / \log n$. Below it, repeated Dijkstra wins; above it, Floyd–Warshall does, and it also wins on constant factors by a wide margin, being three loops over a flat array with no allocation.

Johnson's algorithm is the one gap this book leaves open in shortest paths. It reweights the graph using a Bellman–Ford potential so that all weights become non-negative and shortest paths are preserved, then runs Dijkstra n times. It is the right answer for sparse graphs with negative arcs, and it is a genuinely clever trick rather than a routine combination — the potential function is $h(v) = d(s, v)$ from a new vertex joined to everything at weight 0, and the reweighted arc $w'(u, v) = w(u, v) + h(u) - h(v)$ is non-negative exactly because h satisfies the triangle inequality.

Try it

Break the loop order deliberately and see how often it shows:

```
python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.digraph import Digraph, INF, random_digraph
from graphs.paths import floyd_warshall, brute_force_shortest

def wrong(d):
    n = d.n
    dist = [[INF]*n for _ in range(n)]
    for v in range(n): dist[v][v] = 0.0
    for u, v, w in d.arcs(): dist[u][v] = min(dist[u][v], w)
    for i in range(n):          # i outermost -- the classic mistake
        for j in range(n):
            for k in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

# a plain path: the broken version gets this right
d = Digraph(4, [(0,1,1), (1,2,1), (2,3,1)])
print('path graph, 0->3:  correct', floyd_warshall(d)[0][3], ' broken', wrong(d)[0][3])

# and one where it does not
d = Digraph(4, [(0,3,2), (2,1,1), (3,0,1), (3,2,4)])
print('witness,    0->1:  correct', floyd_warshall(d)[0][1], ' broken', wrong(d)[0][1],
      ' true', brute_force_shortest(d, 0, 1))

rng = random.Random(1); bad = 0
for _ in range(4000):
    g = random_digraph(rng.randint(3, 6), 0.4, rng)
    if any(floyd_warshall(g)[i][j] != wrong(g)[i][j] for i in g.vertices() for j in g.vertices()):
        bad += 1
print(f'broken version differs on {bad}/4000 random digraphs')
"
```

```
path graph, 0->3:  correct 3  broken 3
witness,    0->1:  correct 7  broken inf
broken version differs on 942/4000 random digraphs
```

This is the dangerous kind of bug, and the numbers say why. On a path graph the broken version is right. On a randomly chosen digraph it is wrong about **a quarter of the time**. An implementation that failed on every input would be caught by the first test anyone wrote; one that is right on the simple cases and wrong on 24% of the rest will pass a hand-checked example and ship.

The witness shows the mechanism. The true route $0 \rightarrow 3 \rightarrow 2 \rightarrow 1$ costs $2 + 4 + 1 = 7$. With i outermost, the entry `dist[0][1]` is finalised while `dist[3][1]` is still infinite, because row 3 has not been processed yet — and nothing ever revisits row 0. The correct order finalises *all* paths through vertex k before moving to $k+1$, so no row can be left behind.

Exercises

1. What is Floyd–Warshall's running time, and what data structure does it need?

2. Why must the k loop be outermost?
3. What does a negative entry on the diagonal of the output mean?
4. Give a two-vertex digraph where $d(u, v)$ is finite and $d(v, u)$ is infinite.

Solutions in Appendix E.

Takeaways

- Floyd–Warshall inducts on *which vertices a path may pass through*, not on distance from a source. That is why it does all pairs at once.
- k must be the outermost loop. It indexes the induction; i and j do not.
- $O(n^3)$, negative arcs allowed, no data structures. On dense graphs it beats n runs of Dijkstra on both asymptotics and constants.
- A negative diagonal entry means that vertex lies on a negative cycle — a different question from Bellman–Ford's "is one reachable from s ", and both are worth asking.
- Undirected distance is a metric; directed distance is only a quasimetric, and asymmetry is not an edge case.

Chapter 12 — Connectivity and Menger's theorem

How hard is it to disconnect a graph? There are two answers — count the vertices you must delete, or count the disjoint paths you must destroy — and Menger's theorem says they are the same number. It is the first **min–max theorem** in this book, and the pattern it establishes runs through the next two chapters and most of combinatorial optimisation.

Two measures

A set $S \subseteq V \setminus \{s, t\}$ is an s – t **vertex cut** if deleting it leaves no s – t path. A set of edges is an s – t **edge cut** if deleting it does the same.

Two paths are **internally disjoint** if they share no vertex except s and t , and **edge-disjoint** if they share no edge.

One direction is free, and it is worth seeing why it is free:

Lemma (weak duality). The number of pairwise internally-disjoint s – t paths is at most the size of any s – t vertex cut.

Proof. Every s – t path must contain a vertex of the cut, or it would survive the deletion. Internally disjoint paths cannot share such a vertex. So the cut has at least one vertex per path. ■

That argument gives $\max \leq \min$ for free and tells you nothing about whether the bound is tight. **The content of every min–max theorem is the other direction** — and there is no general reason to expect it. Plenty of natural pairs of quantities satisfy weak duality with a genuine gap; Chapter 15's $\omega(G) \leq \chi(G)$ is one, and Chapter 19 is about the graphs where it closes.

The theorem

Theorem (Menger, 1927). For non-adjacent s and t , the maximum number of pairwise internally-disjoint s – t paths equals the minimum size of an s – t vertex cut.

The edge version holds for all $s \neq t$: the maximum number of edge-disjoint s – t paths equals the minimum size of an s – t edge cut.

The hypothesis **non-adjacent** in the vertex form is not fussiness. If s and t are adjacent, no set of *other* vertices separates them, so the minimum cut does not exist — the quantity on the right is undefined rather than large. This book's `vertex_connectivity` returns infinity in that case, which is a statement about the definition rather than a computational convenience.

Proof (edge form), by induction on the number of edges. Let k be the minimum edge cut size. Weak duality gives at most k disjoint paths, so we need k of them.

Pick a minimum cut F with $|F| = k$ and take any edge $e \in F$. In $G - e$, the minimum s – t cut has size $k - 1$: it is at most that, since $F - e$ works, and at least that, since adding e back to any smaller cut would beat F . By induction $G - e$ has $k - 1$ edge-disjoint paths, and they avoid e . Adding e back, we need one more

path avoiding all of them — which exists precisely because `e` was in every minimum cut. The full argument requires care about the case where contracting rather than deleting is needed, and the clean modern route is the one below. ■

The clean route: it is a flow problem

Rather than finish that induction, observe that Menger is **max-flow min-cut with every capacity set to 1**, and Chapter 13 proves that. This is not a dodge; it is the right organisation, because one algorithm then yields three theorems.

For the **edge** version, replace each edge by two unit-capacity arcs:

```
def edge_connectivity(g, s, t):
    net = FlowNetwork(g.n)
    for u, v in g.edges():
        net.add_arc(u, v, 1.0)
        net.add_arc(v, u, 1.0)
    return net.max_flow(s, t)[0]
```

A flow of value `k` decomposes into `k` unit paths, and unit capacities force them to be edge-disjoint. A cut of capacity `k` is a set of `k` edges.

For the **vertex** version, the trick is **vertex splitting**: replace each vertex `v` by `v_in` → `v_out` carrying capacity 1, and route every edge into `v_in` and out of `v_out`.

```
def vertex_connectivity(g, s, t):
    net = FlowNetwork(2 * g.n)
    for v in g.vertices():
        net.add_arc(2*v, 2*v + 1, INF if v in (s, t) else 1.0)
    for u, v in g.edges():
        net.add_arc(2*u + 1, 2*v, INF)
        net.add_arc(2*v + 1, 2*u, INF)
    return net.max_flow(2*s + 1, 2*t)[0]
```

The internal arc caps how much flow passes *through* `v` at 1, so paths cannot share a vertex. The edges get infinite capacity because the theorem counts vertices, not edges, and `s` and `t` get infinite capacity because deleting them is not allowed.

Vertex splitting is a general technique, not a one-off: any time a constraint is on vertices rather than edges, split the vertex and put the constraint on the internal arc.

Both are checked against exhaustive deletion — a completely different computation:

```
held      ch12 Menger (edge form): max edge-disjoint s-t paths = min s-t edge
cut (49 graphs)
held      ch12 Menger (vertex form): max internally-disjoint paths = min s-t
vertex cut (51 graphs)
```

Global connectivity

The **vertex connectivity** $\kappa(G)$ is the fewest vertices whose deletion disconnects G or reduces it to a single vertex; the **edge connectivity** $\lambda(G)$ is the edge analogue. A graph is **k-connected** if $\kappa(G) \geq k$.

Theorem (Whitney). $\kappa(G) \leq \lambda(G) \leq \delta(G)$.

Proof. The right inequality: deleting all edges at a minimum-degree vertex isolates it. The left: given a minimum edge cut, pick one endpoint of each cut edge on the source side. Deleting those vertices destroys every $s-t$ path, so $\kappa \leq \lambda$. ■

Both inequalities can be strict, and the Petersen graph is not the witness this time — it has $\kappa = \lambda = \delta = 3$. Take instead two triangles joined by a single long path: $\delta = 2$, $\lambda = 1$, $\kappa = 1$.

A useful reformulation, which is Menger applied to every pair at once:

Corollary. G is k -connected if and only if every pair of vertices is joined by k internally-disjoint paths.

That is the form to remember, because it converts a statement about *destroying* the graph into one about *routes through it* — which is why k -connectivity is the right notion of robustness for a network.

Try it

Confirm both forms on the Petersen graph, whose connectivity is 3 in every sense:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import petersen, cycle
from graphs.flow import edge_connectivity, vertex_connectivity,
brute_force_edge_cut, brute_force_vertex_cut

p = petersen()
print('petersen, vertices 0 and 2 (non-adjacent):')
print('  edge-disjoint paths ', edge_connectivity(p, 0, 2), ' exhaustive edge
cut ', brute_force_edge_cut(p, 0, 2))
print('  internally-disjoint ', vertex_connectivity(p, 0, 2), ' exhaustive vertex
cut', brute_force_vertex_cut(p, 0, 2))
c = cycle(6)
print('C_6, opposite vertices:')
print('  edge-disjoint paths ', edge_connectivity(c, 0, 3), ' exhaustive edge
cut ', brute_force_edge_cut(c, 0, 3))
"
```

```
petersen, vertices 0 and 2 (non-adjacent):
  edge-disjoint paths  3.0  exhaustive edge cut  3.0
  internally-disjoint  3.0  exhaustive vertex cut 3.0
C_6, opposite vertices:
  edge-disjoint paths  2.0  exhaustive edge cut  2.0
```

A cycle gives 2 both ways, which is the smallest interesting case: two routes round, and you must cut two edges to stop them.

Exercises

1. State weak duality for vertex cuts, and explain why it proves nothing on its own.
2. Why does the vertex form of Menger's theorem require s and t to be non-adjacent?
3. Give a graph where $\kappa(G) < \lambda(G) < \delta(G)$ — or explain why one of those inequalities cannot be strict.
4. In the vertex-splitting construction, why is the internal arc given capacity 1 and the edge arcs infinity?

Solutions in Appendix E.

Takeaways

- Menger is a min–max theorem: disjoint paths versus cut size, in both a vertex and an edge form.
- Weak duality ($\max \leq \min$) is free and proves nothing. The theorem is the other inequality, and there is no general reason such a bound should be tight.
- The vertex form needs s and t non-adjacent, because otherwise no cut exists at all.
- Both forms are max-flow min-cut with unit capacities. Vertex splitting — $v_{in} \rightarrow v_{out}$ with capacity 1 — is the general way to move a constraint from edges onto vertices.
- $\kappa(G) \leq \lambda(G) \leq \delta(G)$, and G is k -connected exactly when every pair has k internally-disjoint paths.

Chapter 13 — Max-flow min-cut

This is the theorem the previous chapter deferred to, and the one the next chapter reduces to. It is the central algorithmic result in graph theory, and its proof is shorter than its reputation suggests.

The setup

A **flow network** is a digraph with a non-negative **capacity** $c(u, v)$ on each arc, a **source** s , and a **sink** t . A **flow** f assigns each arc a value with

- **capacity:** $0 \leq f(u, v) \leq c(u, v)$;
- **conservation:** for every v other than s and t , flow in equals flow out.

The **value** of a flow is the net amount leaving s . A **cut** is a partition (S, T) with $s \in S$ and $t \in T$; its **capacity** is the total capacity of arcs from S to T . Arcs running back from T to S contribute nothing — a point that is easy to get wrong and that the proof below depends on.

Weak duality again, and again it is free:

Lemma. Every flow's value is at most every cut's capacity.

Proof. All flow leaving s must eventually cross from S to T , and the crossing arcs carry at most their capacity.

■

The residual network, and why reversals matter

The algorithm is: find a path from s to t with spare capacity, push as much as it takes, repeat. Done naively this gets stuck — an early greedy choice can block a better later one.

The fix is the **residual network**. Alongside each arc (u, v) carrying f , maintain a reverse arc (v, u) with residual capacity f . Pushing flow along the reverse arc *cancels* flow on the forward one.

```
def add_arc(self, u, v, c):
    self.cap[(u, v)] = self.cap.get((u, v), 0.0) + c
    self.cap.setdefault((v, u), 0.0)      # the reverse arc, initially empty
```

That single line is what makes the whole method work. Without reverse arcs the algorithm cannot revise a decision, and greedy path-pushing is genuinely wrong. With them, every augmenting path either routes new flow or *reroutes* old flow, and the search never needs to backtrack.

An **augmenting path** is any s – t path in the residual network. The algorithm — the **Ford–Fulkerson method** — is to augment until none exists.

Edmonds–Karp

Ford–Fulkerson does not say which augmenting path to take, and the choice matters more than it looks. Take them arbitrarily, and on a network with irrational capacities the method can run forever, converging to a value

below the maximum. With integer capacities it terminates but can take time proportional to the flow *value*, which is exponential in the input size.

Take shortest augmenting paths — BFS, not DFS — and it becomes Edmonds–Karp, running in $O(n \cdot m^2)$ regardless of capacities. The bound comes from the fact that the BFS distance from s to t in the residual network never decreases and must strictly increase every $O(m)$ augmentations.

```
def _shortest_augmenting_path(self, residual, source, sink):
    parent = {source: source}
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in self.adj[u]:
            if v not in parent and residual.get((u, v), 0.0) > 1e-12:
                parent[v] = u
                if v == sink:
                    return parent
                queue.append(v)
    return None
```

The choice of container is the entire difference between an algorithm with a polynomial bound and one without. It is the same `deque-versus-stack` distinction as Chapter 8, and it matters more here.

The theorem

Theorem (Ford–Fulkerson, Elias–Feinstein–Shannon, 1956). The maximum flow value equals the minimum cut capacity.

Proof. Weak duality gives $\max \leq \min$. For the reverse, run the algorithm to completion and let S be the set of vertices reachable from s in the final residual network. Then $t \notin S$, since otherwise there would be another augmenting path. So (S, T) is a cut.

Every arc from S to T is **saturated** — if it had spare residual capacity its head would be in S . Every arc from T to S carries **zero** flow — if it carried anything, its reverse residual arc would put its tail in S .

So the net flow across the cut equals the cut's total capacity, and the flow's value equals that. Hence this flow's value equals this cut's capacity, and by weak duality both are optimal. ■

The proof is constructive: it does not merely assert a matching cut exists, it hands you one. That is what `min_cut` returns, and the harness checks the construction separately from the number, because they are different claims:

held	ch13	Max-flow equals min-cut (80 graphs)
held	ch13	The cut the algorithm reports really has the flow's value (80 graphs)
held	ch13	Integer capacities give an integer maximum flow (80 graphs)

The second line is not redundant. A correct maximum value with a wrongly-extracted cut is a plausible bug, and checking only the number would miss it.

Integrality

Corollary. If every capacity is an integer, some maximum flow is integral.

Proof. Every augmentation pushes the path's bottleneck, which is an integer if all residual capacities are. By induction they remain integers throughout. ■

This corollary is why flow solves combinatorial problems at all. Chapter 12's Menger and Chapter 14's matching both need the answer to be a *set of paths* or a *set of edges*, not a fractional assignment — and integrality is what promises that the optimum can be read off as one. A linear program with the same constraints would give you 0.5 on three arcs and no way to round it.

```
net = FlowNetwork(6, [(0,1,16), (0,2,13), (1,2,10), (2,1,4), (1,3,12),
                      (3,2,9), (2,4,14), (4,3,7), (3,5,20), (4,5,4)])
print(net.max_flow(0, 5)[0])          # 23.0
print(sorted(net.min_cut(0, 5)[1]))    # [0, 1, 2, 4]
```

The cut $\{0,1,2,4\}$ versus $\{3,5\}$ has capacity $12 + 7 + 4 = 23$. Note that arc $(3,2)$ with capacity 9 runs *backwards* across this cut and contributes nothing — the point flagged at the top of the chapter.

What it reduces to

Flow is the workhorse of combinatorial optimisation because so many problems are it in disguise:

Problem	Encoding
Menger, edge form	unit capacities (Ch 12)
Menger, vertex form	unit capacities + vertex splitting (Ch 12)
Bipartite matching	source $\rightarrow$ left $\rightarrow$ right $\rightarrow$ sink, all unit (Ch 14)
Vertex-disjoint routing	vertex splitting
Project selection, image segmentation	min-cut directly
Baseball elimination	max-flow feasibility

The skill worth acquiring is not implementing Edmonds–Karp — you will use a library — but recognising that a problem *is* a flow problem, which is usually a matter of asking what the capacities should be.

Try it

Watch a reverse arc do its job. First a network where the greedy first path is a mistake:


```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.flow import FlowNetwork
# 0->1->3 and 0->2->3, plus a crossing arc 1->2
net = FlowNetwork(4, [(0,1,1), (0,2,1), (1,2,1), (1,3,1), (2,3,1)])
v, residual = net.max_flow(0, 3)
print('max flow:', v, ' min cut:', net.brute_force_min_cut(0, 3))
print('flow on the crossing arc 1->2:', net.cap[(1,2)] - residual[(1,2)])
"
```

```
max flow: 2.0  min cut: 2.0
flow on the crossing arc 1->2: 0.0
```

The maximum is 2 — one unit down each side — and the crossing arc ends up carrying nothing. Edmonds–Karp finds this immediately because BFS takes the two-arc paths first. Ford–Fulkerson taking the three-arc path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ first would have to *undo* it via the reverse arc, which is exactly the situation reverse arcs exist for.

Exercises

1. Why does the residual network need reverse arcs? What breaks without them?
2. Ford–Fulkerson does not specify which augmenting path to take. What goes wrong if you choose badly?
3. State the integrality corollary and explain why it is what makes flow useful for combinatorial problems.
4. An arc runs from the sink side back to the source side of a cut. How much does it contribute to the cut's capacity?

Solutions in Appendix E.

Takeaways

- Max-flow equals min-cut, and the proof is constructive: the residual-reachable set of the saturated network is a minimum cut.
- Reverse residual arcs are the mechanism. Without them, greedy augmentation cannot revise a decision and is simply wrong.
- Ford–Fulkerson does not specify which augmenting path. Taking shortest ones (BFS) gives Edmonds–Karp and $O(n^3)$; taking arbitrary ones can fail to terminate on irrational capacities.
- Integer capacities give an integer optimum. That corollary is why flow solves combinatorial problems rather than merely numerical ones.
- Arcs crossing back from the sink side contribute zero to a cut's capacity.

Chapter 14 — Matching

A **matching** is a set of edges, no two sharing a vertex. Finding a largest one is one of the few genuinely hard-looking combinatorial problems that turns out to be solvable in polynomial time — and on bipartite graphs it is Chapter 13 in a hat.

Augmenting paths

Let M be a matching. A vertex is **covered** if some edge of M touches it, and **exposed** otherwise. An **alternating path** alternates between edges outside M and edges inside it. An **augmenting path** is an alternating path whose two endpoints are both exposed.

Flip an augmenting path — take its non-matching edges into M and remove its matching ones — and you get a matching with exactly one more edge, since the path has one more non-matching edge than matching edges.

Theorem (Berge, 1957). M is maximum if and only if there is no M -augmenting path.

Proof. If an augmenting path exists, flipping it gives a larger matching, so M is not maximum.

Conversely, suppose M is not maximum and let N be larger. Consider the symmetric difference $M \Delta N$, and look at the graph it forms. Every vertex has degree at most 2 in it — at most one edge from each matching — so every component is a path or an even cycle, with edges alternating between M and N . Cycles use equally many from each. Since $|N| > |M|$, some component must have more N edges than M edges, and that component can only be a path starting and ending with N edges. Its endpoints are exposed in M , so it is an M -augmenting path. ■

The symmetric-difference argument is the technique to keep. Comparing your object to a hypothetical better one and looking at where they differ is how essentially every matching result is proved, and the observation that $M \Delta N$ has maximum degree 2 is what makes the components tractable.

Berge's theorem turns finding a maximum matching into: search for an augmenting path, flip it, repeat. At most $n/2$ iterations.

The bipartite case, and what is hidden in it

On a bipartite graph, searching for an augmenting path is a straightforward alternating search from each exposed left vertex:

```
def try_augment(v, visited):
    for w in sorted(g.neighbours(v)):
        if w in visited:
            continue
        visited.add(w)
        if w not in match or try_augment(match[w], visited):
            match[w] = v
            match[v] = w
            return True
    return False
```

$O(nm)$ overall. Hopcroft–Karp improves this to $O(m\sqrt{n})$ by finding a maximal set of shortest augmenting paths per phase.

That search is correct on bipartite graphs and **wrong in general**, and the reason is worth seeing. In a graph with an odd cycle, an alternating walk can return to a vertex on the opposite parity and the search cannot tell whether it has found an augmenting path or gone round a loop. Edmonds' 1965 blossom algorithm fixes this by contracting odd cycles — "blossoms" — and it is the paper that introduced the idea of polynomial time as the definition of tractable.

The harness registers the limitation as a theorem expected to be refuted:

```
refuted    ch14 Augmenting paths find a maximum matching in any graph (1 graphs)
```

The witness is C_7 with the left side declared as $\{0,1,2,6\}$: the routine returns a matching of size 2 where 3 exists. It is worth noting how *narrow* that witness is. On C_3 and C_5 the same routine returns the correct answer for **every** choice of left set, so a test on small odd cycles would find nothing and conclude the code was fine.

Hall's theorem

Theorem (Hall, 1935). A bipartite graph with parts L and R has a matching covering every vertex of L if and only if $|N(S)| \geq |S|$ for every $S \subseteq L$.

The condition is obviously necessary: S needs $|S|$ distinct partners, all inside $N(S)$.

Proof of sufficiency. Suppose no matching covers L , and take a maximum matching M with some exposed $u \in L$. Let Z be all vertices reachable from u by alternating paths. No vertex of $Z \cap R$ is exposed — that would give an augmenting path, contradicting maximality — so every vertex of $Z \cap R$ is matched, and matched into $Z \cap L$. Setting $S = Z \cap L$, we get $N(S) = Z \cap R$, and $|Z \cap L| = |Z \cap R| + 1$ because of u . So $|N(S)| = |S| - 1 < |S|$, violating the condition. ■

Checking Hall's condition requires looking at every subset, and the harness does exactly that — $2^{|L|}$ subsets. Anything cheaper would be assuming the theorem being tested:

```
held      ch14 Hall: every left vertex can be matched iff |N(S)| >= |S| for all
S (120 graphs)
```

Hall's theorem is a min–max theorem in disguise, and the disguise is thin: it says the obstruction to a perfect matching is always a single "too-crowded" set.

König's theorem

Theorem (König, 1931). In a bipartite graph, the maximum matching size equals the minimum vertex cover size.

Weak duality is free — every edge of a matching needs its own cover vertex — and once again the content is the other direction. The proof is a *construction*:

```
def konig_cover(g, left, right):
    match = bipartite_matching(g, left)
    unmatched_left = [v for v in left if v not in match]
    # Z: everything reachable from an exposed left vertex by alternating paths
    ...
    return (set(left) - reachable) | (set(right) & reachable)
```

Proof that this is a cover of the right size. Take an edge uv with $u \in L$, $v \in R$. If $u \notin Z$ then u is in the cover. If $u \in Z$, then $v \in Z$ too — either uv is a non-matching edge, so the alternating path extends through it, or it is a matching edge and u was reached along it. Either way v is covered. So it is a cover.

For the size: every vertex of $L \setminus Z$ is matched (an exposed left vertex is in Z by definition), and every vertex of $R \cap Z$ is matched (shown in Hall's proof above). No matching edge has both endpoints in the cover — that would need $u \in L \setminus Z$ and $v \in R \cap Z$, but $v \in Z$ reached along its matching edge forces $u \in Z$. So the cover has exactly one vertex per matching edge. ■

The harness checks the number and the construction separately, since a right answer produced by a wrong construction is a real failure mode:

```
held      ch14 König: in a bipartite graph, max matching = min vertex cover
(120 graphs)
held      ch14 König's construction returns a cover of exactly the matching's
size (120 graphs)
```

König fails on non-bipartite graphs, and the smallest witness is the triangle: maximum matching 1, minimum vertex cover 2. Chapter 19 identifies exactly which graphs the equality holds for, and it is a much larger class than the bipartite ones.

The min–max family

Three chapters, one shape:

Theorem	max	min
Menger (Ch 12)	disjoint $s-t$ paths	$s-t$ cut
Max-flow min-cut (Ch 13)	flow value	cut capacity
König (Ch 14)	matching	vertex cover
Hall (Ch 14)	matching saturating L	violating set (obstruction)

All four are the same theorem seen from different angles, and all four are instances of linear programming duality where the polyhedron happens to have integral vertices. That last sentence is the whole reason bipartite matching is easy and general matching needed Edmonds: the bipartite constraint matrix is **totally unimodular**, and the general one is not.

Try it

Watch König's construction produce a cover the same size as the matching, on a graph where neither is obvious:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.matching import bipartite_matching, bipartition, matching_size,
konig_cover, min_vertex_cover_bruteforce

# left {0,1,2}, right {3,4,5}
g = Graph(6, [(0,5), (1,3), (1,4), (1,5), (2,5)])
left, right = bipartition(g)
m = bipartite_matching(g, left)
cover = konig_cover(g, left, right)
print('matching size      ', matching_size(m))
print('konig cover       ', sorted(cover), 'size', len(cover))
print('true minimum cover ', min_vertex_cover_bruteforce(g))
print('is a cover         ', all(u in cover or v in cover for u, v in g.edges()))
"
```

```
matching size      2
konig cover       [1, 5] size 2
true minimum cover 2
is a cover        True
```

Five edges covered by two vertices, matching a maximum matching of size 2. The cover takes **one vertex from each side** — 1 from the left and 5 from the right — and which side each comes from is exactly what the alternating-reachability set **Z** decides. It is not "one endpoint of each matching edge, chosen arbitrarily": choosing wrongly here gives a set that is not a cover at all.

Picking this example took a search. The obvious small bipartite graphs give a cover that is just the whole left side, which is trivially a cover and demonstrates nothing.

Exercises

1. Define an augmenting path, and say what flipping one does to the matching size.
2. State Berge's theorem and name the technique its proof uses.
3. The simple augmenting search is wrong on non-bipartite graphs. What structure defeats it?
4. König's theorem says max matching equals min vertex cover. Compute both for a triangle and explain the result.

Solutions in Appendix E.

Takeaways

- Berge: a matching is maximum iff no augmenting path exists. Proved by symmetric difference, which is the technique to carry forward.

- The simple augmenting search is correct only on bipartite graphs. Odd cycles defeat it, and Edmonds' blossom algorithm is the fix. The witness here is C_7 ; C_3 and C_5 do not expose the bug at all.
- Hall: L can be saturated iff no subset of L has too few neighbours. Checking it honestly means examining all $2^{|L|}$ subsets.
- König: bipartite max matching equals min vertex cover, and the proof builds the cover from alternating reachability. It fails on the triangle.
- Menger, max-flow min-cut, König and Hall are one theorem in four costumes — LP duality with an integral polyhedron.

Chapter 15 — Colouring

A **proper colouring** assigns each vertex a colour so that adjacent vertices differ. The **chromatic number** $\chi(G)$ is the fewest colours that suffice. Computing it is NP-hard (Chapter 22), so most of the subject is bounds — and the bounds are where the ideas are.

The obvious bounds

Lemma. $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$.

The lower bound is immediate: a clique of size ω needs ω distinct colours. The upper bound comes from the greedy algorithm below.

Both can be far from the truth. The gap on the left is Chapter 19's subject. The gap on the right is enormous for a star: $K_{1,7}$ has $\Delta = 7$ and $\chi = 2$.

Greedy, and the order it depends on

Take the vertices in some order; give each the smallest colour not used by an already-coloured neighbour.

```
def greedy_colouring(g, order=None):
    colour = {}
    for v in order:
        if order is not None:
            v = order.pop()
        else:
            v = g.vertices().pop()
        used = {colour[w] for w in g.neighbours(v) if w in colour}
        c = 0
        while c in used:
            c += 1
        colour[v] = c
    return colour
```

It never uses more than $\Delta + 1$ colours, since a vertex has at most Δ coloured neighbours and $\Delta + 1$ colours always leave one free. That proves the upper bound above.

The result depends entirely on the order, and the dependence is not mild. The standard witness is the **crown graph**: $K_{n,n}$ with a perfect matching deleted. It is bipartite, so $\chi = 2$. Order the vertices by alternating sides and greedy uses n colours:

```
crown n=3: chi=2 greedy(natural)=2 greedy(interleaved)=3 degeneracy=2
crown n=4: chi=2 greedy(natural)=2 greedy(interleaved)=4 degeneracy=3
crown n=5: chi=2 greedy(natural)=2 greedy(interleaved)=5 degeneracy=4
```

Greedy on a two-colourable graph using five colours is as bad as an approximation can plausibly get, and it is not a contrived pathology — it is what happens if you happen to process vertices in the wrong order.

Note the natural order gets it right every time. If you test greedy on crown graphs without thinking about ordering, you will conclude it works well.

Degeneracy: the bound worth knowing

$\Delta + 1$ is weak because it is driven by the single worst vertex. The fix is to ask what happens as you peel the graph apart.

The **degeneracy** $d(G)$ is the largest k such that every subgraph has a vertex of degree at most k . Equivalently: repeatedly delete a minimum-degree vertex, and $d(G)$ is the largest degree you ever delete.

Theorem. $\chi(G) \leq d(G) + 1$, and $d(G) \leq \Delta(G)$.

Proof. Delete minimum-degree vertices one at a time, recording the order; then colour greedily in the **reverse** of that order. When each vertex is coloured, its already-coloured neighbours are exactly those that were still present when it was deleted — at most $d(G)$ of them. So $d(G) + 1$ colours suffice. ■

This is strictly better than $\Delta + 1$ and never worse:

```
star K_1,7: Delta = 7  degeneracy = 1  chi = 2
a tree:      Delta = 3  degeneracy = 1  chi = 2
```

Every forest has degeneracy 1, giving $\chi \leq 2$ — Chapter 6's result, recovered as a special case. Every planar graph has degeneracy at most 5 (Chapter 17), giving the six colour theorem for free.

```
held      ch15 Greedy colouring uses at most degeneracy + 1 colours (52 graphs)
held      ch15 chi <= degeneracy + 1, which is never worse than Delta + 1 (52
graphs)
```

Brooks' theorem

$\Delta + 1$ is tight for exactly two families, and that is the whole content of the result.

Theorem (Brooks, 1941). If G is connected and is neither a complete graph nor an odd cycle, then $\chi(G) \leq \Delta(G)$.

The two exceptions are not decoration. K_n has $\Delta = n - 1$ and $\chi = n$. An odd cycle has $\Delta = 2$ and $\chi = 3$. Drop either exception and the theorem is false at K_3 , which is both.

The harness encodes the exceptions as *hypothesis failures* rather than as special cases that pass, which is the distinction Chapter 3 argued for:

```
if canonical(g) == canonical(complete(g.n)):
    return None # says nothing, rather than "held"
if g.n % 2 == 1 and canonical(g) == canonical(cycle(g.n)):
    return None
return chromatic_number(g) <= max_degree(g)
```

The proof is a case analysis on connectivity that this book does not reproduce in full; the readable core is that a non-complete connected graph with $\Delta \geq 3$ has an ordering in which greedy leaves a colour free at the last vertex, obtained by rooting a spanning tree at a suitable vertex and colouring inward.

Exact colouring

$\chi(G)$ is computed here by trying $k = 1, 2, 3, \dots$ with backtracking:

```
def chromatic_number(g):
    for k in range(1, g.n + 1):
        if _colourable(g, k):
            return k
    return g.n
```

Exponential, and unavoidably so unless $P = NP$. Fine to about twelve vertices, which is enough for the verification harness and for nothing else. Chapter 23 covers what to do instead when you actually need an answer.

Try it

Watch the ordering matter, on a graph that is two-colourable:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph
from graphs.algorithms import chromatic_number, greedy_colouring
from graphs.planar import degeneracy_order

# crown graph:  $K_{4,4}$  minus a perfect matching
n = 4
g = Graph(2*n, [(i, n+j) for i in range(n) for j in range(n) if i != j])
interleaved = [x for i in range(n) for x in (i, n+i)]
print('chromatic number      ', chromatic_number(g))
print('greedy, natural order ', max(greedy_colouring(g).values()) + 1)
print('greedy, interleaved   ', max(greedy_colouring(g, interleaved).values()) + 1)
print('greedy, degeneracy    ', max(greedy_colouring(g,
degeneracy_order(g)).values()) + 1)
"
```

```
chromatic number      2
greedy, natural order 2
greedy, interleaved   4
greedy, degeneracy    2
```

Four colours for a bipartite graph, from nothing but a bad order. The degeneracy order recovers the right answer here, though it is not guaranteed to in general — it guarantees only $d + 1$, which is 4 for this graph.

Exercises

1. Show that $\chi(G) \geq n / \alpha(G)$, where $\alpha(G)$ is the size of the largest independent set.
2. Find a graph with $\omega(G) = 2$ and $\chi(G) = 3$. What is the smallest such graph?
3. Prove that every graph with degeneracy d has at most $d \cdot n$ edges.
4. The crown graph above has $\chi = 2$ but greedy can use n colours. What is its degeneracy, and why does that not contradict the $d + 1$ bound?

Solutions in Appendix E.

Takeaways

- $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$. Both inequalities can be arbitrarily loose.
- Greedy's answer depends entirely on vertex order. On the crown graph a bad order uses n colours where 2 suffice — and the natural order gets it right, so a careless test sees nothing wrong.
- Degeneracy is the bound to use: $\chi \leq d + 1$, never worse than $\Delta + 1$, and it gives forests and planar graphs their bounds for free.
- Brooks: $\chi \leq \Delta$ unless the graph is complete or an odd cycle. Both exceptions are necessary and K_3 is both.
- Exact χ is NP-hard; the implementation here is for verification, not for use.

Chapter 16 — Bipartite graphs

A graph is **bipartite** if its vertices split into two sets with every edge crossing between them. Equivalently, $\chi(G) \leq 2$. This is the cleanest characterisation in the book, and the one place where a structural property has a genuinely simple test.

The theorem

Theorem (König, 1916). G is bipartite if and only if it contains no odd cycle.

Proof. If G is bipartite with parts A and B , any cycle alternates between them, so returning to its start requires an even number of steps.

Conversely, suppose G has no odd cycle. Handle each component separately. Pick a root r and colour each vertex by the parity of $d(r, v)$. Suppose some edge uv joined two vertices of equal parity. Take shortest paths from r to u and to v , and let x be their last common vertex. The two path sections from x , plus the edge uv , form a cycle of length $(d(r, u) - d(r, x)) + (d(r, v) - d(r, x)) + 1$, which is odd because $d(r, u)$ and $d(r, v)$ have the same parity. Contradiction. ■

The proof is an algorithm: BFS, colour by level parity, and any edge *within* a level names an odd cycle. That is Chapter 8's observation about BFS levels, now doing real work.

```
def two_colouring(g):
    colour = {}
    for source in g.vertices():
        if source in colour:
            continue
        colour[source] = 0
        queue = deque([source])
        while queue:
            v = queue.popleft()
            for w in g.neighbours(v):
                if w not in colour:
                    colour[w] = 1 - colour[v]
                    queue.append(w)
                elif colour[w] == colour[v]:
                    return None          # an edge inside a level: odd cycle
    return colour
```

$O(n + m)$. Compare this with $\chi(G) \leq 3$, which is NP-complete. The jump in difficulty between two colours and three is the sharpest complexity boundary in graph theory, and it happens here.

The check that would have been circular

Verifying "bipartite iff no odd cycle" is a trap. The obvious test — compare `is_bipartite` against `has_odd_cycle` — is worthless if `has_odd_cycle` is implemented as `not is_bipartite`, which is exactly how a sensible library implements it.

So the harness enumerates cycles directly:

```
def _has_odd_cycle_bruteforce(g):
    for size in range(3, g.n + 1, 2):
        for subset in itertools.combinations(g.vertices(), size):
            first, *rest = subset
            for tail in itertools.permutations(rest):
                walk = (first,) + tail
                if all(g.has_edge(walk[i], walk[(i+1) % size]) for i in range(size)):
                    return True
    return False
```

Exponential, and the only honest way to check this particular theorem:

```
held      ch16  Konig: G is bipartite iff it has no odd cycle  (52 graphs)
```

Necessary is not sufficient

Bipartite graphs are triangle-free. The converse fails, and the harness registers it as a theorem expected to be refuted:

```
refuted   ch16  Triangle-free does not imply bipartite  (27 graphs)
```

The witness is C_5 : no triangle, but an odd cycle, so $\chi = 3$. This matters more than it looks. "Triangle-free" is a *local* condition — check every three vertices — while bipartiteness is *global*. No amount of local checking establishes it, and Chapter 27 is about how far triangle-freeness alone constrains a graph.

```
print(is_bipartite(cycle(5)), is_bipartite(cycle(6)))  # False True
print(is_bipartite(petersen()), chromatic_number(petersen()))  # False 3
```

The Petersen graph has girth 5 — no triangles, no squares — and is still not bipartite.

Why bipartite graphs are easy

A long list of NP-hard problems becomes polynomial on bipartite graphs:

Problem	General	Bipartite
Maximum matching	$O(n^3)$ (blossoms)	$O(m\sqrt{n})$
Minimum vertex cover	NP-hard	= max matching (König, Ch 14)
Maximum independent set	NP-hard	$n - \text{max matching}$
3-colouring	NP-complete	trivial: $\chi \leq 2$

The reason is uniform, and it is the one from Chapter 14: the constraint matrix of a bipartite graph is **totally unimodular**, so the natural linear program has integral optimal vertices and LP duality gives you the combinatorial min–max theorem directly. Odd cycles are precisely what destroys that property.

So "contains no odd cycle" is not a curiosity. It is the structural reason an entire family of problems collapses in difficulty.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import cycle, petersen, complete_bipartite
from graphs.algorithms import is_bipartite, two_colouring, chromatic_number
for name, g in [('C5', cycle(5)), ('C6', cycle(6)), ('K33',
complete_bipartite(3,3)), ('petersen', petersen())]:
    print(f'{name:<9} bipartite={is_bipartite(g)!s:<6} chi={chromatic_number(g)!s:<6}')
print()
print('C6 two-colouring:', two_colouring(cycle(6)))
print('C5 two-colouring:', two_colouring(cycle(5)))
"
```

```
C5      bipartite=False  chi=3
C6      bipartite=True   chi=2
K33     bipartite=True   chi=2
petersen bipartite=False chi=3

C6 two-colouring: {0: 0, 1: 1, 5: 1, 2: 0, 4: 0, 3: 1}
C5 two-colouring: None
```

Exercises

1. Show that a graph is bipartite if and only if every subgraph has an independent set containing at least half its vertices.
2. How many edges can a bipartite graph on n vertices have? Which graph achieves it?
3. Give a graph of girth 5 that is not bipartite, other than C_5 and the Petersen graph.
4. Explain why "triangle-free" can be checked in $O(n^3)$ while "bipartite" needs $O(n + m)$ — and why the cheaper test is the stronger property.

Solutions in Appendix E.

Takeaways

- Bipartite $\iff$ no odd cycle $\iff \chi \leq 2$, all testable in $O(n + m)$ by BFS level parity.
- An edge inside a BFS level is an odd cycle. The proof and the algorithm are the same object.
- Verifying this theorem requires enumerating cycles independently; the natural implementation of "has an odd cycle" is "is not bipartite", and checking one against the other proves nothing.
- Triangle-free does not imply bipartite. C_5 is the smallest witness; the Petersen graph has girth 5 and still fails.
- Bipartiteness makes matching, vertex cover and independent set all polynomial, because it makes the constraint matrix totally unimodular. Odd cycles are what break that.

Chapter 17 — Planarity

A graph is **planar** if it can be drawn in the plane with no edges crossing. Chapter 1 insisted a graph has no geometry; planarity is the exception that proves the rule, because it quantifies over *all* drawings. The drawing you made says nothing. The existence of a good one is a property of the graph.

What a drawing actually is

To reason about drawings without doing topology, replace them with something finite. A **rotation system** assigns each vertex a cyclic order of its neighbours. That is the entire combinatorial content of an embedding in an orientable surface: no coordinates, no lengths, just "which edge comes next as you turn around this vertex".

From a rotation system you can trace **faces** mechanically. Arriving at v along uv , leave along whichever neighbour follows u in v 's cyclic order; keep going until you return to where you started.

```
def _trace_faces(g, rotation):
    nxt = {}
    for v, order in rotation.items():
        pos = {w: i for i, w in enumerate(order)}
        for u in order:
            nxt[(u, v)] = (v, order[(pos[u] + 1) % len(order)])
    unvisited, count = set(nxt), 0
    while unvisited:
        arc = next(iter(unvisited)); count += 1
        while arc in unvisited:
            unvisited.discard(arc); arc = nxt[arc]
    return count
```

This is how `is_planar` works in this book: try rotation systems until one gives $n - m + f = 2$. It costs $\prod(\deg(v) - 1)!$ — 1024 systems for the Petersen graph, hopeless for K_6 . Real planarity testing is $O(n)$ (Hopcroft–Tarjan, or the left–right algorithm), and this book does not implement it, because its correctness is a separate twenty-page argument and would teach you nothing about planarity.

Euler's formula

Theorem (Euler, 1758). For a connected plane graph, $n - m + f = 2$.

Proof. Induction on m . If G is a tree, $m = n - 1$ and $f = 1$, giving $n - (n-1) + 1 = 2$. Otherwise G has a cycle; remove one of its edges. The graph stays connected, and the two faces on either side of that edge merge into one, so m and f each drop by one and $n - m + f$ is unchanged. ■

The verification here has a trap that is worth naming, because it is the same shape as the `get_graph()` problem from other books: **you cannot check Euler's formula by computing f from Euler's formula**. The harness gets f by tracing faces of an actual embedding, and only then checks the identity:

```
held      ch17 Euler's formula: n - m + f = 2 for connected planar graphs (30
graphs)
```

For disconnected graphs the formula reads $n - m + f = 1 + k$ with k components. The usual statement quietly assumes connectivity.

The edge bound, and why it is not enough

Corollary. A simple planar graph with $n \geq 3$ has $m \leq 3n - 6$. If it is also triangle-free, $m \leq 2n - 4$.

Proof. Every face is bounded by at least 3 edges, and every edge borders at most 2 faces, so $2m \geq 3f$. Substituting $f = 2 - n + m$ gives $2m \geq 3(2 - n + m)$, hence $m \leq 3n - 6$. Triangle-free means every face has at least 4 edges, giving $2m \geq 4f$ and the second bound. ■

That is another double count — the technique from Chapter 3, third appearance.

K_5 has $n = 5$, $m = 10 > 9$, so it is not planar. Done in one line.

$K_{3,3}$ has $n = 6$, $m = 9 \leq 12$, so the bound says nothing. It is still not planar, and this is the point: **the Euler bound is necessary, not sufficient**. The harness registers the converse as a theorem expected to be refuted:

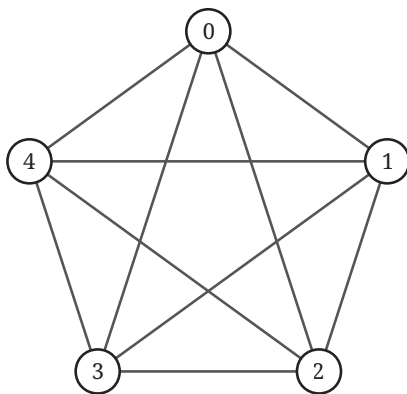
```
refuted  ch17  m <= 3n - 6 implies planar  (9 graphs)
```

The triangle-free bound does catch it: $K_{3,3}$ is bipartite, and $9 > 2 \cdot 6 - 4 = 8$. Two bounds, and you need to know which applies.

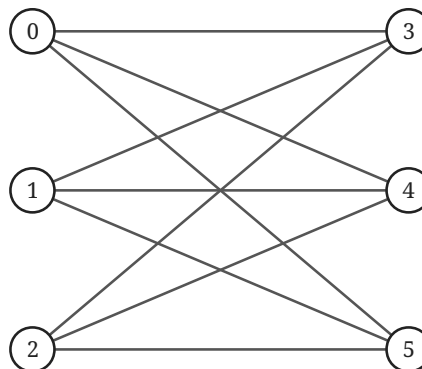
K4	n=4	m=6	planar=True	euler_bound=True	bip_bound=False	faces=4
K5	n=5	m=10	planar=False	euler_bound=False	bip_bound=False	
faces=None						
K33	n=6	m=9	planar=False	euler_bound=True	bip_bound=False	
faces=None						
petersen	n=10	m=15	planar=False	euler_bound=True	bip_bound=True	
faces=None						

The Petersen graph passes **both** bounds and is still not planar. No counting argument on n and m will ever settle planarity, because planarity is not about how many edges there are.

Kuratowski and Wagner



K_5 - the first non-planar graph



$K_{3,3}$ - bipartite, and not planar

The two forbidden minors. Every drawing of either has a crossing, and every non-planar graph contains one of them.

Theorem (Kuratowski, 1930). G is planar if and only if it contains no *subdivision* of K_5 or $K_{3,3}$.

Theorem (Wagner, 1937). G is planar if and only if it has no K_5 *minor* and no $K_{3,3}$ *minor*.

A **subdivision** replaces edges by paths; a **minor** is obtained by deleting vertices, deleting edges, and contracting edges. The two theorems are equivalent here, though minors are the more robust notion and the one Chapter 31 builds on.

Two forbidden graphs, and that is the complete answer — a remarkably small obstruction set for such a rich property. Chapter 31 explains why every minor-closed property has a finite obstruction set, which is Robertson–Seymour, and why that theorem is nonconstructive.

The harness checks Wagner against the embedding search — two entirely unrelated computations:

```
held      ch17  Wagner: planar iff no K5 minor and no K3,3 minor  (52 graphs)
```

The Petersen graph is the standard illustration: contract the five spokes and you get K_5 .

Consequences

Planar graphs are sparse ($m < 3n$), so $m = O(n)$ and they have a vertex of degree at most 5 — average degree is under 6. That single fact gives:

- **Six colour theorem**, immediately: degeneracy ≤ 5 , so $\chi \leq 6$ by Chapter 15.
- **Five colour theorem**, with a Kempe chain argument (Chapter 18).
- **Four colour theorem**, with 633 configurations and a computer (Chapter 18).

Planar graphs also have $O(\sqrt{n})$ separators (Lipton–Tarjan), which makes divide-and-conquer work on them and is why many NP-hard problems have subexponential algorithms restricted to planar inputs.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete, complete_bipartite, petersen, cycle
from graphs.planar import is_planar, euler_bound, bipartite_euler_bound,
planar_face_count
for name, g in [('K4', complete(4)), ('K5', complete(5)),
                ('K3,3', complete_bipartite(3,3)), ('petersen', petersen())]:
    print(f'{name:<9} m<=3n-6: {euler_bound(g)!s:<6} m<=2n-4:
{bipartite_euler_bound(g)!s:<6} planar: {is_planar(g)!s:<6}')
f = planar_face_count(complete(4))
print()
print('K4 faces from a real embedding:', f, ' n - m + f =', 4 - 6 + f)
"
```



```

K4      m<=3n-6: True   m<=2n-4: False  planar: True
K5      m<=3n-6: False  m<=2n-4: False  planar: False
K3,3    m<=3n-6: True   m<=2n-4: False  planar: False
petersen m<=3n-6: True   m<=2n-4: True   planar: False

K4 faces from a real embedding: 4  n - m + f = 2

```

K_4 drawn in the plane has four faces — three triangles and the outer region — and Euler's formula checks out on a number that was traced rather than assumed.

Exercises

1. Use Euler's formula to show that every planar graph has a vertex of degree at most 5.
2. Show that K_5 minus any single edge is planar.
3. The Petersen graph passes both edge bounds. Give a short argument that it is nonetheless non-planar, without invoking Kuratowski.
4. How many faces does a planar embedding of a tree on n vertices have? Check against Euler's formula.

Solutions in Appendix E.

Takeaways

- Planarity quantifies over all drawings, so it is a property of the graph even though a drawing is not.
- A rotation system is a drawing's entire combinatorial content. Faces can be traced from it mechanically, and $n - m + f = 2$ characterises planarity.
- Check Euler's formula with a face count from an actual embedding. Computing f by rearranging the formula tests nothing.
- $m \leq 3n - 6$ is necessary, not sufficient — $K_{3,3}$ passes it. The triangle-free refinement $m \leq 2n - 4$ catches $K_{3,3}$ but not the Petersen graph. No edge-counting argument decides planarity.
- Kuratowski and Wagner: two forbidden structures, and that is the whole answer.
- Planar graphs are sparse and 5-degenerate, which hands you the six colour theorem for free.

Chapter 18 — The five and four colour theorems

Two theorems about the same objects. One has a proof you can read in a page. The other has a proof you cannot read at all, and this chapter is partly about what that means.

Six colours, for free

Chapter 17 established that every planar graph has a vertex of degree at most 5 — otherwise $2m = \sum \deg(v) \geq 6n$, contradicting $m \leq 3n - 6$. So planar graphs are 5-degenerate, and Chapter 15's bound gives $\chi \leq 6$ immediately.

held ch18 Every planar graph has a vertex of degree at most 5 (51 graphs)

That took two lines. Getting to five takes a page, and getting to four took a century.

Five colours

Theorem (Heawood, 1890). Every planar graph is 5-colourable.

Proof. Induction on n . Take a vertex v with $\deg(v) \leq 5$ and 5-colour $G - v$.

If v 's neighbours use at most 4 colours, a colour is free — done.

Otherwise $\deg(v) = 5$ and its neighbours use all five. Fix a planar embedding and label the neighbours $v_1, \dots, v_5$ in rotation order, with v_i coloured i .

Consider the subgraph induced by colours 1 and 3, and let H be the component containing v_1 . If $v_3 \notin H$, swap colours 1 and 3 throughout H . This stays proper — H is a whole component of that subgraph, so no edge leaves it with a colour conflict — and now v_1 is coloured 3, freeing colour 1 for v .

If $v_3 \in H$, there is a path from v_1 to v_3 using only colours 1 and 3. Together with v it encloses a region. Now consider colours 2 and 4 and the component containing v_2 . Any path from v_2 to v_4 would have to cross that first path — and in a *planar* embedding it cannot, because crossings do not exist. So v_4 is not in v_2 's component, and the same swap frees colour 2. ■

That is a **Kempe chain** argument, and the two-colour component swap is the technique. Note exactly where planarity enters: the last step. A path cannot cross another path. That is the only topological input, and it is doing all the work.

Four colours

Theorem (Appel–Haken, 1976). Every planar graph is 4-colourable.

Kempe published a proof in 1879. It stood for eleven years until Heawood found an error — the chain-swapping argument fails when two swaps interact, and Kempe had not handled the case. What survived the wreckage was the five colour theorem above.

The eventual proof has two parts:

- an **unavoidable set**: a list of 1,936 configurations (later reduced to 633) such that every planar graph must contain at least one;
- **reducibility**: a demonstration that a minimal counterexample cannot contain any of them.

The second part is where the computer comes in. Each configuration requires checking a large number of colourings, and doing it by hand for 633 configurations is not feasible.

The proof has been verified since — Robertson, Sanders, Seymour and Thomas simplified it in 1997, and Gonthier produced a fully machine-checked version in Coq in 2005. That last one is worth dwelling on: the proof is now more rigorously verified than most proofs humans write, and it is still unreadable.

This book cannot prove it. What it can do is check it:

```
held      ch18  Every planar graph is 4-colourable  (51 graphs)
```

That line is not evidence for the theorem. It says that every planar graph on at most 6 vertices is 4-colourable, which was never in doubt — the first potential counterexample would have to be enormous. It is in the harness for a different reason: to catch the statement being written down wrongly, and to fail loudly if `is_planar` or `chromatic_number` breaks. Confusing "the check passes" with "the theorem is supported" is exactly the error this book's harness is designed not to encourage.

What the difficulty means

The gap between five and four colours is not a gap in effort. It is structural:

- **Six** follows from a degree bound, which follows from counting.
- **Five** follows from a local swap argument that planarity makes valid.
- **Four** has no known argument that avoids case analysis on hundreds of configurations.

Nobody has found a short proof and there is no reason to expect one. The four colour theorem may simply be a true statement whose shortest proof is large — and if so, the computer is not a crutch but the only available instrument.

It is worth contrasting with a nearby question that *is* easy. On the torus the answer is 7, proved by Heawood's counting argument in a page, with no cases. Higher genus is *easier* than the plane, because the counting bound is tight there and in the plane it is not.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete, cycle, petersen
from graphs.algorithms import chromatic_number
from graphs.planar import is_planar, degeneracy

for name, g in [('K4', complete(4)), ('C5', cycle(5)), ('petersen', petersen())]:
    print(f'{name:<9} planar={is_planar(g)!s:<6} degeneracy={degeneracy(g)}
chi={chromatic_number(g)}')
print()
print('K4 is planar and needs all four colours - the bound is tight.')
"
```

```
K4      planar=True   degeneracy=3 chi=4
C5      planar=True   degeneracy=2 chi=3
petersen planar=False degeneracy=3 chi=3
```

```
K4 is planar and needs all four colours - the bound is tight.
```

K_4 shows four colours are sometimes necessary, so the theorem cannot be improved. The Petersen graph shows the converse fails in the other direction: non-planar graphs can still be 3-colourable, so planarity is sufficient for $\chi \leq 4$ and nowhere near necessary.

Exercises

1. Prove the six colour theorem directly from $m \leq 3n - 6$.
2. Where exactly does the five colour proof use planarity? Identify the single step.
3. Give a planar graph requiring exactly 4 colours, other than K_4 .
4. Kempe's 1879 argument was wrong. Without looking it up, what goes wrong if you try to apply two Kempe swaps in sequence?

Solutions in Appendix E.

Takeaways

- Six colours follow from 5-degeneracy, which follows from $m \leq 3n - 6$. Two lines.
- Five colours follow from a Kempe chain argument. Planarity is used in exactly one step: two paths cannot cross.
- Four colours required 633 configurations and a computer, and there is still no readable proof. It has since been machine-checked in Coq.
- The harness's 4-colour check confirms the *statement* on small graphs. It is not evidence for the theorem, and treating it as such would be exactly the mistake this book's verification approach is meant to avoid.
- K_4 shows the bound is tight; the Petersen graph shows planarity is far from necessary.

Chapter 19 — Perfect and chordal graphs

Chapter 15 proved $\omega(G) \leq \chi(G)$ in one line and left the gap unexplained. This chapter is about the graphs where there is no gap — a class large enough to include most of the easy cases in the book, and characterised by a theorem that took forty years.

The definition, and why it quantifies over subgraphs

G is **perfect** if $\chi(H) = \omega(H)$ for every **induced** subgraph H .

The quantifier is essential and is the whole reason the definition works. Asking only $\chi(G) = \omega(G)$ gives a class with no useful structure — C_5 plus a disjoint K_3 has $\chi = \omega = 3$ while containing C_5 , which is the canonical imperfect graph. Requiring it of every induced subgraph makes the property **hereditary**, which is what allows induction, and hereditary classes are exactly the ones definable by forbidden induced subgraphs.

C_5 is the smallest imperfect graph: $\omega = 2$ (no triangle) but $\chi = 3$ (odd cycle).

C4	chordal=False	perfect=True	chi=2	omega=2	odd_hole=False
C5	chordal=False	perfect=False	chi=3	omega=2	odd_hole=True
K4	chordal=True	perfect=True	chi=4	omega=4	odd_hole=False
P4	chordal=True	perfect=True	chi=2	omega=2	odd_hole=False

Chordal graphs

A graph is **chordal** if every cycle of length at least 4 has a chord — equivalently, it has no induced cycle longer than a triangle.

Theorem. Every chordal graph is perfect.

The proof turns on a structural fact worth stating separately.

Lemma (Dirac). Every chordal graph has a **simplicial** vertex: one whose neighbours form a clique.

Given that, the theorem is an easy induction: take a simplicial vertex v , colour $G - v$ with $\omega(G - v)$ colours, and put v back. Its neighbourhood is a clique of size $\deg(v)$, so together with v it is a clique of size $\deg(v) + 1 \leq \omega(G)$. Hence at most $\omega(G) - 1$ colours appear on its neighbours, and one is free. ■

Repeatedly removing simplicial vertices gives a **perfect elimination ordering**, and a graph is chordal exactly when it has one. That yields a genuinely linear test — no exponential fallback, unusually for this book:

```
def is_chordal(g):
    order = maximum_cardinality_order(g)
    position = {v: i for i, v in enumerate(order)}
    for v in order:
        earlier = [w for w in g.neighbours(v) if position[w] < position[v]]
        if not earlier:
            continue
        parent = max(earlier, key=lambda w: position[w])
        for w in earlier:
            if w != parent and not g.has_edge(parent, w):
                return False
    return True
```

Checking it required an oracle that does not share the idea. The harness searches for an induced cycle of length ≥ 4 directly:

```
held      ch19 Chordality test agrees with searching for a chordless long cycle
(52 graphs)
held      ch19 Every chordal graph is perfect (44 graphs)
```

On chordal graphs, colouring, maximum clique, maximum independent set and minimum clique cover are all linear or near-linear — every one of them NP-hard in general. Chordal graphs are also exactly the intersection graphs of subtrees of a tree, which is why they turn up in sparse matrix elimination and in probabilistic graphical models.

The two big theorems

Theorem (Lovász, 1972 — the weak perfect graph theorem). G is perfect if and only if its complement is perfect.

A surprising statement: perfection is about cliques and colourings, and complementation swaps cliques with independent sets, so there is no obvious reason the property should survive. It does, and the theorem immediately doubles every result about perfect graphs.

Theorem (Chudnovsky, Robertson, Seymour, Thomas, 2006 — the strong perfect graph theorem). G is perfect if and only if neither G nor its complement contains an induced odd cycle of length at least 5.

Conjectured by Berge in 1961, proved 45 years later in a 150-page paper. The forbidden structures are the **odd holes** (induced odd cycles of length ≥ 5) and **odd antiholes** (their complements). The whole class is defined by excluding two infinite families, which is the same shape of answer as Kuratowski's in Chapter 17 — and it is why perfect graphs are recognisable in polynomial time.

The harness checks the equivalence, which for graphs up to five vertices is a real check of the *statement* even though it is no evidence for the theorem:

```
held      ch19 Berge: perfect iff no odd hole and no odd antihole (52 graphs)
```

Note C_5 is its own complement, so it is both an odd hole and an odd antihole — the minimal obstruction in both directions at once.

Where perfection has already appeared

This chapter names a pattern that Part III kept using without a name:

Class	Perfect?	Which theorem became easy
Bipartite	yes	König (Ch 14): $\chi = \omega = 2$, matching = cover
Forests	yes	trivially 2-colourable (Ch 6)
Chordal	yes	greedy colouring is optimal
Interval graphs	yes	scheduling by earliest finish time
Complements of bipartite	yes	by Lovász's theorem
Odd cycles C_5 , C_7 , ...	no	$\chi = 3$, $\omega = 2$

Chapter 14's König theorem is the bipartite case of a much more general phenomenon. Whenever you meet a graph problem that is unexpectedly easy on some class, perfection is the first thing to check.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import cycle, complete, path
from graphs.algorithms import chromatic_number, max_clique_size
from graphs.perfect import is_chordal, is_perfect, has_odd_hole, has_odd_antihole
for name, g in [('C4', cycle(4)), ('C5', cycle(5)), ('K4', complete(4)), ('P4',
path(4))]:
    print(f'{name:<4} chordal={is_chordal(g)!s:<6} perfect={is_perfect(g)!s:<6} '
          f'chi={chromatic_number(g)} omega={max_clique_size(g)} '
          f'odd_hole={has_odd_hole(g)!s:<6} odd_antihole={has_odd_antihole(g)!s:<6}')
"
```

```
C4  chordal=False  perfect=True   chi=2  omega=2  odd_hole=False  odd_antihole=False
C5  chordal=False  perfect=False  chi=3  omega=2  odd_hole=True   odd_antihole=True
K4  chordal=True   perfect=True   chi=4  omega=4  odd_hole=False  odd_antihole=False
P4  chordal=True   perfect=True   chi=2  omega=2  odd_hole=False  odd_antihole=False
```

C_4 is perfect but **not** chordal — it is an induced 4-cycle with no chord. Chordality is sufficient for perfection and not necessary, and C_4 is the smallest thing showing it.

Exercises

1. Verify by hand that C_5 has $\omega = 2$ and $\chi = 3$.
2. Show that every interval graph is chordal.

3. Why does the definition of perfect quantify over induced subgraphs rather than all subgraphs? Give a graph that would break the alternative.
4. C_5 is self-complementary. Find another self-complementary graph, and say whether it is perfect.

Solutions in Appendix E.

Takeaways

- Perfect means $\chi = \omega$ on **every induced subgraph**. The quantifier makes the class hereditary, which is what makes it useful.
- C_5 is the smallest imperfect graph, and is its own complement — an odd hole and an odd antihole simultaneously.
- Chordal graphs are perfect, via simplicial vertices and perfect elimination orderings, and chordality is testable in linear time.
- Chordal is sufficient for perfect, not necessary: C_4 is perfect and not chordal.
- Lovász: perfection survives complementation. Chudnovsky–Robertson–Seymour–Thomas: perfection is exactly the absence of odd holes and odd antiholes.
- Bipartite graphs are the perfect case you already met. König is a corollary of a much larger story.

Chapter 20 — Hamiltonicity

A **Hamiltonian cycle** visits every vertex exactly once and returns to its start. Compare with an Eulerian circuit, which uses every *edge* exactly once — and which has a perfect, one-line characterisation: a connected graph has one exactly when every degree is even.

Nothing like that is known for Hamiltonicity, and this chapter is about what you get instead.

Why the analogy fails

Euler's condition is **local**: check each vertex's degree independently. Hamiltonicity is irreducibly **global** — whether a cycle exists depends on how the whole graph fits together, and no amount of local inspection settles it.

That is not merely a failure of imagination. Hamiltonian cycle is **NP-complete** (Chapter 22), so a local checkable characterisation would give $P = NP$. The absence of a Hamiltonian analogue of Euler's theorem is a theorem-shaped hole with a complexity-theoretic explanation.

So the field offers **sufficient** conditions: hypotheses strong enough to force a cycle. All of them say some version of "enough edges, spread evenly".

Dirac and Ore

Theorem (Dirac, 1952). If $n \geq 3$ and every vertex has degree at least $n/2$, then G is Hamiltonian.

Theorem (Ore, 1960). If $n \geq 3$ and $\deg(u) + \deg(v) \geq n$ for every non-adjacent pair u, v , then G is Hamiltonian.

Ore implies Dirac — if every degree is at least $n/2$ then every pair sums to at least n — so Ore is the stronger theorem, applying to strictly more graphs.

Proof of Ore. Suppose not, and let G be a counterexample with the most edges: it satisfies Ore's condition, is not Hamiltonian, and adding any edge makes it Hamiltonian.

Take non-adjacent u, v . Adding uv creates a Hamiltonian cycle, which must use uv , so G has a Hamiltonian **path** $u = x_1, x_2, \dots, x_n = v$.

Now consider the sets $S = \{i : u \text{ adjacent to } x_{i+1}\}$ and $T = \{i : v \text{ adjacent to } x_i\}$, both subsets of $\{1, \dots, n-1\}$. We have $|S| + |T| = \deg(u) + \deg(v) \geq n$, and both are inside a set of size $n - 1$, so by pigeonhole they share an index i .

But then $u x_{i+1}$ and $x_i v$ are both edges, and the cycle $u, x_2, \dots, x_i, v, x_{n-1}, \dots, x_{i+1}, u$ — running the path forward to x_i , jumping to v , then backward to x_{i+1} , then back to u — is Hamiltonian. Contradiction. ■

The **extremal counterexample** setup (take the one with the most edges) is the same move as Chapter 7's spanning-tree proof, and the pigeonhole step is the same as Chapter 3's.

Sufficient is very far from necessary

The conditions are demanding, and most Hamiltonian graphs fail them badly:

C5	ham=True	dirac=False	ore=False	mindeg=2	n/2=2.5
K4	ham=True	dirac=True	ore=True	mindeg=3	n/2=2.0
petersen	ham=False	dirac=False	ore=False	mindeg=3	n/2=5.0
K33	ham=True	dirac=True	ore=True	mindeg=3	n/2=3.0
K23	ham=False	dirac=False	ore=False	mindeg=2	n/2=2.5

C₅ is Hamiltonian by construction and fails Dirac — minimum degree 2 against a threshold of 2.5. The harness registers the converse as a theorem expected to be refuted:

```
refuted    ch20  Every Hamiltonian graph satisfies Dirac's condition  (5 graphs)
```

Compare the last two rows. $K_{3,3}$ and $K_{2,3}$ both fail to look special, and one is Hamiltonian while the other is not — for a reason Dirac cannot see. $K_{a,b}$ is Hamiltonian exactly when $a = b$, because a cycle in a bipartite graph must alternate sides.

Bondy–Chvátal, and why Ore works

There is a cleaner statement behind Ore's theorem.

Theorem (Bondy–Chvátal, 1976). Let the **closure** of G be the result of repeatedly joining non-adjacent u, v with $\deg(u) + \deg(v) \geq n$. Then G is Hamiltonian if and only if its closure is.

This is genuinely surprising: adding edges obviously cannot destroy Hamiltonicity, but that these particular additions cannot *create* it is the content.

Ore's theorem is now a corollary. A graph satisfying Ore's condition has closure K_n , which is Hamiltonian, so the graph is.

```
held       ch20  Bondy-Chvatal: G is Hamiltonian iff its closure is  (49 graphs)
```

Necessary conditions

Going the other way, the useful facts are negative — ways to prove a graph is *not* Hamiltonian:

- A Hamiltonian graph is **2-connected**: the cycle gives two disjoint paths between any pair, so Menger (Chapter 12) applies.
- Removing k vertices from a Hamiltonian graph leaves at most k components, since the cycle is broken into at most k arcs. This is often the quickest disproof.
- A Hamiltonian bipartite graph has equal parts.

The Petersen graph is not Hamiltonian, and the standard proof is a case analysis on how many spokes the cycle uses. It *does* have a Hamiltonian path, which is why it is called **hypohamiltonian**: not Hamiltonian, but $G - v$ is Hamiltonian for every v . It is, predictably, this book's counterexample here too.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import cycle, complete_bipartite, petersen
from graphs.hamilton import hamiltonian_cycle, hamiltonian_path, dirac_condition,
is_hamiltonian
print('C5 hamiltonian cycle:', hamiltonian_cycle(cycle(5)), ' dirac says:',
dirac_condition(cycle(5)))
print('K2,3 hamiltonian:      ', is_hamiltonian(complete_bipartite(2,3)), '(unequal
parts)')
print('K3,3 hamiltonian:      ', is_hamiltonian(complete_bipartite(3,3)), '(equal
parts)')
print('petersen cycle:        ', hamiltonian_cycle(petersen()))
print('petersen path:         ', hamiltonian_path(petersen()) is not None)
"
```

```
C5 hamiltonian cycle: [0, 1, 2, 3, 4] dirac says: False
K2,3 hamiltonian:      False (unequal parts)
K3,3 hamiltonian:      True (equal parts)
petersen cycle:        None
petersen path:         True
```

C_5 is the cleanest illustration of the gap: a five-cycle *is* a Hamiltonian cycle, and the best-known sufficient condition cannot tell.

Exercises

1. Show that $K_{a,b}$ is Hamiltonian if and only if $a = b \geq 2$.
2. Prove that a Hamiltonian graph has no cut vertex.
3. Verify that Ore's condition implies Dirac's, and give a graph satisfying Ore but not Dirac.
4. Removing k vertices from a Hamiltonian graph leaves at most k components. Use this to show a specific graph of your choice is not Hamiltonian.

Solutions in Appendix E.

Takeaways

- Eulerian circuits have a local characterisation; Hamiltonian cycles have none, and NP-completeness explains why one should not be expected.
- Dirac ($\delta \geq n/2$) and Ore ($\deg u + \deg v \geq n$ for non-adjacent pairs) are sufficient, not necessary. C_5 is Hamiltonian and fails both.
- Ore's proof uses an extremal counterexample plus pigeonhole — the same two moves as Chapters 7 and 3.
- Bondy–Chvátal's closure theorem is the real statement; Ore is its corollary.
- To disprove Hamiltonicity, use the negative facts: 2-connectivity, the k -vertices- k -components bound, and equal parts in the bipartite case.

Chapter 21 — Cliques, independent sets, covers

Three famous NP-hard problems. They are the same problem, and seeing that clearly is worth more than any individual algorithm for them.

Three definitions

- A **clique** is a set of pairwise adjacent vertices. $\omega(G)$ is the largest.
- An **independent set** is a set of pairwise non-adjacent vertices. $\alpha(G)$ is the largest.
- A **vertex cover** is a set touching every edge. $\tau(G)$ is the smallest.

They are one problem

Proposition. S is a clique in G if and only if S is an independent set in $\bar{G}$. Hence $\omega(G) = \alpha(\bar{G})$.

Proof. "Every pair adjacent in G " and "no pair adjacent in $\bar{G}$ " are the same statement, because $\bar{G}$ has exactly the non-edges of G . ■

Theorem (Gallai, 1959). $\alpha(G) + \tau(G) = n$.

Proof. S is independent if and only if $V \setminus S$ is a vertex cover: no edge has both endpoints in S exactly when every edge has an endpoint outside it. That is a bijection between independent sets and vertex covers, reversing size. So the largest of one corresponds to the smallest of the other. ■

Both are checked against independent exhaustive searches rather than against each other:

held	ch21	Gallai: $\alpha(G) + \tau(G) = n$ (52 graphs)
held	ch21	A clique in G is an independent set in the complement (52 graphs)

C5	n=5	alpha=2	tau=3	alpha+tau=5	omega=2	omega(comp)=2
petersen	n=10	alpha=4	tau=6	alpha+tau=10	omega=2	omega(comp)=4
K4	n=4	alpha=1	tau=3	alpha+tau=4	omega=4	omega(comp)=1

Read the Petersen row: $\alpha = 4$, and the complement's largest clique is also 4. The identity is not a coincidence of small cases; it is the same set of vertices viewed twice.

The consequence is practical. **Solve one, and you have solved all three** — an exact algorithm, an approximation, or a hardness proof for any one transfers immediately. It also means you cannot hope to find one of them easy: if independent set had a polynomial algorithm, so would clique and vertex cover.

Why complementation is not free

There is a catch, and it is the sort of thing asymptotics hide.

The reduction from clique to independent set complements the graph. A sparse graph with $m = O(n)$ has a complement with $\Theta(n^2)$ edges. So an algorithm whose running time depends on m gets much slower after the reduction, even though the reduction is "linear time" in the sense that matters to complexity theory.

This is where Chapter 2's bitset representation earns its place. Complementing a bitmask row is one `~` and a mask; intersecting neighbourhoods is one `&`. Clique search on a dense graph in an adjacency-list representation is exactly the worst case the measurements in Chapter 2 identified.

```
def common_neighbours(self, u, v):
    return (self.rows[u] & self.rows[v]).bit_count()
```

Every serious maximum-clique implementation is built on that operation.

The natural greedy bound

The three problems all admit an obvious greedy heuristic, and it is worth knowing how badly each behaves before Chapter 23 addresses it properly:

- **Vertex cover** has a 2-approximation (Chapter 23), and under the unique games conjecture nothing better is possible.
- **Independent set** and **clique** have *no* constant-factor approximation, and in fact cannot be approximated within $n^{1-\epsilon}$ unless $P = NP$. That is one of the strongest inapproximability results known.

That asymmetry is startling, given the problems are equivalent. The resolution is that the equivalence preserves *exact* answers but not *ratios*: a cover of size $\tau + 1$ is a 1.01-approximation when $\tau = 100$, while the corresponding independent set of size $\alpha - 1$ may be a 2-approximation when $\alpha = 2$. Complementation maps "slightly too big" to "proportionally much too small".

Equivalent problems need not have equivalent approximation behaviour, and this is the cleanest example of it in the book.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import petersen, cycle, complete
from graphs.approx import max_independent_set, min_vertex_cover, max_clique
for name, g in [('C5', cycle(5)), ('K4', complete(4)), ('petersen', petersen())]:
    a = max_independent_set(g); t = min_vertex_cover(g); w = max_clique(g)
    print(f'{name:<9} alpha={len(a)} tau={len(t)} sum={len(a)+len(t)} n={g.n} '
          f'omega={len(w)} omega(complement)={len(max_clique(g.complement()))}')
    assert set(g.vertices()) - a == t or len(set(g.vertices()) - a) == len(t)
"
```

```
C5      alpha=2 tau=3 sum=5 n=5 omega=2 omega(complement)=2
K4      alpha=1 tau=3 sum=4 n=4 omega=4 omega(complement)=1
petersen alpha=4 tau=6 sum=10 n=10 omega=2 omega(complement)=4
```

Every row has $\alpha + \tau = n$, and every $\omega(\bar{G})$ matches the α of the same graph.

Exercises

1. Compute α , τ and ω for C_6 and check Gallai's identity.
2. Show that $\omega(G) + \alpha(G) \geq n$ is false in general, and find the smallest counterexample.
3. If G is bipartite, what is $\tau(G)$ in terms of its maximum matching? (Chapter 14.)
4. Explain why complementing a sparse graph makes a clique algorithm slower even though the reduction is linear time.

Solutions in Appendix E.

Takeaways

- Clique, independent set and vertex cover are one problem in three notations: $\omega(G) = \alpha(\bar{G})$ and $\alpha(G) + \tau(G) = n$.
- Solve one and you have solved all three, exactly. That cuts both ways — none of them can be easy.
- The reduction is linear time but not free: complementing a sparse graph produces a dense one, and algorithms whose cost depends on m suffer. Bitset representations (Chapter 2) are the standard answer.
- Equivalence of exact problems does **not** transfer to approximation. Vertex cover has a 2-approximation; independent set has essentially none.

Chapter 22 — NP-hardness

Several problems in this book have had "and this is NP-hard" attached without justification. This chapter makes the claim precise and shows the technique, on the graph problems already in hand.

What the claim means

A decision problem is in **NP** if a *yes* instance has a certificate checkable in polynomial time. "Does G have an independent set of size k ?" is in **NP**: hand over the set, and checking takes $O(k^2)$.

A problem is **NP-hard** if every problem in **NP** reduces to it in polynomial time, and **NP-complete** if it is both. A reduction from A to B is a polynomial-time map f with $x \in A \Rightarrow f(x) \in B$, and it means B is at least as hard as A .

Two things are commonly muddled. NP-hard does not mean "not in **P**" — that is the open question. And it does not mean "hard in practice": SAT solvers routinely dispatch instances with millions of variables. NP-hardness is a statement about worst cases over all instances, which is a much weaker claim about your instance than it sounds.

The reduction pattern

The direction that confuses everyone: to show B is hard, reduce a **known-hard** A to B . You are showing that solving B would let you solve A , so B inherits A 's difficulty. Reducing B to A shows nothing about B .

Every reduction has three obligations, and skipping the third is the standard error:

1. f runs in polynomial time;
2. $x \in A \Rightarrow f(x) \in B$;
3. $f(x) \in B \Rightarrow x \in A$.

The third is where reductions actually fail. It is easy to build a gadget that turns solutions into solutions and forget to rule out solutions of $f(x)$ that came from nowhere.

The reductions we already have

Chapter 21's identities are reductions, and unusually simple ones:

```
def clique_to_independent_set(g):
    return g.complement()           # omega(G) = alpha(complement)

def independent_set_to_vertex_cover(g, independent):
    return set(g.vertices()) - independent   # alpha(G) + tau(G) = n
```

Both are $O(n^2)$ and both preserve optima in each direction, which is obligation 3 done properly. The harness checks it:

```
held      ch22 The complement of a maximum independent set is a minimum vertex
cover    (52 graphs)
```

Note what that check does and does not establish. It confirms the *reduction is correct* — that the map really does carry optima to optima. It says nothing about NP-hardness, which is not the sort of statement a finite check can address.

3-SAT to independent set

The classical starting point is 3-SAT, which Cook and Levin proved NP-complete directly. From there, everything else is reductions.

Given a 3-CNF formula with k clauses, build a graph:

- for each clause, a **triangle** whose three vertices are its literals;
- an edge between any two vertices holding **contradictory** literals (x and $\neg x$).

Then the formula is satisfiable **if and only if** the graph has an independent set of size k .

Proof. ($\implies$) Given a satisfying assignment, pick one true literal per clause. No two are contradictory, since they are all true under one assignment, and no two are in the same triangle. That is an independent set of size k .

($\impliedby$) Given an independent set of size k : the triangles force at most one vertex per clause, so exactly one from each. No two chosen literals contradict, so setting each chosen literal true is consistent, and it satisfies every clause. Variables left unset can go either way. ■

The two gadgets do exactly one job each. The triangle enforces "at most one per clause"; the contradiction edges enforce consistency. That separation of concerns is what a good reduction looks like, and the ($\impliedby$) direction is where the triangles earn their keep.

What is hard, and what is not

Problem	Status
Shortest path (non-negative)	P — Chapter 10
Maximum flow	P — Chapter 13
Bipartite matching	P — Chapter 14
General matching	P — Edmonds' blossoms, Chapter 14
Planarity	P, in fact $O(n)$ — Chapter 17
2-colouring	P — Chapter 16
3-colouring	NP-complete
Independent set, clique, vertex cover	NP-complete — Chapter 21
Hamiltonian cycle	NP-complete — Chapter 20
Graph isomorphism	in NP, not known either way — Chapter 5

The boundary is sharp and often surprising. Two colours easy, three hard. Eulerian circuit easy, Hamiltonian cycle hard. Matching easy, independent set hard. In each pair the two problems look comparably difficult and are not, which is why "this looks hard" is never an argument.

Try it

Watch the reduction carry an optimum across, both ways:

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import petersen, cycle
from graphs.approx import (max_clique, max_independent_set, min_vertex_cover,
                           clique_to_independent_set,
                           independent_set_to_vertex_cover,
                           is_vertex_cover)
for name, g in [('C5', cycle(5)), ('petersen', petersen())]:
    ind = max_independent_set(g)
    cover = independent_set_to_vertex_cover(g, ind)
    print(f'{name}: alpha={len(ind)} -> cover size {len(cover)},
valid={is_vertex_cover(g, cover)}, '
        f'optimal={len(cover) == len(min_vertex_cover(g))}')
    print(f'    omega(G)={len(max_clique(g))} equals
alpha(complement)={len(max_independent_set(clique_to_independent_set(g)))}')
"
```

```
C5: alpha=2 -> cover size 3, valid=True, optimal=True
    omega(G)=2 equals alpha(complement)=2
petersen: alpha=4 -> cover size 6, valid=True, optimal=True
    omega(G)=2 equals alpha(complement)=2
```

Exercises

1. To prove that problem **B** is **NP**-hard, do you reduce **B** to a known-hard problem or the other way round? Explain why the other direction proves nothing.
2. In the 3-SAT reduction, what goes wrong if you omit the contradiction edges?
3. What goes wrong if you omit the triangles?
4. Vertex cover is **NP**-complete in general but easy on bipartite graphs (Chapter 14). Does that contradict **NP**-hardness?

Solutions in Appendix E.

Takeaways

- To prove **B** hard, reduce a known-hard **A** to **B**. The other direction proves nothing.
- A reduction has three obligations; the reverse implication is the one that fails in practice.
- The 3-SAT to independent set reduction uses two gadgets with one job each: triangles for "one literal per clause", contradiction edges for consistency.
- Checking a reduction on examples confirms it maps optima correctly. It cannot confirm **NP**-hardness, which is not that kind of statement.

- The P/NP -complete boundary does not track apparent difficulty: 2-colouring easy, 3-colouring hard; Euler easy, Hamilton hard; matching easy, independent set hard.

Chapter 23 — Living with hardness

NP-hard is not the end of the conversation. There are three honest responses — accept an approximate answer with a *proved* ratio, accept exponential time in a parameter that stays small, or restrict to a graph class where the problem is easy — and this chapter is about choosing between them.

Approximation with a guarantee

An algorithm is a ρ -**approximation** for a minimisation problem if it always returns a solution of cost at most $\rho \cdot \text{OPT}$. The word that matters is *always*: a ratio is a proved worst-case bound, not an observed average.

The standard example is beautifully crude.

Theorem. Taking both endpoints of every edge of a **maximal** matching gives a vertex cover of size at most $2 \cdot \text{OPT}$.

```
def vertex_cover_2approx(g):
    cover = set()
    for u, v in g.edges():
        if u not in cover and v not in cover:
            cover.add(u); cover.add(v)
    return cover
```

Proof. Let M be the matching built. Its edges are pairwise disjoint, so any vertex cover needs at least one vertex from each: $\text{OPT} \geq |M|$. The algorithm returns $2|M|$. Also, M is *maximal* — no edge can be added — so every edge of G touches a matched vertex, and the result really is a cover. ■

Two things about this proof. It never computes OPT , it only bounds it from below; that is how every approximation proof works. And it needs only a **maximal** matching, not a **maximum** one — greedy suffices, so the algorithm is linear time.

```
held      ch23  The matching heuristic returns a cover at most twice optimal  (47
graphs)
```

The heuristic that looks better and is worse

The obvious improvement is to be greedy about degree: repeatedly take the vertex covering the most remaining edges. It is more work, it uses more information, and it is intuitively smarter.

It has no constant approximation ratio at all. It is $\Theta(\log n)$ in the worst case.

Seeing this requires a real instance. Take a left side L of size k , and for each $i = 2 \dots k$ add $\lfloor k/i \rfloor$ right-hand vertices each joined to i distinct left vertices. L is a cover, so $\text{OPT} \leq k$. Greedy takes the high-degree right-hand vertices first and ends up consuming the whole right side, about $k \ln k$ vertices.

k=8	n=20	m=48	OPT<=8	greedy=12	ratio>=1.50	matching=16	ratio<=2.00
k=12	n=35	m=115	OPT<=12	greedy=23	ratio>=1.92	matching=24	ratio<=2.00
k=16	n=50	m=204	OPT<=16	greedy=34	ratio>=2.12	matching=32	ratio<=2.00
k=20	n=66	m=319	OPT<=20	greedy=46	ratio>=2.30	matching=40	ratio<=2.00

Read the `k=16` row twice. Greedy returns **34**; the crude matching heuristic returns **32**. The algorithm that uses more information does worse, on an instance built to expose exactly that, and it keeps getting worse as `k` grows.

Now read the first two rows. At `k = 8` and `k = 12`, greedy wins. **You need a fifty-vertex instance before the difference appears at all**, and nothing smaller hints at it. Anyone comparing these two heuristics on small graphs would conclude greedy is better and ship it.

refuted ch23 The max-degree heuristic is at most twice optimal (3 graphs)

That is why the guarantee matters more than the measurements. The matching heuristic's 2 is a theorem; greedy's apparent superiority was a sampling artefact.

Note also how the ratio is established. Computing `OPT` on a fifty-vertex graph is infeasible, but `L` is a cover of size `k` by construction, so $OPT \leq k$ and $|greedy| / k$ is a rigorous **lower bound** on the true ratio. Exhibiting a feasible solution to bound the optimum is the standard move, and it is the same one the 2-approximation proof makes in the other direction.

Parameterised complexity

The second response: keep exactness, and confine the exponential to a parameter.

Vertex cover of size at most `k` is solvable in $O(2^k \cdot (n + m))$. Pick any uncovered edge; one of its endpoints must be in the cover, so branch on the two choices with `k - 1`. The recursion depth is `k`, so the tree has 2^k leaves — independent of `n`.

```
def vertex_cover_at_most_k(g, k):
    uncovered = next(((u, v) for u, v in g.edges()), None)
    if uncovered is None:
        return set()
    if k <= 0:
        return None
    u, v = uncovered
    for choice in (u, v):
        smaller = Graph(g.n, [e for e in g.edges() if choice not in e])
        rest = vertex_cover_at_most_k(smaller, k - 1)
        if rest is not None:
            return {choice} | rest
    return None
```

Compare with brute force over $C(n, k)$ subsets. For `n = 106` and `k = 10`, that is 10^{60} against 10^{24} . A problem admitting such an algorithm is **fixed-parameter tractable**, and the distinction between $f(k) \cdot \text{poly}(n)$ and $n^{f(k)}$ is the entire subject.

held ch23 Bounded search finds a cover of size <= k exactly when one exists (52 graphs)

Not every problem cooperates. Independent set parameterised by solution size is $W[1]$ -hard, which is the parameterised analogue of NP -hard — so the equivalence of Chapter 21 breaks here too. Vertex cover is FPT; its exact twin under complementation is not.

Restricting the input

The third response, and often the best: notice your graphs are not arbitrary.

Restriction	What becomes easy
Bipartite	vertex cover, independent set, matching (Ch 14, 16)
Chordal / perfect	colouring, clique, independent set (Ch 19)
Planar	4-colouring is free; PTAS for many problems (Ch 17)
Bounded treewidth	almost everything, by dynamic programming (Ch 31)
Bounded degree	approximation ratios improve

This is why Part IV spent five chapters on structure. The practical answer to "this problem is NP -hard" is very often "yes, but my graphs are planar", and knowing the classes is what lets you notice.

What cannot be done

Honesty requires the negative results too:

- **Vertex cover** cannot be approximated better than $2 - \epsilon$ under the unique games conjecture, and better than 1.36 unless $P = NP$. The crude algorithm above is essentially optimal.
- **Independent set and clique** cannot be approximated within $n^{\{1-\epsilon\}}$ unless $P = NP$.
- **Colouring** cannot be approximated within $n^{\{1-\epsilon\}}$ either.
- **Metric TSP** has a 3/2-approximation (Christofides); general TSP has none, since any constant-factor approximation would decide Hamiltonicity.

So the 2-approximation for vertex cover is not a placeholder awaiting improvement. It is, under standard assumptions, the end of the road.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.approx import (greedy_lower_bound_instance, greedy_max_degree_cover,
                           vertex_cover_2approx, is_vertex_cover)
for k in (8, 12, 16, 20):
    g, known = greedy_lower_bound_instance(k)
    gr, mm = greedy_max_degree_cover(g), vertex_cover_2approx(g)
    assert is_vertex_cover(g, gr) and is_vertex_cover(g, mm)
    print(f'k={k:<3} n={g.n:<3} OPT<={k:<3} greedy={len(gr):<3} (ratio>={len(gr)/
k:.2f}) '
          f'matching={len(mm):<3} (ratio<={len(mm)/k:.2f})')
"
```

k=8	n=20	OPT≤8	greedy=12	(ratio≥1.50)	matching=16	(ratio≤2.00)
k=12	n=35	OPT≤12	greedy=23	(ratio≥1.92)	matching=24	(ratio≤2.00)
k=16	n=50	OPT≤16	greedy=34	(ratio≥2.12)	matching=32	(ratio≤2.00)
k=20	n=66	OPT≤20	greedy=46	(ratio≥2.30)	matching=40	(ratio≤2.00)

The matching column never exceeds 2.00, because it cannot. The greedy column passes it at $k = 16$ and keeps climbing.

Exercises

1. Prove that the matching heuristic returns a cover of size exactly $2|M|$, and that M being maximal is what makes it a cover.
2. Give a graph where the matching heuristic returns exactly $2 \cdot \text{OPT}$.
3. Why does $O(2^k \cdot (n+m))$ beat $O(n^k)$ for large n and small k ? Give numbers.
4. Vertex cover is FPT but independent set is $W[1]$ -hard, even though Chapter 21 showed the problems are equivalent. Explain how both can be true.

Solutions in Appendix E.

Takeaways

- A p -approximation is a proved worst-case bound. Approximation proofs never compute OPT ; they bound it, usually by exhibiting a feasible solution or a disjoint structure.
- The maximal-matching vertex cover is a 2-approximation, linear time, and essentially optimal under standard assumptions.
- The max-degree heuristic looks smarter and has no constant ratio. It beats the matching heuristic on graphs up to about 35 vertices and loses from 50 on — so testing on small instances gives exactly the wrong answer.
- Fixed-parameter tractability confines the exponential to k : $2^k \cdot (n+m)$ rather than n^k . Vertex cover is FPT; independent set is $W[1]$ -hard despite being the same problem.
- Restricting the input class is frequently the best of the three responses, which is what Part IV was for.

Chapter 24 — The probabilistic method

This is the hinge of the book. Every proof so far has been constructive: it built the object, or gave an algorithm that would. From here the arguments stop doing that.

The method, due to Erdős, is one sentence: **to prove an object exists, put a probability distribution on a space containing it and show the probability of a suitable object is positive.** You never exhibit one. You show one must be there.

The smallest possible example

Theorem. Every graph has a bipartite subgraph with at least $m/2$ edges.

Proof. Colour each vertex red or blue independently with probability $1/2$. An edge crosses if its endpoints differ, which happens with probability $1/2$. By linearity of expectation, the expected number of crossing edges is $m/2$.

A random variable is at least its expectation with positive probability. So some colouring achieves at least $m/2$ crossing edges, and the crossing edges form a bipartite subgraph. ■

Read that again for what it does not do. It does not tell you which colouring. It does not give an algorithm. It asserts that among 2^n colourings, at least one is good, on the grounds that the average is good.

Linearity of expectation is doing all the work, and it is the method's most-used tool precisely because it needs no independence. The edges here are *not* independent — two edges sharing a vertex are correlated — and it does not matter at all.

```
held      ch24  Every graph has a bipartite subgraph with at least m/2 edges  (52
graphs)
```

Derandomising

The proof is non-constructive, but this particular one can be repaired. Process vertices in order, and place each on whichever side gives more crossing edges *so far*:

```
def greedy_cut(g):
    side = {}
    for v in g.vertices():
        zero = sum(1 for w in g.neighbours(v) if side.get(w) == 0)
        one = sum(1 for w in g.neighbours(v) if side.get(w) == 1)
        side[v] = 1 if zero >= one else 0
    return sum(1 for u, v in g.edges() if side[u] != side[v])
```

This is the **method of conditional expectations**: at each step, choose the option whose conditional expectation is at least the current expectation. Since the expectation starts at $m/2$ and never decreases, the final deterministic answer is at least $m/2$.

Not every probabilistic proof derandomises this cleanly, and the ones that do not are the interesting ones.

Ramsey lower bounds

The classic application, and the one that shows the method's real power.

Theorem (Erdős, 1947). If $C(n, k) \cdot 2^{1-C(k, 2)} < 1$, then there is a 2-colouring of K_n with no monochromatic K_k . Hence $R(k, k) > 2^{\lfloor k/2 \rfloor}$ for $k \geq 3$.

Proof. Colour each edge red or blue independently at random. For a fixed set of k vertices, the probability that it is monochromatic is $2 \cdot 2^{-C(k, 2)}$. There are $C(n, k)$ such sets, so by the union bound the probability that *some* set is monochromatic is at most $C(n, k) \cdot 2^{1-C(k, 2)}$.

If that is less than 1, the probability of no monochromatic set is positive, so such a colouring exists. ■

This is where the method earns its reputation. The bound $R(k, k) > 2^{\lfloor k/2 \rfloor}$ was, in 1947, a dramatic improvement on anything known — and **nobody has ever constructed** a colouring achieving anything close to it. Sixty years of effort has produced explicit constructions far weaker than what a paragraph of averaging gives. The gap between what we can prove exists and what we can build is, in this problem, enormous and permanent-looking.

Chapter 28 gives the upper bound and the resulting state of ignorance about $R(5, 5)$.

The three tools

Almost every argument in this style uses one of three ideas.

Linearity of expectation. $E[X + Y] = E[X] + E[Y]$, always, with no independence assumption. Used above for the cut.

The union bound. $P(\cup A_i) \leq \sum P(A_i)$, always. Used above for Ramsey. Crude, and usually enough.

The deletion method. Show a random object has few bad parts *on average*, then delete one vertex per bad part. What remains is good and still large. This gives, for example, that a graph with n vertices and m edges has an independent set of size at least $n^2/(4m)$ — take each vertex with probability p , delete one endpoint of each surviving edge, and optimise p .

When these three fail, the machinery gets much heavier — the Lovász local lemma, martingale concentration, the second moment method — and Chapter 26 needs the last of those.

What changes from here

The rest of Part VI is written in this style, and it is worth being explicit about the shift.

The verification harness cannot help in the same way it did. A claim like "there exists a colouring with no monochromatic K_k " is checkable only by searching all colourings, which is exactly what the method exists to avoid. Chapter 28 checks $R(3, 3) = 6$ exhaustively because 32,768 colourings is small; $R(5, 5)$ involves more colourings than there are atoms in the observable universe, and no amount of computation will settle it.

So from here the book's claims are checkable at small n and the theorems are about large n , and the gap between those is a gap the harness cannot close. Chapters 25 and 26 handle this by measuring *sharpness* rather

than truth — showing the transition is narrow, which is a finite fact — and saying plainly that a limit statement is not what is being tested.

Try it

```
python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.core import petersen, complete
from graphs.extremal import max_cut_bruteforce, greedy_cut, random_bipartition_cut
rng = random.Random(4)
for name, g in [('K5', complete(5)), ('petersen', petersen())]:
    trials = [random_bipartition_cut(g, rng) for _ in range(1000)]
    print(f'{name}: m={g.m}  m/2={g.m/2}')
    print(f'    random cut, mean over 1000 trials: {sum(trials)/1000:.2f}')
    print(f'    greedy (derandomised):                {greedy_cut(g)}')
    print(f'    true maximum:                            {max_cut_bruteforce(g)}')
"
```

```
K5: m=10  m/2=5.0
    random cut, mean over 1000 trials: 5.01
    greedy (derandomised):                6
    true maximum:                        6
petersen: m=15  m/2=7.5
    random cut, mean over 1000 trials: 7.41
    greedy (derandomised):                12
    true maximum:                        12
```

The random means land near $m/2$ — 5.01 against 5.0, and 7.41 against 7.5 — exactly as linearity of expectation predicts. The derandomised greedy comfortably beats the bound, reaching the true maximum on both graphs here. That is normal and is not a guarantee: the theorem promises $m/2$ and nothing more, and a heuristic that usually does better is precisely the situation Chapter 23 warned about.

Exercises

1. Prove that every graph has a bipartite subgraph with at least $m/2$ edges, in your own words, and say where independence would have been needed if you had used it.
2. A tournament is a complete graph with each edge directed. Show that some tournament on n vertices has at least $n!/2^{n-1}$ Hamiltonian paths.
3. Use the deletion method to show a graph with n vertices and $m \geq n/2$ edges has an independent set of size at least $n^2/(4m)$.
4. Why does the $m/2$ cut bound derandomise easily, while the Ramsey lower bound does not?

Solutions in Appendix E.

Takeaways

- To prove existence, show a random object works with positive probability. You never build one.
- Linearity of expectation needs no independence, which is why it is the workhorse.
- The union bound is crude and usually sufficient; the deletion method handles "mostly good" objects.

- $R(k, k) > 2^{\{k/2\}}$ follows from a paragraph of averaging, and no explicit construction has ever come close. That gap is the method's clearest demonstration.
- From here the theorems are asymptotic and the harness is finite. The book measures sharpness, and does not pretend that is the same as proof.

Chapter 25 — Random graphs

$G(n, p)$ is the graph on n vertices where each of the $C(n, 2)$ possible edges appears independently with probability p . Erdős and Rényi's insight was that this is not merely a source of test cases — it has a rich theory, and almost every property appears *suddenly* as p grows.

Thresholds

A property is **monotone increasing** if adding edges cannot destroy it: connectivity, having a triangle, containing a Hamiltonian cycle.

Theorem (Bollobás–Thomason, 1987). Every monotone increasing property has a threshold function $p^*(n)$: if $p \ll p^*$ the property holds with probability tending to 0, and if $p \gg p^*$ with probability tending to 1.

Every monotone property has one. The question is always *where*, never *whether*.

Property	Threshold
Contains a triangle	$1/n$
Contains any fixed H (balanced)	$n^{-\{v(H)/e(H)\}}$
Giant component appears	$1/n$ (Chapter 26)
No isolated vertices	$\ln n / n$
Connected	$\ln n / n$
Hamiltonian	$\ln n / n$

The last three sharing a threshold is not a coincidence, and it is the most instructive row in the table. Below $\ln n / n$ a random graph has isolated vertices; that alone makes it disconnected and non-Hamiltonian. Above it, the isolated vertices vanish — and it turns out that *nothing else* was ever the obstruction. **The last isolated vertex disappearing is essentially the same moment the graph becomes Hamiltonian**, which is a far stronger statement than connectivity and comes for free.

The first moment method

The expected number of triangles in $G(n, p)$ is $C(n, 3) p^3 \approx n^3 p^3 / 6$.

If $p \ll 1/n$ this tends to 0, and since the triangle count is a non-negative integer, Markov's inequality gives $P(\text{at least one triangle}) \leq E[\text{count}] \rightarrow 0$. That is the **first moment method**: expectation going to zero proves the object usually does not exist.

The converse needs more. Expectation going to *infinity* does not prove existence — a variable can have a huge mean and still be zero almost always, if it is occasionally enormous. Ruling that out requires the variance, which is the **second moment method**, and Chapter 26 uses it.

That asymmetry is worth internalising: **first moment for non-existence, second moment for existence**. Getting it backwards is the standard error in this subject.

Measuring sharpness

A threshold is an asymptotic statement, so no finite experiment can confirm it. What an experiment *can* show is that the transition is narrow — a fact about $n = 400$ rather than about the limit.

```
python scripts/random_graph_experiments.py
```

```
Connectivity, n = 400, p = c ln(n)/n, 40 trials per row
```

c	connected fraction
0.4	0.00
0.6	0.00
0.8	0.03
1.0	0.47
1.2	0.65
1.5	0.93
2.0	1.00

At $c = 0.6$ essentially nothing is connected; at $c = 2.0$ everything is. The whole transition happens within a factor of about three in p , and at $c = 1$ — exactly the predicted threshold — it is near a coin flip.

This is not evidence that the threshold is $\ln n / n$. It is consistent with it, at one value of n . The theorem is about $n \rightarrow \infty$, and a single n cannot distinguish $\ln n / n$ from any function that happens to be close to it at 400. What the table honestly shows is sharpness, and sharpness is the surprising part — a property that goes from never to always over a narrow band is not what naive intuition predicts.

Almost every graph

Random graph results are usually stated as facts about "almost every graph", and the translation is worth making explicit. $G(n, 1/2)$ is the uniform distribution over all labelled graphs on n vertices, since every graph has probability $2^{-\binom{n}{2}}$. So a property holding **with high probability** in $G(n, 1/2)$ is a property of almost every graph.

Several such facts are startling:

- Almost every graph has diameter 2.
- Almost every graph has a trivial automorphism group — no symmetries at all.
- Almost every graph has clique number about $2 \log_2 n$, concentrated on **two values**.

That last one is remarkable. $\omega(G)$ for a random graph is not merely near $2 \log_2 n$; it is one of two specific integers with probability tending to 1. Yet finding a clique of size $(1+\epsilon) \log_2 n$ in a random graph is a famous open problem — greedy finds $\log_2 n$ and nothing does better. **We know precisely how large the answer is and cannot find it**, which is the same gap Chapter 24 identified for Ramsey, in a different disguise.

Try it

```
python -c "
import sys, random, math; sys.path.insert(0, '.')
from graphs.generate import random_graph
from graphs.algorithms import distances
rng = random.Random(2)
n = 200
for p in (0.02, 0.05, 0.1, 0.3):
    diam = 0
    g = random_graph(n, p, rng)
    for v in g.vertices():
        d = distances(g, v)
        if len(d) == n:
            diam = max(diam, max(d.values()))
        else:
            diam = -1; break
    print(f'p={p:<5} m={g.m:<6} diameter={diam if diam > 0 else \"disconnected\"}')
"
```

```
p=0.02  m=407    diameter=disconnected
p=0.05  m=1006   diameter=4
p=0.1   m=1925   diameter=3
p=0.3   m=5970   diameter=2
```

At $p = 0.3$ the diameter is already 2, and it will stay there — the "almost every graph has diameter 2" result arriving well before $p = 1/2$.

Exercises

1. Compute the expected number of triangles in $G(n, p)$ and find the threshold at which it stops tending to zero.
2. Why does the expected number of copies of H tending to infinity fail to prove H appears? Give the shape of a counterexample.
3. Show that the expected number of isolated vertices in $G(n, p)$ is $n(1-p)^{n-1}$, and find where it tends to a constant.
4. $G(n, 1/2)$ is uniform over labelled graphs. Why does that make "with high probability" statements into statements about almost every graph?

Solutions in Appendix E.

Takeaways

- Every monotone increasing property has a threshold; the question is where it sits.
- Connectivity, no isolated vertices, and Hamiltonicity share the threshold $\ln n / n$. The last isolated vertex vanishing is essentially the moment the graph becomes Hamiltonian.
- First moment (expectation $\rightarrow 0$) proves non-existence. Existence needs the second moment, because a large mean can hide a variable that is usually zero.

- A finite experiment cannot confirm an asymptotic threshold. It can show the transition is sharp, which is a different and still-surprising claim, and the book says which it is showing.
- Almost every graph has diameter 2, no symmetries, and clique number concentrated on two values — which we cannot find.

Chapter 26 — The giant component

At $p = c/n$, something abrupt happens to $G(n, p)$ as c crosses 1. Below it every component is tiny. Above it there is exactly one enormous component and everything else is still tiny. This is the most studied phase transition in combinatorics, and it is worth understanding both what happens and why the mechanism is so simple.

The theorem

Theorem (Erdős–Rényi, 1960). Let $p = c/n$.

- If $c < 1$, the largest component has size $O(\log n)$ with high probability.
- If $c > 1$, there is a unique component of size $\approx \beta n$ where β is the positive root of $\beta = 1 - e^{-c\beta}$, and every other component is $O(\log n)$.
- If $c = 1$, the largest component has size $\Theta(n^{2/3})$ — neither logarithmic nor linear.

Three regimes, and the third is the strangest: at exactly the critical point the answer is a fractional power, which is why the critical window is studied separately and in far more detail than either side.

Why: the branching process

The mechanism is a heuristic first and a proof second, and the heuristic is the part to remember.

Explore a component by BFS from a vertex. Each vertex you reach has about n potential neighbours, each present with probability c/n , so it produces about c **new vertices** in expectation. Early in the exploration almost nothing has been visited, so the process looks like a **branching process with mean offspring c** .

The theory of branching processes then supplies the answer, and it is a dichotomy:

- $c < 1$ (subcritical): extinction with probability 1, and the expected total progeny is $1/(1-c)$, a constant. Components are tiny.
- $c > 1$ (supercritical): survival with probability $\beta > 0$, where β solves $\beta = 1 - e^{-c\beta}$. A positive fraction of starting vertices produce unbounded growth, and they all end up in the same component.
- $c = 1$ (critical): extinction with probability 1, but the expected time to extinction is infinite. This is why the critical case is delicate.

$c = 1$ is where each vertex replaces itself exactly once. Below, the exploration dies out; above, it escapes. Everything in this chapter is that one sentence.

The branching approximation is not exact — the graph has only n vertices, so the process must eventually run out — and turning it into a proof needs the second moment method (Chapter 25) to show the large component's size is concentrated, plus a separate argument for uniqueness. Uniqueness has a nice heuristic too: two components of size εn each have $\varepsilon^2 n^2$ potential edges between them, and at $p = c/n$ they would be joined with probability tending to 1.

Measuring it

```
python scripts/random_graph_experiments.py
```

Giant component, $n = 400$, 5 trials per row

$c = pn$	largest/ n	2nd/ n	#comps
0.4	0.014	0.013	317.6
0.6	0.035	0.024	280.0
0.8	0.052	0.039	237.0
0.9	0.096	0.070	202.4
1.0	0.142	0.044	199.4
1.1	0.217	0.066	176.4
1.2	0.324	0.056	157.6
1.5	0.581	0.023	116.4
2.0	0.793	0.011	65.6
3.0	0.937	0.006	24.2

Two columns, and the second is the one to read.

The largest component grows steadily from 1.4% to 94% of the graph — but that is a gradual-looking curve, and at $n = 400$ the transition is genuinely blurred. The theorem is asymptotic; $\log n \approx 6$ and $n^{2/3} \approx 54$ and $n = 400$ are simply not far enough apart at this size to look like three regimes.

The second-largest component is the sharp signal. It rises to 7% of the graph at $c = 0.9$, then falls away — 4.4% at $c = 1.0$, 2.3% at $c = 1.5$, 1.1% at $c = 2.0$, 0.6% at $c = 3.0$. That non-monotone peak near $c = 1$ is the transition making itself visible: below the threshold the largest two components are comparable, above it the largest runs away and leaves everything else behind.

Compare the $c = 0.4$ row — largest 1.4%, second 1.3%, essentially the same — with $c = 3.0$ — largest 94%, second 0.6%. The *ratio* between first and second is the order parameter, and it is what changes character, not the size of the largest alone.

At $n = 400$ with 5 trials these numbers are noisy, and the point is the shape rather than any individual figure. A finite experiment cannot confirm an asymptotic theorem; what it can do is show you where to look.

Why it matters outside graph theory

The same transition, with the same $c = 1$ mechanism, governs:

- **Percolation:** fluid spreading through porous rock, above a critical density.
- **Epidemics:** the basic reproduction number R_0 is exactly c , and $R_0 = 1$ is exactly this threshold. The giant component is the outbreak.
- **Network resilience:** how many nodes can fail before a network fragments.
- **Random SAT:** satisfiability has an analogous sharp threshold in the clause-to-variable ratio.

The claim that an epidemic either dies out or reaches a constant fraction of the population, with nothing in between, is this theorem. The branching heuristic — each case produces R_0 new cases — is the same one, and it is why $R_0 = 1$ is the number everyone watches.

Try it

Watch the second-largest component peak and fall:

```
python -c "
import sys, random; sys.path.insert(0, '.')
from graphs.generate import random_graph
from graphs.algorithms import components
rng = random.Random(11)
n, trials = 800, 15
for c in (0.5, 0.9, 1.0, 1.1, 1.5, 2.5):
    first = second = 0
    for _ in range(trials):
        s = sorted((len(x) for x in components(random_graph(n, c/n, rng))),
reverse=True)
        first += s[0]; second += s[1] if len(s) > 1 else 0
    a, b = first/trials, second/trials
    print(f'c={c:<4} largest={a:<7.1f} second={b:<6.1f} ratio={a/max(b,1):.1f}')
```

```
c=0.5 largest=11.2 second=8.4 ratio=1.3
c=0.9 largest=51.3 second=26.0 ratio=2.0
c=1.0 largest=106.5 second=29.9 ratio=3.6
c=1.1 largest=128.1 second=45.1 ratio=2.8
c=1.5 largest=470.1 second=11.9 ratio=39.6
c=2.5 largest=717.0 second=3.3 ratio=219.5
```

The ratio is the story: near 1 below the threshold, and climbing without bound above it.

This needs averaging over fifteen trials, and that is itself the finding. A single run per row produces a non-monotone mess — in one attempt `c = 1.1` gave a *lower* ratio than `c = 1.0`. That is not experimental sloppiness; near the critical point the component sizes genuinely have enormous variance, which is precisely why the `c = 1` case has its own $\Theta(n^{2/3})$ theorem and its own literature. If your measurements near a phase transition look noisy, the noise may be the phenomenon.

Exercises

1. Solve $\beta = 1 - e^{-c\beta}$ numerically for `c = 1.5` and compare with the measured 0.581.
2. Why does the branching-process heuristic break down once the explored set is a constant fraction of the graph?
3. Give the argument that two giant components cannot coexist.
4. In epidemic terms, what does `c < 1` mean, and what does the second-largest component correspond to?

Solutions in Appendix E.

Takeaways

- At $p = c/n$: all components $O(\log n)$ for $c < 1$; a unique βn component for $c > 1$; $\Theta(n^{2/3})$ exactly at `c = 1`.

- The mechanism is a branching process with mean offspring c . $c = 1$ is where each vertex replaces itself exactly once — that single sentence is the whole chapter.
- β solves $\beta = 1 - e^{-c\beta}$, which comes from the extinction probability of that process.
- At $n = 400$ the largest component's growth looks gradual. The **ratio of largest to second-largest** is the sharp signal, and the second-largest peaking near $c = 1$ is the transition becoming visible at finite size.
- The same transition is percolation, $R_0 = 1$ in epidemics, and the random-SAT threshold.

Chapter 27 — Extremal graph theory

One question, asked over and over: **how many edges can a graph have before some structure is forced to appear?** Chapter 16 showed triangle-free does not imply bipartite; this chapter asks how many edges triangle-freeness costs you.

Mantel

Theorem (Mantel, 1907). A triangle-free graph on n vertices has at most $n^2/4$ edges, and the complete bipartite graph $K_{\lfloor n/2 \rfloor, \lfloor n/2 \rfloor}$ is the unique extremal example.

Proof. Let uv be an edge. Since G is triangle-free, u and v have no common neighbour, so $\deg(u) + \deg(v) \leq n$. Summing over all edges,

$$\sum_{uv \in E} (\deg(u) + \deg(v)) \leq mn.$$

The left side counts each vertex v once for each edge at it, contributing $\deg(v)^2$. So $\sum_v \deg(v)^2 \leq mn$. By Cauchy–Schwarz, $\sum \deg(v)^2 \geq (\sum \deg(v))^2/n = 4m^2/n$. Combining, $4m^2/n \leq mn$, so $m \leq n^2/4$. ■

Two techniques in five lines: **double counting** (Chapter 3, fourth appearance) and **Cauchy–Schwarz**, which is the standard way to turn a sum of squares into a bound. Both recur throughout this part of the book.

held ch27 Mantel: a triangle-free graph has $m \leq n^2/4$ (27 graphs)

Turán

The generalisation replaces the triangle with K_{r+1} .

Theorem (Turán, 1941). A graph with no K_{r+1} has at most $(1 - 1/r) n^2/2$ edges, and the unique extremal graph is the **Turán graph** $T(n, r)$: the complete r -partite graph with parts as equal as possible.

Mantel is the case $r = 2$.

The extremal example is worth staring at. To avoid K_{r+1} while having as many edges as possible, split the vertices into r groups and join everything *between* groups. Any clique picks at most one vertex per group, so cliques have size at most r . Making the parts equal maximises the edge count — an application of the same convexity that Cauchy–Schwarz encodes.

The harness checks this by enumerating **every** graph on n vertices and finding the true maximum, rather than trusting the formula:

```
def max_edges_without_clique(n, k):
    pairs = list(itertools.combinations(range(n), 2))
    best = 0
    for mask in range(1 << len(pairs)):
        chosen = [p for i, p in enumerate(pairs) if mask >> i & 1]
        if len(chosen) > best and not has_clique(Graph(n, chosen), k):
            best = len(chosen)
    return best
```

```
held      ch27 Turan: the K_{r+1}-free maximum is exactly the Turan graph's edge
count    (8 graphs)
```

$2^{\binom{n}{2}}$ graphs, so this stops at $n = 5$. It is enough to catch a misstated formula, which is what it is for.

Erdős–Stone: the general answer

Turán handles complete graphs. What about forbidding an arbitrary H ?

Theorem (Erdős–Stone, 1946). For any graph H with chromatic number $\chi(H) = r + 1$, the maximum number of edges in an H -free graph on n vertices is

$$(1 - 1/r) n^2/2 + o(n^2).$$

This is sometimes called the fundamental theorem of extremal graph theory, and the reason is its scope: **the answer depends on H only through its chromatic number**. Forbidding the Petersen graph and forbidding K_4 give the same leading term, because both have $\chi = 3$. Every detail of H 's structure is invisible at this resolution.

There is one enormous exception, and it is where the subject's hard open problems live. If $\chi(H) = 2$ — that is, H is **bipartite** — the theorem gives $o(n^2)$ and says nothing further. Determining the true order for bipartite H is the **Zarankiewicz problem**, and it is open in general. Even $H = C_8$ is not fully resolved. The known cases:

- $H = C_4$: the answer is $\frac{1}{2} n^{\frac{3}{2}} (1 + o(1))$, and the extremal graphs come from finite geometry.
- $H = K_{\{3,3\}}$: $\Theta(n^{\frac{5}{3}})$, upper and lower bounds matching only up to constants.

So extremal graph theory is essentially solved for non-bipartite H and essentially open for bipartite H . That is an unusual shape for a field, and worth knowing before you go looking for open problems.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.extremal import max_edges_without_clique, turan_bound, turan_graph
print(f\" {n:>3} {r:>3} {exhaustive max:>15} {Turan bound:>12} {(1-1/
r)n^2/2:>14}\")
for n in range(3, 6):
    for r in (2, 3):
        exact = max_edges_without_clique(n, r + 1)
        print(f\" {n:>3} {r:>3} {exact:>15} {turan_bound(n, r):>12} {(1 - 1/r) * n
* n / 2:>14.2f}\")
"
```

n	r	exhaustive max	Turan bound	$(1-1/r)n^2/2$
3	2	2	2	2.25
3	3	3	3	3.00
4	2	4	4	4.00
4	3	5	5	5.33
5	2	6	6	6.25
5	3	8	8	8.33

The exhaustive maximum matches the Turán graph's edge count exactly, every time. The final column — the clean $(1 - 1/r)n^2/2$ formula — is an *upper bound*, tight only when r divides n . At $n = 4$, $r = 2$ it is exact at 4; at $n = 5$, $r = 2$ it says 6.25 against a true 6, and at $n = 5$, $r = 3$ it says 8.33 against a true 8.

The gaps are small here, and they are the reason extremal results are stated in terms of the extremal *object* rather than the rounded formula. The Turán graph's edge count is the theorem; $(1 - 1/r)n^2/2$ is a convenient asymptotic that is never smaller and rarely equal.

Exercises

1. Verify Mantel's bound for $n = 5$ by finding a triangle-free graph with 6 edges.
2. Why does making the Turán graph's parts equal maximise the edge count? Use convexity.
3. Erdős–Stone says the answer depends on H only through $\chi(H)$. Find the chromatic numbers of K_3 , K_4 and the Petersen graph, and say which two of the three give the same leading term.
4. Why does Erdős–Stone say nothing useful when H is bipartite?

Solutions in Appendix E.

Takeaways

- Mantel: triangle-free implies $m \leq n^2/4$, extremal at the balanced complete bipartite graph. Proved by double counting plus Cauchy–Schwarz.
- Turán: no K_{r+1} implies $m \leq (1 - 1/r)n^2/2$, extremal at the balanced complete r -partite graph.
- The clean formula is an upper bound; the exact answer is the Turán graph's edge count, and the two differ when r does not divide n .
- Erdős–Stone: for any H , the answer depends only on $\chi(H)$. All structure beyond the chromatic number is invisible.
- When $\chi(H) = 2$ the theorem is silent, and that silence is the Zarankiewicz problem — where the subject's open questions live.

Chapter 28 — Ramsey theory

Chapter 27 asked how many edges force a structure. Ramsey theory asks something stronger: how large must a graph be before **any** 2-colouring of its edges contains monochromatic order. The answer is always "finite", and almost never "known".

$R(3,3) = 6$

Theorem. Every 2-colouring of the edges of K_6 contains a monochromatic triangle, and some 2-colouring of K_5 does not.

Proof. Take any vertex v in K_6 . It has 5 edges, so by pigeonhole at least 3 share a colour — say v is joined in red to a, b, c . If any of ab, bc, ac is red, that edge with v forms a red triangle. If none is, then abc is a blue triangle. ■

That is the whole proof, and it is the friendliest theorem in this part of the book: one pigeonhole and one case split. It is also the standard "party puzzle" — among any six people, three are mutual acquaintances or three are mutual strangers.

For the lower bound, colour K_5 red on a 5-cycle and blue on the complementary 5-cycle. Neither colour class contains a triangle, because C_5 is triangle-free and self-complementary.

Both halves are checkable exhaustively, and this is one of the few places in Part VI where the harness can settle a real question rather than merely spot-check a statement:

```
held      ch28 R(3,3) = 6: every 2-colouring of K_6 has a monochromatic
triangle  (3 graphs)
```

32,768 colourings of K_6 , all checked; 1,024 of K_5 , and a counterexample found. Together those establish $R(3,3) = 6$ completely. **This is a proof, not a spot check** — the search is exhaustive over the entire space the theorem quantifies over, which is the only situation in this book where a finite computation settles a theorem outright.

The general theorem

Theorem (Ramsey, 1930). For all s, t there is a least $R(s, t)$ such that every 2-colouring of $K_{R(s, t)}$ contains a red K_s or a blue K_t .

The existence proof gives $R(s, t) \leq R(s-1, t) + R(s, t-1)$, hence $R(s, s) \leq 4^s$. The probabilistic method of Chapter 24 gives $R(s, s) > 2^{\lfloor s/2 \rfloor}$.

Both bounds are from the 1930s and 40s. **The base of the exponential is still unknown.** In eighty years the gap between $\sqrt{2}$ and 4 has been narrowed only in the lower-order terms — a 2023 result improved the upper bound to 3.993^s , which was major news and did not change the picture.

The known values

	$t=3$	$t=4$	$t=5$	$t=6$
$s=3$	6	9	14	18
$s=4$		18	25	36–40
$s=5$			43–46	58–85

$R(5, 5)$ is unknown. It is known to be between 43 and 46, and that is where the table stops being a table.

The reason is scale. Deciding $R(5, 5) = 43$ means checking every 2-colouring of K_{43} , of which there are 2^{903} . That is not a matter of waiting for faster computers: 2^{903} exceeds the number of atoms in the observable universe raised to a substantial power. No conceivable computation touches it, and no better idea has been found.

Erdős's remark is the standard summary, and it is not really a joke: if aliens demanded $R(5, 5)$ on pain of destruction, we should marshal all our computers and mathematicians and try to find it; if they demanded $R(6, 6)$, we should attempt to destroy the aliens.

The pattern beyond graphs

Ramsey's theorem is one instance of a general phenomenon: **complete disorder is impossible**. Any sufficiently large structure contains a large ordered substructure, whatever "ordered" means in context.

- **Van der Waerden:** any 2-colouring of $\{1, \dots, N\}$ contains a monochromatic arithmetic progression of length k , for N large enough.
- **Szemerédi:** any subset of the integers with positive upper density contains arbitrarily long arithmetic progressions.
- **Green–Tao:** the primes contain arbitrarily long arithmetic progressions — despite having density zero, so Szemerédi does not apply directly.
- **Hales–Jewett:** high-dimensional tic-tac-toe cannot be drawn.

All share the same shape, and all share the same defect: the bounds are astronomically bad. Van der Waerden's original bound was not primitive recursive. Szemerédi's regularity lemma — the main tool — has bounds that are towers of exponentials, and Gowers proved that is necessary rather than an artefact.

Ramsey-type theorems tell you a structure exists and give you no way to find it. That is the same complaint Chapter 24 made about the probabilistic method, and it is not a coincidence: the lower bounds are probabilistic and the upper bounds are Ramsey-type, and neither is constructive.

Try it

```
python -c "
import sys, time; sys.path.insert(0, '.')
from graphs.extremal import ramsey_holds, ramsey_counterexample
t0 = time.perf_counter()
print('every colouring of K_6 has a mono triangle:', ramsey_holds(6, 3, 3))
witness = ramsey_counterexample(5, 3, 3)
print('K_5 counterexample exists: ', witness is not None)
print('  its red edges:', sorted(witness.edges()))
print('  red graph is a 5-cycle, blue is its complement - also a 5-cycle')
print(f'  ({time.perf_counter() - t0:.1f}s for 32768 + 1024 colourings)')
"
```

```
every colouring of K_6 has a mono triangle: True
K_5 counterexample exists: True
  its red edges: [(0, 3), (0, 4), (1, 2), (1, 4), (2, 3)]
  red graph is a 5-cycle, blue is its complement - also a 5-cycle
  (0.5s for 32768 + 1024 colourings)
```

Half a second settles $R(3,3)$. Scaling that approach to $R(5,5)$ would take 2^{903} colourings, and the difference between those two numbers is the entire difficulty of the subject.

Exercises

1. Prove $R(3,3) \leq 6$ using the pigeonhole argument, in your own words.
2. Show the K_5 counterexample really has no monochromatic triangle, by checking both colour classes.
3. Use $R(s,t) \leq R(s-1,t) + R(s,t-1)$ to derive $R(3,4) \leq 10$. (The true value is 9.)
4. Why can no computation settle $R(5,5)$ by exhaustive search? Give the number of colourings.

Solutions in Appendix E.

Takeaways

- $R(3,3) = 6$, provable by one pigeonhole step, and settled here by exhaustive search over all 32,768 colourings — the one place in this book where a finite computation is a proof.
- $2^{\lceil s/2 \rceil} < R(s,s) < 4^s$, both bounds from the 1930s–40s, and the base is still unknown after eighty years.
- $R(5,5)$ is known only to lie in $[43, 46]$. Settling it by search needs 2^{903} colourings, so it will not be settled by search.
- The general pattern — van der Waerden, Szemerédi, Green–Tao — is that complete disorder is impossible. All of them have astronomically bad bounds, provably so.
- Like the probabilistic method, Ramsey theory proves existence and offers no construction.

Chapter 29 — Spectral graph theory

Chapter 4 observed that A^k counts walks and left the matrix there. This chapter takes the matrix seriously: a graph's adjacency matrix is real and symmetric, so it has n real eigenvalues, and those numbers encode a surprising amount of structure.

Forty lines of eigenvalue solver

This book has no dependencies, so it needs its own solver. For real symmetric matrices the **Jacobi rotation method** is short, unconditionally convergent, and readable.

The idea: find the largest off-diagonal entry and rotate in that plane to zero it. Each rotation reduces the sum of squares of off-diagonal entries, so the matrix converges to a diagonal one, whose entries are the eigenvalues.

```
for _ in range(max_sweeps):
    off, p, q = 0.0, 0, 0
    for i in range(n):
        for j in range(i + 1, n):
            if abs(a[i][j]) > off:
                off, p, q = abs(a[i][j]), i, j
    if off < tolerance:
        break
    theta = (math.pi / 4 if a[p][p] == a[q][q]
            else 0.5 * math.atan2(2 * a[p][q], a[p][p] - a[q][q]))
    c, s = math.cos(theta), math.sin(theta)
    # rotate rows p, q and then columns p, q
```

Slower than what a library uses — $O(n^3)$ per sweep — and it is the one you can read. It is also only valid for symmetric input; feeding it anything else produces nonsense rather than an error, which the module docstring says explicitly.

Verifying an eigenvalue solver needs care: comparing it against another solver you also wrote proves nothing. The harness instead uses **trace identities**, which are facts about the graph rather than about linear algebra:

- $\sum \lambda_i = \text{trace}(A) = 0$, since the diagonal is all zeros;
- $\sum \lambda_i^2 = \text{trace}(A^2) = 2m$, since $(A^2)_{vv}$ is the number of closed walks of length 2, which is $\deg(v)$;
- $\sum \lambda_i^3 = \text{trace}(A^3) = 6 \cdot (\text{number of triangles})$, since a closed walk of length 3 is a triangle traversed from one of 3 starting points in one of 2 directions.

held	ch29	The adjacency spectrum sums to 0 and its squares sum to 2m (52 graphs)
held	ch29	The cube of the spectrum sums to six times the triangle count (52 graphs)

The third identity is the good one: the triangle count on the right is computed by direct enumeration, so a spectrum that satisfies it is being checked against combinatorics, not against more linear algebra.

What the spectrum knows

```

K4      spectrum [-1.0, -1.0, -1.0, 3.0]
C4      spectrum [-2.0, -0.0, 0.0, 2.0]
C5      spectrum [-1.618, -1.618, 0.618, 0.618, 2.0]
petersen spectrum [-2.0, -2.0, -2.0, -2.0, 1.0, 1.0, 1.0, 1.0, 1.0, 3.0]
K33     spectrum [-3.0, -0.0, 0.0, 0.0, 0.0, 3.0]

```

Read the Petersen row: 3 once, 1 five times, -2 four times. A 10-vertex graph with only three distinct eigenvalues, which is extremely unusual and is one reason the graph is special — graphs with three distinct adjacency eigenvalues are **strongly regular**, and the Petersen graph is the smallest interesting one.

Several structural facts read straight off the spectrum:

- $\lambda_{\max}$ sits between the average and maximum degree: $2m/n \leq \lambda_{\max} \leq \Delta$. For a d -regular graph both bounds coincide, so $\lambda_{\max} = d$ exactly, with the all-ones eigenvector.
- The graph is bipartite if and only if the spectrum is symmetric about 0. C_4 gives $\{-2, 0, 0, 2\}$ and $K_{3,3}$ gives $\{-3, 0, 0, 0, 0, 3\}$; C_5 gives an asymmetric spectrum and is not bipartite.
- The number of connected components is the multiplicity of $\lambda_{\max}$ for a regular graph.

```

held      ch29 A d-regular graph has spectral radius exactly d (12 graphs)
held      ch29 The spectral radius lies between the average and maximum degree
(52 graphs)

```

What the spectrum does not know

The spectrum is an invariant, so it can prove two graphs are not isomorphic. Like every invariant in Chapter 5, it cannot prove they are.

Two graphs with the same spectrum are **cospectral**. The smallest cospectral pair is $K_{1,4}$ and $C_4 + K_1$ — a star and a square-plus-isolated-vertex, both with spectrum $\{-2, 0, 0, 0, 2\}$. They are obviously not isomorphic; one is connected.

Worse, this is not rare. **Almost all trees are cospectral with some other tree**, so for trees the spectrum is close to useless as a distinguisher. The exact fraction of all graphs that are determined by their spectrum is an open problem, and the numerical evidence suggests it is close to 1 for general graphs — which makes trees an unusually bad case.

This is the same story as colour refinement in Chapter 5: a cheap invariant, sound in one direction, and blind to a family of cases you can characterise.

Interlacing

The deepest elementary tool here is **eigenvalue interlacing**.

Theorem (Cauchy interlacing). If H is an induced subgraph of G on $n - 1$ vertices, with eigenvalues $\mu_1 \leq \dots \leq \mu_{n-1}$ and G having $\lambda_1 \leq \dots \leq \lambda_n$, then

$$\lambda_1 \leq \mu_1 \leq \lambda_2 \leq \mu_2 \leq \dots \leq \mu_{n-1} \leq \lambda_n.$$

Deleting a vertex cannot move the eigenvalues past each other. This yields bounds that are hard to get combinatorially — for instance the **ratio bound**, that an independent set in a d -regular graph has size at most $n \cdot (-\lambda_{\min})/(d - \lambda_{\min})$. For the Petersen graph: $10 \cdot 2/(3 + 2) = 4$, which is exactly $\alpha(\text{Petersen}) = 4$ as Chapter 21 computed.

A tight bound on an NP-hard quantity, from four eigenvalues, is a fair advertisement for the method.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import complete, cycle, petersen, complete_bipartite, Graph
from graphs.spectral import adjacency_spectrum

for name, g in [('C4', cycle(4)), ('C5', cycle(5)), ('K3,3',
complete_bipartite(3,3)),
                ('petersen', petersen())]:
    sp = [round(x, 3) for x in adjacency_spectrum(g)]
    symmetric = all(abs(sp[i] + sp[-1-i]) < 1e-6 for i in range(len(sp)))
    print(f'{name:<9} {sp}')
    print(f'{"\":<9} symmetric about 0: {symmetric} (bipartite iff this)')

print()
star = Graph(5, [(0,1),(0,2),(0,3),(0,4)])
other = Graph(5, [(0,1),(1,2),(2,3),(3,0)])
print('cospectral pair:')
print(' K_1,4      ', [round(x,3) for x in adjacency_spectrum(star)])
print(' C_4 + K_1    ', [round(x,3) for x in adjacency_spectrum(other)])
"
```

```
C4      [-2.0, -0.0, 0.0, 2.0]
        symmetric about 0: True (bipartite iff this)
C5      [-1.618, -1.618, 0.618, 0.618, 2.0]
        symmetric about 0: False (bipartite iff this)
K3,3    [-3.0, -0.0, 0.0, 0.0, 0.0, 3.0]
        symmetric about 0: True (bipartite iff this)
petersen [-2.0, -2.0, -2.0, -2.0, 1.0, 1.0, 1.0, 1.0, 3.0]
        symmetric about 0: False (bipartite iff this)

cospectral pair:
 K_1,4      [-2.0, -0.0, 0.0, 0.0, 2.0]
 C_4 + K_1    [-2.0, -0.0, 0.0, 0.0, 2.0]
```

Identical spectra; one graph is connected and the other is not. The spectrum knows a great deal and not everything.

Exercises

1. Compute the adjacency spectrum of K_n by hand. (Hint: $A = J - I$.)
2. Show that $\sum \lambda_i^2 = 2m$ follows from counting closed walks of length 2.
3. Verify the ratio bound on the Petersen graph and compare with $\alpha = 4$.
4. Why can the spectrum prove two graphs are non-isomorphic but never that they are?

Solutions in Appendix E.

Takeaways

- A graph's adjacency matrix is real symmetric, so the spectrum is real. Forty lines of Jacobi rotation computes it with no dependencies.
- Verify an eigenvalue solver against trace identities — $\sum \lambda = 0$, $\sum \lambda^2 = 2m$, $\sum \lambda^3 = 6 \cdot \text{triangles}$ — not against another solver.
- $\lambda_{\max}$ lies between average and maximum degree, with equality for regular graphs. Bipartite exactly when the spectrum is symmetric about 0.
- Cospectral non-isomorphic graphs exist and are common among trees. The spectrum is a one-sided invariant, like everything in Chapter 5.
- Interlacing gives bounds on combinatorial quantities from eigenvalues; the ratio bound is exactly tight on the Petersen graph.

Chapter 30 — The Laplacian

The adjacency matrix is the obvious matrix to attach to a graph. The **Laplacian** $L = D - A$, with D the diagonal degree matrix, is the useful one — and this chapter is about why that swap changes everything.

The quadratic form

The reason L is better is one identity:

Lemma. For any vector x , $x^T L x = \sum_{uv \in E} (x_u - x_v)^2$.

Proof. Expand: $x^T D x = \sum_v \deg(v) x_v^2$ and $x^T A x = 2 \sum_{uv} x_u x_v$. Subtracting, $x^T L x = \sum_v \deg(v) x_v^2 - 2 \sum_{uv} x_u x_v = \sum_{uv} (x_u^2 + x_v^2 - 2x_u x_v)$, since each vertex appears in $\deg(v)$ edge terms. ■

Everything follows from this. $x^T L x \geq 0$ always, so L is positive semidefinite and all eigenvalues are ≥ 0 . The form is zero exactly when x is constant on each connected component. So:

Theorem. The multiplicity of eigenvalue 0 in L equals the number of connected components.

```
K4      L-spectrum [0.0, 4.0, 4.0, 4.0]
C5      L-spectrum [0.0, 1.382, 1.382, 3.618, 3.618]
petersen L-spectrum [0.0, 2.0, 2.0, 2.0, 2.0, 2.0, 5.0, 5.0, 5.0, 5.0]
P4      L-spectrum [0.0, 0.5858, 2.0, 3.4142]
two triangles L-spectrum [0.0, 0.0, 3.0, 3.0, 3.0, 3.0]
```

The last row has 0 twice, and the graph has two components. Connectivity — a combinatorial property — is read off an eigenvalue multiplicity.

```
held      ch30 The multiplicity of Laplacian eigenvalue 0 is the number of
components (52 graphs)
```

The matrix–tree theorem

Chapter 7 promised this and could not deliver it.

Theorem (Kirchhoff, 1847). The number of spanning trees of G equals any cofactor of L — delete any one row and the corresponding column, and take the determinant.

```
def spanning_tree_count(g):
    lap = laplacian_matrix(g)
    minor = [row[1:] for row in lap[1:]]
    return round(determinant(minor))
```

$O(n^3)$, against Chapter 7's $C(m, n-1)$ enumeration. The harness checks one against the other, which is a determinant against a brute-force search — as independent as two computations get:

```
held      ch30 Matrix-tree: a Laplacian cofactor counts spanning trees (52
graphs)
```

And the promise is now kept. Chapter 7 could verify Cayley's formula only to K_6 :

```
K10 spanning trees: 100000000      10^8 = 100000000
```

K_{10} has 10^8 spanning trees, confirmed instantly from a 9×9 determinant. The Petersen graph has 2000.

Algebraic connectivity

The second-smallest eigenvalue λ_2 is the **algebraic connectivity**, or Fiedler value. It is zero exactly when the graph is disconnected, and larger when the graph is harder to split.

```
held      ch30 Algebraic connectivity is positive exactly when the graph is
connected (51 graphs)
```

Compare P_4 at 0.586 with K_4 at 4.0. Both are connected; one is a path that a single edge deletion breaks, the other is as robustly connected as four vertices allow. The Fiedler value measures that difference on a continuous scale, where $\kappa(G)$ and $\lambda(G)$ from Chapter 12 give integers.

Cheeger's inequality

The precise link between the spectrum and cutting the graph is the deepest result in this book that has a short statement.

Define the **Cheeger constant** — the isoperimetric number:

$$h(G) = \min \text{ over } S \text{ with } |S| \leq n/2 \text{ of } |\text{edges leaving } S| / |S|$$

Computing it means checking every subset, so it is exponential by definition. And yet:

Theorem (Cheeger; Alon–Milman). $\lambda_2/2 \leq h(G) \leq \sqrt{2 \cdot \Delta \cdot \lambda_2}$.

A two-sided bound on an exponentially hard quantity, from a single eigenvalue computable in $O(n^3)$. That is the entire theoretical foundation of spectral clustering.

```
held      ch30 Cheeger: a(G)/2 <= h(G) <= sqrt(2 * Delta * a(G)) (30 graphs)
```

The harness computes $h(G)$ by its exponential definition and λ_2 from the Laplacian, so the two sides come from unrelated computations. That is the only honest way to check an inequality between them.

The bound is not tight — there is a square root separating the two sides — and that gap is real, not an artefact of the proof. It is why spectral clustering gives good partitions rather than optimal ones.

Spectral clustering

The algorithm falls out. To split a graph in two:

1. compute the eigenvector for λ_2 — the **Fiedler vector**;
2. sort the vertices by their entry in it;
3. cut at the best of the $n - 1$ resulting splits.

The justification is the quadratic form again. Minimising $\mathbf{x}^T \mathbf{L} \mathbf{x}$ subject to $\mathbf{x} \perp \mathbf{1}$ and $\|\mathbf{x}\| = 1$ gives exactly the Fiedler vector, and $\mathbf{x}^T \mathbf{L} \mathbf{x} = \sum (\mathbf{x}_u - \mathbf{x}_v)^2$ is small precisely when adjacent vertices get similar values. So the Fiedler vector is the assignment of numbers to vertices that makes adjacent vertices as close as possible — a **relaxation** of the 0/1-valued cut problem, with the integrality dropped.

That is the general recipe worth carrying away: **relax the integer constraint, solve the continuous problem exactly, and round**. Cheeger's inequality is precisely the statement that rounding does not lose too much.

Try it

```
python -c "
import sys, math; sys.path.insert(0, '.')
from graphs.core import complete, cycle, path, petersen, Graph
from graphs.spectral import (algebraic_connectivity, cheeger_constant,
                             laplacian_spectrum, spanning_tree_count)
print(f\" {graph':<10} {'lambda2':>9} {'h(G)':>7} {'lambda2/2':>10}
{'sqrt(2*D*l2)':>13} {'trees':>7}\")
for name, g in [('P4', path(4)), ('C5', cycle(5)), ('K4', complete(4)),
('petersen', petersen())]:
    l2 = algebraic_connectivity(g); h = cheeger_constant(g)
    D = max(g.degree(v) for v in g.vertices())
    print(f' {name:<10} {l2:>9.4f} {h:>7.4f} {l2/2:>10.4f}
{math.sqrt(2*D*l2):>13.4f} {spanning_tree_count(g):>7}')
print()
two = Graph(6, [(0,1),(1,2),(2,0),(3,4),(4,5),(5,3)])
print(' disconnected graph, lambda2 =', round(algebraic_connectivity(two), 10))
"
```

graph	lambda2	h(G)	lambda2/2	sqrt(2*D*l2)	trees
P4	0.5858	0.5000	0.2929	1.5307	1
C5	1.3820	1.0000	0.6910	2.3511	5
K4	4.0000	2.0000	2.0000	4.8990	16
petersen	2.0000	1.0000	1.0000	3.4641	2000

```
disconnected graph, lambda2 = 0.0
```

Every row satisfies $\lambda_2/2 \leq h(G) \leq \sqrt{(2\Delta\lambda_2)}$, and the two bounds are far apart — for P_4 , 0.29 and 1.53 bracketing a true value of 0.50. That factor-of-five window is what "spectral clustering works but is not optimal" means quantitatively.

Note K_4 is the tight case on the left: $\lambda_2/2 = 2.0 = h(G)$.

Exercises

1. Prove $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum_{uv \in E} (x_u - x_v)^2$ by expanding, and deduce $\mathbf{L}$ is positive semidefinite.
2. Compute the Laplacian spectrum of K_3 by hand and check the matrix–tree theorem gives 3.
3. Why is the all-ones vector always an eigenvector of $\mathbf{L}$ with eigenvalue 0?
4. P_4 has $\lambda_2 = 0.586$ and K_4 has $\lambda_2 = 4$. Both are connected — what does the difference measure?

Solutions in Appendix E.

Takeaways

- $\mathbf{L} = \mathbf{D} - \mathbf{A}$, and $\mathbf{x}^\top \mathbf{L} \mathbf{x} = \sum (x_u - x_v)^2$ is the identity everything follows from.
- Zero-eigenvalue multiplicity equals the component count. Connectivity becomes linear algebra.
- Matrix–tree counts spanning trees by one $O(n^3)$ determinant, retiring Chapter 7's exponential enumeration: K_{10} has 10^8 spanning trees, computed instantly.
- λ_2 is zero exactly when disconnected, and measures how hard the graph is to cut.
- Cheeger brackets an exponentially hard quantity between two functions of one eigenvalue. The gap is a genuine square root, which is why spectral clustering is good and not optimal.
- The recipe generalises: relax the integrality, solve exactly, round.

Chapter 31 — Minors and treewidth

Chapter 17 characterised planarity by two forbidden minors. This chapter is about how far that idea goes, and it goes remarkably far — to a theorem that settles infinitely many questions at once and tells you the answer to none of them.

Minors

H is a **minor** of G if it can be obtained by deleting vertices, deleting edges, and contracting edges. A class of graphs is **minor-closed** if it is closed under those operations: planar graphs, forests, graphs embeddable on any fixed surface, graphs of treewidth at most k .

Kuratowski and Wagner gave planarity two forbidden minors. The obvious question is whether every minor-closed class has a finite obstruction set. It does.

Theorem (Robertson–Seymour, 1983–2004). Every minor-closed class of graphs is characterised by a **finite** set of forbidden minors.

Twenty papers over twenty years, and it is one of the deepest results in combinatorics. It has an immediate algorithmic corollary, since testing for a fixed minor H is $O(n^3)$:

Corollary. Membership in any minor-closed class is decidable in $O(n^3)$.

And here is the strangest part. The theorem does not tell you what the forbidden minors *are*. It proves the set is finite without identifying it — so for a given minor-closed class you know a cubic algorithm exists and may have no way to write one down. For graphs embeddable on the torus, over 17,000 forbidden minors are known and the complete list is still unknown.

This is non-constructivity of a different order from Chapter 24's. The probabilistic method proves an object exists without producing it; Robertson–Seymour proves an *algorithm* exists without producing it. Knowing your problem is decidable in cubic time while having no means to decide it is an unusual position to be in.

Treewidth

The other half of the theory is a parameter measuring how tree-like a graph is.

A **tree decomposition** assigns each node of a tree a **bag** of vertices such that:

1. every vertex is in some bag;
2. every edge has both endpoints in some bag;
3. for each vertex, the bags containing it form a connected subtree.

The **width** is the largest bag size minus one; the **treewidth** is the minimum width over all decompositions. The minus one is a convention chosen so that trees have treewidth 1 rather than 2.

Condition 3 is the one that carries the content. Without it, you could put every vertex in one bag and the definition would be vacuous.

Computing treewidth goes more easily through **elimination orderings**:

```
def eliminate(g, order):
    adjacency = {v: set(g.neighbours(v)) for v in g.vertices()}
    remaining, width = set(g.vertices()), 0
    for v in order:
        nbrs = adjacency[v] & remaining - {v}
        width = max(width, len(nbrs))
        for a, b in itertools.combinations(nbrs, 2): # the fill edges
            adjacency[a].add(b); adjacency[b].add(a)
        remaining.discard(v)
    return width
```

`treewidth(G)` is the minimum of this over all $n!$ orderings. That is exact and hopeless past $n = 8$, which is appropriate — computing treewidth is NP-hard, though it is fixed-parameter tractable in the width itself (Bodlaender).

P5	n=5	treewidth=1
tree	n=7	treewidth=1
C5	n=5	treewidth=2
K4	n=4	treewidth=3
K5	n=5	treewidth=4
K33	n=6	treewidth=3
grid2x3	n=6	treewidth=2
grid3x3	n=9	treewidth=3

Forests have treewidth 1, cycles 2, K_n exactly $n - 1$. The $r \times c$ grid has treewidth $\min(r, c)$ — so **planar graphs have unbounded treewidth**, and planarity and bounded treewidth are genuinely different restrictions.

held	ch31	Treewidth is 1 exactly for forests with at least one edge (51
graphs)		
held	ch31	Treewidth of K_n is $n-1$, and of a cycle is 2 (7 graphs)
held	ch31	Treewidth never increases when passing to a subgraph (51 graphs)
held	ch31	Chordal graphs have treewidth = max clique size - 1 (44 graphs)

The last line connects to Chapter 19. Chordal graphs are exactly the graphs whose elimination produces no fill edges, so their treewidth is read straight off their largest clique — which is why chordality and treewidth keep appearing in the same sentences.

Courcelle's theorem

Bounded treewidth is worth having because of what you can do with it.

Theorem (Courcelle, 1990). Every graph property expressible in monadic second-order logic can be decided in linear time on graphs of bounded treewidth.

That covers 3-colourability, Hamiltonicity, independent set, dominating set, and essentially every problem in Part V. All NP-hard in general; all linear time when the treewidth is bounded.

The mechanism is dynamic programming over the decomposition tree: process bags from the leaves up, keeping for each bag a table of partial solutions indexed by the bag's internal configuration. The table has size exponential in the **width** and the number of bags is linear in n , giving $f(k) \cdot n$.

This book does not implement that DP. Doing it correctly means building the decomposition tree and handling introduce, forget and join nodes, and a bug in it would be invisible against the brute-force answers used everywhere else in the harness. The same choice was made for Hopcroft–Tarjan planarity in Chapter 17 and for the four colour theorem in Chapter 18: where the honest options are a long correct implementation or a short wrong one, this book describes and says it is describing.

The practical caveat is that $f(k)$ is often brutal — for Courcelle's theorem in full generality it is a tower of exponentials in the formula size. "Linear time" hides a constant that can exceed the age of the universe. Hand-written DPs for specific problems do far better, and that is what people actually use.

The graph minor structure theorem

Underneath Robertson–Seymour is a structural description worth knowing exists.

Roughly: for any fixed H , every H -minor-free graph can be built by gluing together graphs that are "almost embeddable" on surfaces of bounded genus.

So excluding a minor forces near-topological structure. This is the engine behind both the well-quasi-ordering theorem and the algorithmic results, and the statement in full takes a page. It is the reason "excluded minor" and "bounded treewidth" are the two organising ideas of structural graph theory.

Try it

```
python -c "
import sys; sys.path.insert(0, '.')
from graphs.core import Graph, complete, cycle, path, complete_bipartite
from graphs.treewidth import treewidth, tree_decomposition, is_tree_decomposition,
grid

for name, g in [('P5', path(5)), ('C5', cycle(5)), ('K4', complete(4)),
               ('K3,3', complete_bipartite(3,3)), ('2x3 grid', grid(2,3))]:
    print(f'{name:<9} treewidth = {treewidth(g)}')
print()
g = cycle(5)
bags = tree_decomposition(g, list(g.vertices()))
print('C5 bags from the natural elimination order:', [sorted(b) for b in bags])
print('is a valid tree decomposition:', is_tree_decomposition(g, bags))
print('width of THIS ordering:', max(len(b) for b in bags) - 1, ' optimal:',
      treewidth(g))
"
```

```

P5      treewidth = 1
C5      treewidth = 2
K4      treewidth = 3
K3,3    treewidth = 3
2x3 grid treewidth = 2

C5 bags from the natural elimination order: [[0, 1, 4], [1, 2, 4], [2, 3, 4], [3,
4], [4]]
is a valid tree decomposition: True
width of THIS ordering: 2  optimal: 2

```

The bags shrink as elimination proceeds, and the largest has three vertices — width 2, which is optimal for a cycle. Note the fill edge: eliminating vertex 0 joins its neighbours 1 and 4, which were not adjacent in C_5 .

Exercises

1. Give a tree decomposition of K_4 of width 3, and argue no better one exists.
2. Why does the definition of width subtract one?
3. Show that the $2 \times n$ grid has treewidth 2 for every n .
4. Planar graphs have unbounded treewidth. Which family shows this, and why does it matter for algorithms?

Solutions in Appendix E.

Takeaways

- Minor-closed classes have finite forbidden-minor characterisations (Robertson–Seymour), so membership is decidable in $O(n^3)$ — without the theorem telling you which minors, or therefore how.
- That is non-constructivity one level up from Chapter 24: an *algorithm* proved to exist and not produced.
- Treewidth measures tree-likeness: 1 for forests, 2 for cycles, $n - 1$ for K_n , $\min(r, c)$ for the $r \times c$ grid.
- Planar graphs have unbounded treewidth, so the two restrictions are independent.
- Courcelle: bounded treewidth makes every MSO-expressible property linear time. The hidden constant can be a tower of exponentials, and this book describes the DP rather than implementing it.

Chapter 32 — Expanders, and where to go next

The last idea in this book is a graph that is sparse and yet extremely well connected. Those two demands sound contradictory, and the fact that they are not is one of the most useful discoveries in modern mathematics.

The definition

An **expander** is a family of d -regular graphs on growing n whose Cheeger constant $h(G)$ stays bounded below by a constant. Every set of at most half the vertices has boundary proportional to its size, uniformly, for arbitrarily large graphs — with only $dn/2$ edges to work with.

Chapter 30's Cheeger inequality turns this into a spectral condition. Good expansion is equivalent to a **large spectral gap** $d - \lambda$, where λ is the largest non-trivial eigenvalue in absolute value. So expander construction becomes an eigenvalue problem.

Two lambdas, and getting them confused

There is a subtlety here that this book got wrong on the first pass, and it is worth the paragraph.

For a d -regular graph, d is always an eigenvalue (the all-ones eigenvector). If the graph is **bipartite**, $-d$ is one too. Two quantities are in circulation:

- **spectral_expansion** — excludes both d and $-d$. This is the λ of the Ramanujan condition. A bipartite graph has $-d$ purely *because* it is bipartite, and keeping it would brand every bipartite graph a poor expander for a reason unrelated to connectivity.
- **mixing_lambda** — excludes only d . This is the λ of the expander mixing lemma below, and $-d$ must be kept.

Using one function for both made the mixing lemma fail immediately on $K_{3,3}$. Take $S = \{0\}$ and $T = \{1\}$, both on the same side: $e(S, T) = 0$ while $d|S||T|/n = 0.5$, so the discrepancy is 0.5 . With -3 excluded the bound is 0 , and the lemma is violated. The -3 eigenvalue is exactly what accounts for that discrepancy — it is not noise.

The same symbol, two definitions, and only one is right in each context.

The mixing lemma

Theorem (expander mixing lemma). For a d -regular graph with $\lambda = \text{mixing_lambda}(G)$ and any vertex sets S, T :

$$|e(S, T) - d|S||T|/n| \leq \lambda \sqrt{(|S||T|)}.$$

The middle term is what you would expect if edges were placed at random. So the lemma says a graph with a small λ has an edge distribution **indistinguishable from random** at the resolution of set sizes. That is what "pseudorandom" means precisely, and it is why expanders substitute for randomness in so many places.

```
held      ch32 Expander mixing lemma: |e(S,T) - d|S||T|/n| <= lambda sqrt(|S||T|) (11 graphs)
```

Checked over **every** pair of subsets, against a spectrum from Chapter 29's Jacobi solver — combinatorics on one side, linear algebra on the other.

How good can expansion get?

Theorem (Alon–Boppana). For d -regular graphs, $\lambda \geq 2\sqrt{d-1} - o(1)$ as $n \rightarrow \infty$.

There is a floor. No infinite family of d -regular graphs can have λ below $2\sqrt{d-1}$.

A graph meeting it is called **Ramanujan**: $\lambda \leq 2\sqrt{d-1}$, optimal expansion. They exist — Lubotzky–Phillips–Sarnak and Margulis constructed infinite families in 1988, using deep number theory, and the name comes from the Ramanujan conjecture their proof relies on. Marcus, Spielman and Srivastava gave a very different existence proof for bipartite Ramanujan graphs of every degree in 2013, using interlacing polynomials.

The Petersen graph is Ramanujan: $d = 3$, $\lambda = 2$, and $2\sqrt{2} \approx 2.83$.

```
held      ch32 The Petersen graph is Ramanujan (1 graphs)
```

Why anyone cares

Expanders are sparse objects with dense-graph connectivity, so they appear wherever you need robustness cheaply:

- **Derandomisation.** A random walk on an expander hits any large set quickly, so $O(\log n)$ random bits can substitute for $O(n)$ in many algorithms.
- **Error-correcting codes.** Expander codes decode in linear time and approach the Shannon limit.
- **Network design.** Constant-degree, constant-diameter, and robust to node failure.
- **Property testing.** Distinguishing a graph from one far from having a property.
- **The PCP theorem.** Expanders are used in the gap amplification step, and hence in the inapproximability results quoted in Chapter 23.

The recurring theme: **explicit constructions are much harder than existence proofs**. A random d -regular graph is an expander with probability tending to 1 — a one-paragraph argument in Chapter 24's style — while explicit families took until 1988 and required tools from a different field. That is the same gap as Ramsey lower bounds in Chapter 28, and it is the characteristic shape of the probabilistic method's legacy.

Try it

```
python -c "
import sys, math; sys.path.insert(0, '.')
from graphs.core import complete, cycle, petersen, complete_bipartite
from graphs.spectral import (spectral_expansion, mixing_lambda, is_ramanujan,
                             cheeger_constant, algebraic_connectivity)
print(f\" {'graph':<10} {'d':>2} {'expansion l':>12} {'mixing l':>9}
{'2sqrt(d-1)':>11} {'ram':>5} {'h(G)':>6}\")
for name, g in [('C6', cycle(6)), ('K4', complete(4)), ('K3,3',
complete_bipartite(3,3)),
                ('petersen', petersen())]:
    d = g.degree(0)
    print(f'  {name:<10} {d:>2} {spectral_expansion(g):>12.4f}
{mixing_lambda(g):>9.4f} '
          f' {2*math.sqrt(d-1):>11.4f} {str(is_ramanujan(g)):>5}
{cheeger_constant(g):>6.3f}')
```

graph	d	expansion l	mixing l	2sqrt(d-1)	ram	h(G)
C6	2	1.0000	2.0000	2.0000	True	0.667
K4	3	1.0000	1.0000	2.8284	True	2.000
K3,3	3	0.0000	3.0000	2.8284	True	1.667
petersen	3	2.0000	2.0000	2.8284	True	1.000

Look at the $K_{3,3}$ row: expansion $\lambda = 0$ and mixing $\lambda = 3$. Same graph, same spectrum, and the two columns differ by the whole degree — because one excludes -3 and the other must not. That single row is the distinction this chapter opened with.

Compare C_6 and the Petersen graph. Both have mixing $\lambda = 2$ and both are nominally "Ramanujan", yet a cycle is not an expander in any useful sense — and the single-graph table cannot show that. Expansion is a property of a *growing family*, so you have to watch it grow:

C_4	$h = 1.0000$	$(4/n = 1.0000)$
C_8	$h = 0.5000$	$(4/n = 0.5000)$
C_{12}	$h = 0.3333$	$(4/n = 0.3333)$
C_{14}	$h = 0.2857$	$(4/n = 0.2857)$

$h(C_n) = 4/n \rightarrow 0$. You can always cut a cycle with two edges, however large it gets, so the boundary-to-size ratio vanishes. An expander family keeps h bounded *below* by a constant, and no amount of inspecting one finite cycle reveals that this one does not.

That is the trap this chapter's table sets and the reason the definition quantifies over a family: **no single finite graph is or is not an expander**. Calling C_6 Ramanujan is technically correct and tells you nothing about the family it belongs to.

Where to go next

This book stops here. The natural continuations:

Structural graph theory. Diestel's *Graph Theory* is the standard modern reference and covers minors, connectivity and extremal theory properly.

Extremal and probabilistic. Alon and Spencer's *The Probabilistic Method* is the book on Chapter 24's subject and goes far past it — the local lemma, martingales, entropy.

Spectral. Spielman's lecture notes on spectral and algebraic graph theory, and Chung's *Spectral Graph Theory* for the normalised Laplacian this book skipped.

Algorithms. Williamson and Shmoys on approximation; Cygan et al. on parameterised algorithms, which is the modern treatment of Chapter 23's second half.

Open problems you are now equipped to read: the values of $R(5,5)$ and $R(6,6)$; the Zarankiewicz problem for bipartite H (Chapter 27); whether $P = NP$; the reconstruction conjecture, that a graph on at least three vertices is determined by its multiset of vertex-deleted subgraphs — open since 1942 and embarrassingly easy to state.

Exercises

1. Why is a cycle not an expander, despite being connected and regular?
2. Verify the Petersen graph is Ramanujan from its spectrum $\{3, 1^5, (-2)^4\}$.
3. Explain why $-d$ must be included for the mixing lemma but excluded for the Ramanujan condition.
4. Give the one-paragraph argument that a random d -regular graph is likely an expander, and say why this does not produce one.

Solutions in Appendix E.

Takeaways

- An expander is a *family* of sparse regular graphs with expansion bounded below. No single finite graph is an expander.
- Cheeger (Chapter 30) makes expansion a spectral condition: good expansion is a large spectral gap.
- Two different λ s are in circulation. The mixing lemma keeps $-d$; the Ramanujan condition drops it. Confusing them breaks the mixing lemma on $K_{3,3}$ immediately.
- The mixing lemma says small λ implies edges distributed as if at random — the precise content of "pseudorandom".
- Alon–Boppana puts a floor at $2\sqrt{d-1}$; Ramanujan graphs meet it, and the Petersen graph is one.
- Random d -regular graphs are expanders almost surely; explicit constructions took until 1988 and needed number theory. Existence is easy, construction is hard — the shape of this whole final part of the book.

Appendix A — Notation

Every symbol used in this book, with the chapter that introduces it.

The graph itself

Symbol	Meaning	Ch
$G = (V, E)$	a graph: vertex set and edge set	1
n	number of vertices, $ V $	1
m	number of edges, $ E $	1
uv	the edge joining u and v	1
$\bar{G}$	the complement	1
$G[S]$	the subgraph induced on vertex set S	1
$G - v, G - e$	delete a vertex, delete an edge	4
G / e	contract an edge	17

Named graphs

Symbol	Meaning	Ch
K_n	complete graph, $n(n-1)/2$ edges	1
P_n	path on n vertices, $n-1$ edges	1
C_n	cycle on n vertices, n edges, $n \geq 3$	1
$K_{\{a,b\}}$	complete bipartite graph, ab edges	1
$T(n, r)$	Turán graph: complete r -partite, parts as equal as possible	27

Vertices and degrees

Symbol	Meaning	Ch
$N(v)$	the neighbours of v	1
$\deg(v)$	the degree of v , $ N(v) $	3
$\Delta(G)$	maximum degree	3
$\delta(G)$	minimum degree	3
$d(G)$	degeneracy	15

Distance and connectivity

Symbol	Meaning	Ch
$d(u, v)$	distance: length of a shortest $u-v$ path	4
$\kappa(G)$	vertex connectivity	12
$\lambda(G)$	edge connectivity	12
INF	unreachable, <code>float("inf")</code> in the code	10

Optimisation parameters

Symbol	Meaning	Ch
$\chi(G)$	chromatic number	15
$\omega(G)$	clique number	15
$\alpha(G)$	independence number	21
$\tau(G)$	vertex cover number	21
$h(G)$	Cheeger constant / isoperimetric number	30
$\text{tw}(G)$	treewidth	31

Gallai's identity ties two of these together: $\alpha(G) + \tau(G) = n$ (Chapter 21).

Matrices and spectra

Symbol	Meaning	Ch
A	adjacency matrix	2
D	diagonal degree matrix	30
$L = D - A$	the Laplacian	30
$\lambda_1 \leq \dots \leq \lambda_n$	eigenvalues, ascending	29
λ_2	algebraic connectivity, for L	30
λ	largest non-trivial <code> eigenvalue </code> — two definitions , see Ch 32	32

The last row is a genuine trap. `mixing_lambda` excludes only d ; `spectral_expansion` also excludes $-d$ for bipartite graphs. Chapter 32 explains why the mixing lemma needs the first and the Ramanujan condition needs the second.

Asymptotics and probability

Symbol	Meaning	Ch
O, Ω, Θ	upper, lower, tight asymptotic bounds	2
$o(f)$	strictly smaller order than f	27
$G(n, p)$	random graph: each edge independently with probability p	25
$E[X]$	expectation	24
whp	with high probability: tending to 1 as $n \rightarrow \infty$	25

Conventions this book fixes

Texts differ on these, and a proof can quietly depend on the choice.

- **Simple by default.** No loops, no repeated edges, no directions, unless a chapter says otherwise. Directions arrive in Chapter 10, weights in Chapter 9.
- **The empty graph is connected.** It has zero components, so this is a convention, chosen so that "every graph is the disjoint union of its components" needs no special case (Chapter 4).
- **Treewidth subtracts one**, so trees have treewidth 1 rather than 2 (Chapter 31).
- **Eigenvalues are listed ascending**, so λ_n is the largest.
- $e(S, T)$ counts **ordered** pairs, so an edge inside $S \cap T$ is counted twice — the convention the expander mixing lemma is stated in (Chapter 32).

Appendix B — Glossary

Definitions as this book uses them, with the chapter that introduces each. Where a term has competing definitions in the literature, the disagreement is noted.

Adjacent. Two vertices joined by an edge. → Ch 1

Algebraic connectivity. The second-smallest Laplacian eigenvalue λ_2 , also called the Fiedler value. Zero exactly when the graph is disconnected. → Ch 30

Alternating path. A path alternating between edges outside a matching and edges inside it. → Ch 14

Augmenting path. In matching, an alternating path whose two endpoints are both unmatched; flipping it grows the matching by one. In flow, any $s-t$ path with spare residual capacity. → Ch 13, 14

Bipartite. Vertices split into two sets with every edge crossing. Equivalently: no odd cycle; equivalently $\chi \leq 2$. → Ch 16

Bridge. An edge whose removal increases the number of components. Equivalently: an edge on no cycle. → Ch 4, 12

Chordal. Every cycle of length at least 4 has a chord. Equivalently: has a perfect elimination ordering. Chordal graphs are perfect. → Ch 19

Chromatic number $\chi(G)$. Fewest colours in a proper colouring. NP-hard to compute. → Ch 15

Clique. A set of pairwise adjacent vertices. $\omega(G)$ is the largest. → Ch 21

Complement $\bar{G}$. Same vertices, edges exactly where G has none. → Ch 1

Component. An equivalence class of the "joined by a path" relation. → Ch 4

Connected. Exactly one component. This book calls the empty graph connected — a convention, not a consequence. → Ch 4

Cospectral. Two graphs with the same adjacency spectrum. They need not be isomorphic; $K_{1,4}$ and $C_4 + K_1$ are the smallest pair. → Ch 29

Cut. For flow, a partition (S, T) with $s \in S$, $t \in T$; its capacity counts only arcs from S to T . For connectivity, a set whose deletion separates two vertices. → Ch 12, 13

Cut property. The strictly-lightest edge across any cut is in every minimum spanning tree. "Strictly" is load-bearing. → Ch 9

Degeneracy $d(G)$. The largest k such that every subgraph has a vertex of degree at most k . Gives $\chi \leq d + 1$, never worse than $\Delta + 1$. → Ch 15

Degree. The number of neighbours. → Ch 3

Degree sequence. Degrees in non-increasing order. An invariant, so it can disprove isomorphism and never prove it. → Ch 3, 5

Directed graph (digraph). Edges are ordered pairs, called **arcs**. Introduced when negative weights make undirected edges meaningless. → Ch 10

Expander. A *family* of d -regular graphs whose Cheeger constant stays bounded below. No single finite graph is an expander. → Ch 32

Forest. An acyclic graph; a disjoint union of trees. → Ch 6

Girth. The length of a shortest cycle. The Petersen graph has girth 5. → Ch 17

Graphical. A sequence of integers that is the degree sequence of some simple graph. Decided by Havel–Hakimi or Erdős–Gallai. → Ch 3

Hamiltonian. Has a cycle visiting every vertex exactly once. NP-complete, and with no known good characterisation. → Ch 20

Hypohamiltonian. Not Hamiltonian, but $G - v$ is Hamiltonian for every v . The Petersen graph is the standard example. → Ch 20

Independent set. Pairwise non-adjacent vertices. $\alpha(G)$ is the largest. → Ch 21

Induced subgraph. $G[S]$: keep S and *all* edges between its members. Distinct from a subgraph, which may drop edges. The distinction matters for perfection. → Ch 1, 19

Invariant. Anything preserved by isomorphism. One-sided: differences disprove, agreement proves nothing. → Ch 5

Isomorphism. A bijection of vertices preserving adjacency. In NP, not known to be in P, and believed not NP-complete. → Ch 5

Leaf. A vertex of degree 1. Every tree on $n \geq 2$ vertices has at least two. → Ch 6

Matching. Edges no two of which share a vertex. → Ch 14

Minor. Obtainable by deleting vertices, deleting edges, and contracting edges. → Ch 17, 31

Min–max theorem. A result equating a maximum and a minimum. Weak duality ($\max \leq \min$) is free; the content is always the other direction. Menger, max-flow min-cut, König and Hall are one theorem in four costumes. → Ch 12–14

Perfect. $\chi(H) = \omega(H)$ for **every induced subgraph** H . The quantifier makes the class hereditary. C_5 is the smallest imperfect graph. → Ch 19

Planar. Drawable with no crossings. A property of the graph, since it quantifies over all drawings. → Ch 17

Proper colouring. Adjacent vertices get different colours. → Ch 15

Ramanujan. A d -regular graph with $\lambda \leq 2\sqrt{d-1}$ — optimal expansion by Alon–Boppana. The Petersen graph is one. → Ch 32

Regular. Every vertex has the same degree. d -regular graphs have spectral radius exactly d . → Ch 29

Residual network. The flow network augmented with reverse arcs carrying the current flow. The reverse arcs are what let the algorithm undo a decision. → Ch 13

Rotation system. A cyclic order of neighbours at each vertex — the entire combinatorial content of an embedding. → Ch 17

Simple. No loops, no repeated edges. The default throughout. → Ch 1

Simplicial vertex. One whose neighbours form a clique. Every chordal graph has one. → Ch 19

Spanning tree. A subgraph that is a tree and includes every vertex. K_n has $n^{(n-2)}$ labelled ones. → Ch 7

Threshold. For a monotone property in $G(n, p)$, the function p^* below which it almost never holds and above which it almost always does. Every monotone property has one. → Ch 25

Tree. Five equivalent definitions; connected and acyclic is the usual one. → Ch 6

Tree decomposition. A tree of bags satisfying three conditions, the third — that the bags containing any vertex form a connected subtree — carrying all the content. → Ch 31

Treewidth. Minimum width over all tree decompositions, where width is largest bag size minus one. NP-hard to compute. → Ch 31

Vertex cover. A set of vertices touching every edge. $\tau(G)$ is the smallest. → Ch 21

Walk / path / cycle. A walk may repeat; a path repeats nothing; a cycle closes. Every walk contains a path with the same endpoints, which is what makes connectivity an equivalence relation. → Ch 4

Weisfeiler–Leman (colour refinement). A fast, sound, incomplete isomorphism heuristic. Blind to any two regular graphs of the same size and degree. → Ch 5

Appendix C — Further reading

Where to go for each part of this book, and what each source is actually good at.

General references

Diestel, *Graph Theory*. The standard modern graduate text, and the one to own if you own one. Complete and rigorous on connectivity, minors, extremal theory and the Robertson–Seymour programme. Freely available from the author's site. → Parts I–IV, VII

Bondy and Murty, *Graph Theory*. Broader and gentler than Diestel, with more algorithmic content and a large stock of exercises. → Parts I–V

West, *Introduction to Graph Theory*. The most approachable of the three, and the best source of worked examples if this book's proofs moved too quickly. → Parts I–IV

By subject

Alon and Spencer, *The Probabilistic Method*. The book on Chapter 24's subject, and it goes far past what this book covers — the Lovász local lemma, martingale concentration, entropy methods, and the second moment arguments Chapter 26 gestured at. → Ch 24–26, 28

Bollobás, *Random Graphs*. The reference for Chapters 25 and 26. Thresholds, the critical window at $c = 1$, and the $\Theta(n^{2/3})$ behaviour this book only stated. → Ch 25, 26

Bollobás, *Extremal Graph Theory*. Turán-type problems in depth, including the bipartite cases where Erdős–Stone falls silent. → Ch 27

Graham, Rothschild and Spencer, *Ramsey Theory*. Van der Waerden, Hales–Jewett, and the general "complete disorder is impossible" programme. → Ch 28

Godsil and Royle, *Algebraic Graph Theory*. The thorough treatment of Chapters 29 and 30 — interlacing, strongly regular graphs, and the automorphism-group material this book skipped entirely. → Ch 29, 30

Chung, *Spectral Graph Theory*. Built around the **normalised** Laplacian $L = I - D^{-1/2} A D^{-1/2}$, which this book did not use and which is the right object for irregular graphs. → Ch 30

Spielman, lecture notes on spectral and algebraic graph theory. Freely available, modern, and the clearest route into Cheeger's inequality and spectral clustering. → Ch 30, 32

Hoory, Linial and Wigderson, "Expander graphs and their applications" (*Bulletin of the AMS*, 2006). A survey, and the single best entry point to Chapter 32. → Ch 32

Algorithms

Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*. The reference implementation of everything in Part III. The max-flow network in Chapter 13 is theirs. → Ch 8–14

Williamson and Shmoys, *The Design of Approximation Algorithms*. Chapter 23's first half, properly: LP rounding, primal–dual, and the inapproximability results only quoted here. Freely available. → Ch 23

Cygan et al., *Parameterized Algorithms*. Chapter 23's second half. Bounded search trees, kernelisation, treewidth DP done correctly — including the dynamic programming Chapter 31 described but did not implement. Freely available. → Ch 23, 31

Kleinberg and Tardos, *Algorithm Design*. The best explanations anywhere of why flow reductions work, if Chapter 13's table of encodings felt like a list of tricks. → Ch 13

Things this book deliberately skipped

Each of these is a real gap, not an oversight:

- **Hopcroft–Tarjan planarity testing** in $O(n)$. Chapter 17 searches rotation systems instead, which is exponential and visibly the theorem. See Diestel or the original paper.
- **Edmonds' blossom algorithm** for general matching. Chapter 14 gives only the bipartite case and shows where it breaks.
- **Johnson's algorithm** for all-pairs shortest paths with negative arcs on sparse graphs. Chapter 11 describes the reweighting trick without implementing it.
- **Treewidth dynamic programming** behind Courcelle's theorem. Chapter 31 says why.
- **The normalised Laplacian**, which is the correct spectral object for irregular graphs.
- **Directed graph theory** beyond shortest paths and flow: strong connectivity, tournaments, and the directed analogues of most of Parts IV–VI.

The four colour theorem

Worth its own entry, since Chapter 18 could not prove it.

- Appel and Haken's original 1976 papers, and Robertson, Sanders, Seymour and Thomas's 1997 simplification, which reduced the configurations from 1936 to 633.
- **Gonthier, "Formal Proof — The Four-Color Theorem"** (*Notices of the AMS*, 2008), on the fully machine-checked Coq version. The most interesting reading of the three, because it is about what it means for a proof to be verified rather than understood.

Open problems from this book

Stated in the chapters, and all still open:

- $R(5, 5)$, known only to lie in $[43, 46]$. → Ch 28
- The Zarankiewicz problem: the extremal number for bipartite H . → Ch 27
- The reconstruction conjecture: a graph on ≥ 3 vertices is determined by its multiset of vertex-deleted subgraphs. Open since 1942, and embarrassingly easy to state. → Ch 5
- Whether graph isomorphism is in P . → Ch 5
- Finding a clique of size $(1+\epsilon) \log_2 n$ in a random graph, when we know precisely how large the answer is. → Ch 25
- P versus NP . → Ch 22

Sequences

The counts in this book come from the OEIS, and checking against it is the cheapest possible verification of an enumerator:

- **A000088** — graphs on n vertices up to isomorphism: 1, 2, 4, 11, 34, 156, 1044. → Ch 5
- **A000272** — labelled trees, n^{n-2} : 1, 1, 3, 16, 125, 1296. → Ch 7

Appendix E — Solutions to the exercises

Every numerical answer here has been run against the book's own code rather than asserted. Where an exercise asks you to find something, one witness is given; others usually exist.

Chapter 1 — What a graph is

1. K_6 has $6 \cdot 5 / 2 = 15$ edges: each of 6 vertices meets 5 others, and each edge is counted twice. `complete(6).m` returns 15.
2. Sum the degrees: $3 \times 10 = 30$, and the handshake lemma (Chapter 3) says this is $2m$, so $m = 15$. No counting required.
3. $[(0,1), (1,2), (2,0)]$ and $[(2,0), (0,1), (1,2)]$ are the same labelled graph — order in the edge list is irrelevant, since E is a set. $[(0,1), (1,2), (2,3)]$ on four vertices is a different graph, and $[(0,2), (2,1), (1,0)]$ on three is the *same* graph again, since the same pairs are joined.
4. Three pairwise edges are indistinguishable from three separate two-author collaborations that never met. The information "these three worked together *as a group*" is not recoverable from the triangle. A **hypergraph**, whose edges are arbitrary subsets, retains it.

Chapter 2 — Representations

1. Expected edges = $p \cdot C(1000, 2) = 0.01 \times 499500 \approx 4995$. The matrix has $1000^2 = 1,000,000$ entries, so about 99.5% of it is zeros.
2. The adjacency matrix. Counting triangles through v means intersecting neighbourhoods, and with bitmask rows that is a single `&` plus a popcount — 64 vertices per machine word. Chapter 2 measured a $48\times$ win at $p = 0.5$.
3. The row is a Python arbitrary-precision integer, so `rows[u] >> v` shifts a 600-or-more-bit number and touches every digit below v . That is $O(n)$, not $O(1)$. The measurement showed the gap widening from $1.7\times$ at $n = 64$ to about $6\times$ at $n = 16384$.
4. A Python `set` carries roughly two kilobytes of overhead before holding anything, and there are n of them; a 600-bit integer is about 100 bytes. In C an adjacency list at $p = 0.05$ really would be smaller — the result is a fact about Python's containers, not about adjacency lists.

Chapter 3 — Degree

1. No. Nine vertices of degree 3 gives a degree sum of 27, which is odd, contradicting $\sum \deg(v) = 2m$. Equivalently, the number of odd-degree vertices must be even, and 9 is odd.
2. Not graphical. Havel–Hakimi: remove the 4 and subtract one from the next four entries, giving $[2, 1, 0, -1]$ — negative, so it fails. Intuitively, a vertex of degree 4 in a 5-vertex graph must be adjacent to everything, so nothing can have degree 0.
3. $\sum \deg(v) = 3 \times 10 = 30$, so $m = 15$.

4. On one vertex there is no pair to compare, so the conclusion is not even meaningful. The proof's pigeonhole step needs n degrees drawn from a set of size $n - 1$, which requires $n \geq 2$ for the set to be non-empty. The harness returns `None` for $n < 2$ rather than `True`, since the theorem says nothing there.

Chapter 4 — Walks, paths, connectivity

1. In C_5 with vertices $0..4$: the walk $0, 1, 0, 1$ repeats vertices and edges, so it is not a path. The path $0, 1, 2$ is not a cycle — it does not return to its start.

2. $(A^2)_{vv} = \deg(v)$. A closed walk of length 2 from v goes to a neighbour and comes straight back, so there is exactly one per neighbour.

3. $n - 1$, achieved exactly by the trees (Chapter 6).

4. Concatenating two paths gives a *walk*, which may repeat vertices. The walk-to-path lemma — a shortest walk between two vertices repeats nothing — converts it back to a path. Without that lemma transitivity fails and "connected component" is not well defined.

Chapter 5 — Isomorphism

1. No. C_6 is connected and two disjoint triangles are not, and connectivity is preserved by isomorphism. No computation needed.

2. Isomorphic graphs must agree on every invariant, so disagreement proves non-isomorphism; but many non-isomorphic graphs agree on any given invariant, so agreement proves nothing.

3. Refinement recolours a vertex by its own colour plus the multiset of its neighbours' colours. In a d -regular graph every vertex starts identical and has d identical neighbours, so the partition is already stable at round zero and never splits.

4. 11. `len(all_graphs_up_to_iso(4))` returns 11 — the sequence is 1, 2, 4, 11, 34, 156 (OEIS A000088).

Chapter 6 — Trees

1. 11 edges: a tree has $m = n - 1$.

2. Two, achieved by the path P_n . The longest-path argument shows both ends of a maximal path are leaves, so there are always at least two.

3. Exactly one. The two endpoints already had a unique path between them, and the new edge closes precisely that path into a cycle.

4. No. C_4 is bipartite and has a cycle, so it is not a tree. Bipartite is much weaker: it forbids odd cycles only.

Chapter 7 — Spanning trees and Cayley's formula

1. 16, and Cayley gives $4^{(4-2)} = 4^2 = 16$. `len(spanning_trees(complete(4)))` confirms it.

2. `[1, 2]`. Remove the smallest leaf (0) and record its neighbour 1; then the smallest leaf is 1, whose neighbour is 2. Two vertices remain, so we stop — the sequence has length $n - 2 = 2$.

3. A vertex appearing $\deg(v) - 1$ times means a leaf, with $\deg = 1$, appears zero times. So the labels missing from the sequence are exactly the leaves.
4. 1296 counts trees on a *fixed labelled* vertex set; 6 counts isomorphism classes. Many different labellings give the same shape — Chapter 5's distinction between equality and isomorphism, applied to counting.

Chapter 8 — Traversal

1. BFS. DFS is not merely slower — it returns *a* path, which can be arbitrarily longer than the shortest. Chapter 8's example has DFS reach an adjacent vertex last, after walking a seven-edge detour.
2. Both are $O(n + m)$: each vertex is enqueued or pushed once, and each edge is examined twice, once from each endpoint. The container does not change the count of operations, only their order.
3. An odd cycle. The two endpoints are at equal distance from the source, so the two tree paths plus this edge close a cycle of odd length. This is exactly Chapter 16's algorithm.
4. Python's default recursion limit is 1000 frames, so a recursive DFS crashes on a path graph of ten thousand vertices — a graph that is merely long, not large.

Chapter 9 — Minimum spanning trees

1. The **strictly** lightest edge crossing any cut belongs to every minimum spanning tree. "Strictly" is the load-bearing word: with ties, each tied edge is in *some* MST but none need be in *all* of them.
2. C_4 with every weight 1 and a chord — or the four-vertex graph $\{(0,2), (0,3), (1,2), (1,3)\}$ with unit weights, which Chapter 9 found by exhaustive search as the smallest case. Kruskal returns $\{(0,2), (0,3), (1,2)\}$ and Prim from vertex 1 returns $\{(0,2), (1,2), (1,3)\}$, both of weight 3.
3. A minimum spanning **forest** — one tree per component. It does not fail; the union-find simply never merges across components.
4. When all edge weights are distinct. Then no tie can occur, the cut property applies with strictness everywhere, and the MST is unique.

Chapter 10 — Shortest paths

1. The final step, where the remainder of a path from the newly-reached vertex to the target is said to "only add". With a negative arc that remainder can reduce the total, and the inequality fails.
2. $O(nm)$, against Dijkstra's $O(m \log n)$. It relaxes every arc $n - 1$ times because it cannot settle anything: without non-negativity there is no vertex it can declare final.
3. A negative cycle reachable from the source. After $n - 1$ rounds no simple path can improve, so any further improvement requires a walk using more than $n - 1$ arcs profitably.
4. There is none. Going round the cycle again lowers the total without bound, so the infimum is $-\infty$ and the question is ill-posed rather than merely hard.

Chapter 11 — All-pairs distance

1. $O(n^3)$, and no data structure at all — three nested loops over a flat array, with no priority queue and no allocation.
2. k indexes the induction on *which vertices a path may pass through*; i and j iterate within a fixed stage. With i outermost, $\text{dist}[0][1]$ can be finalised while $\text{dist}[3][1]$ is still infinite. Chapter 11 measured the broken version differing on 942 of 4000 random digraphs while being correct on a path graph.
3. That vertex lies on a negative cycle. Note this differs from Bellman–Ford's test, which asks whether one is *reachable from a given source*.
4. $\text{Digraph}(2, [(0,1,1)])$: $d(0,1) = 1$ and $d(1,0) = \infty$. Directed distance is a quasimetric, not a metric.

Chapter 12 — Connectivity and Menger's theorem

1. Every $s-t$ path meets the cut, and internally-disjoint paths cannot share a cut vertex, so $\#\text{paths} \leq |\text{cut}|$. It proves nothing on its own because it gives no reason the bound is attained — plenty of natural min–max pairs have a genuine gap, $\omega \leq \chi$ among them.
2. If s and t are adjacent, no set of *other* vertices can separate them, so there is no $s-t$ vertex cut at all. The right-hand quantity is undefined, not large.
3. Both inequalities can be strict, though not in the simplest examples. Two triangles joined by a long path gives $\delta = 2$, $\lambda = 1$, $\kappa = 1$ — verified by `edge_connectivity` and `vertex_connectivity` — so $\kappa = \lambda < \delta$. For $\kappa < \lambda$, take two copies of K_4 sharing a single vertex: that cut vertex gives $\kappa = 1$, while $\lambda = 3$ since you must destroy all three edges at some vertex, and $\delta = 3$. Chaining the two constructions gives all three strict.
4. The internal arc caps flow *through* the vertex at 1, so two paths cannot share it — that is what makes the paths internally disjoint. Edges get infinite capacity because the theorem counts vertices, not edges, and an edge should never be the bottleneck.

Chapter 13 — Max-flow min-cut

1. They let the algorithm undo an earlier decision. Without them, an early greedy path can block a better later one and the search has no way to reroute — greedy path-pushing is then simply wrong, not merely suboptimal.
2. With arbitrary paths and irrational capacities, Ford–Fulkerson can run forever, converging to a value below the maximum. With integer capacities it terminates but can take time proportional to the flow *value*, which is exponential in the input size. Shortest paths (BFS) give $O(nm^2)$ regardless.
3. Integer capacities give an integer maximum flow, because every augmentation pushes an integer bottleneck. It matters because Menger and matching need the answer to be a *set* of paths or edges; a fractional optimum could not be read off as one.
4. Nothing. A cut's capacity counts only arcs from the source side to the sink side. In Chapter 13's example the arc $(3,2)$ of capacity 9 runs backwards across the cut and contributes zero.

Chapter 14 — Matching

1. An alternating path — edges alternately outside and inside M — whose two endpoints are both unmatched. Flipping it increases $|M|$ by exactly one, since it has one more non-matching edge than matching edges.
2. M is maximum if and only if no M -augmenting path exists. The proof uses **symmetric difference**: $M \Delta N$ has maximum degree 2, so its components are paths and even cycles, and if $|N| > |M|$ some component must be an augmenting path.
3. An odd cycle. An alternating walk can return to a vertex on the opposite parity, and the search cannot tell an augmenting path from a loop. Edmonds' blossom algorithm fixes it by contracting odd cycles. Chapter 14's witness is C_7 ; C_3 and C_5 do not expose the bug.
4. Maximum matching 1, minimum vertex cover 2 — verified by exhaustive search. They differ because a triangle is not bipartite, and König's theorem requires bipartiteness. This is the smallest counterexample.

Chapter 15 — Colouring

1. A proper colouring partitions the vertices into colour classes, each of which is an independent set of size at most $\alpha(G)$. Covering n vertices therefore needs at least $n / \alpha(G)$ classes.
2. C_5 : $\omega = 2$ (triangle-free) and $\chi = 3$ (odd cycle). It is the smallest such graph — every graph on at most 4 vertices is perfect, as Chapter 19's exhaustive check confirms.
3. Order the vertices by the degeneracy elimination and count: each vertex has at most d neighbours already placed when it is removed, so summing over vertices gives $m \leq d \cdot n$.
4. The crown graph on $2n = 8$ vertices has degeneracy 3, so the bound guarantees only $d + 1 = 4$ colours — and greedy in the interleaved order uses exactly 4. The bound is respected; it is simply much weaker than $\chi = 2$.

Chapter 16 — Bipartite graphs

1. If bipartite, the larger side is an independent set with at least half the vertices, and this is inherited by every subgraph. Conversely, if some subgraph has no such independent set, that subgraph contains an odd cycle, so the graph is not bipartite.
2. $\lfloor n^2/4 \rfloor$, achieved by the balanced complete bipartite graph $K_{\lfloor n/2 \rfloor, \lfloor n/2 \rfloor}$. For $n = 6$ that is $K_{3,3}$ with 9 edges. This is Mantel's theorem (Chapter 27), since bipartite graphs are triangle-free.
3. The Petersen graph has girth 5 and is not bipartite. Any graph containing an induced C_5 and no shorter cycle works, for instance C_7 , or the 5-cycle with a pendant vertex.
4. Triangle-freeness is *local* — inspect every three vertices, $O(n^3)$ — while bipartiteness is *global*, but BFS computes it in $O(n + m)$ because the level structure does all the work at once. The cheaper test yields the stronger property because a single traversal propagates a global constraint.

Chapter 17 — Planarity

1. If every degree were at least 6 then $2m = \sum \deg(v) \geq 6n$, so $m \geq 3n$, contradicting $m \leq 3n - 6$.

2. K_5 minus an edge has $n = 5$, $m = 9 \leq 3 \cdot 5 - 6 = 9$, and `is_planar` confirms it is planar. Removing one edge is exactly enough.
3. The Petersen graph has girth 5, so every face in any embedding would need at least 5 edges, giving $2m \geq 5f$. With $n = 10$, $m = 15$, Euler forces $f = 7$, but $2 \cdot 15 = 30 < 35 = 5 \cdot 7$. Contradiction.
4. One. A tree has $m = n - 1$, so Euler gives $f = 2 - n + (n-1) = 1$ — only the outer face. `planar_face_count(path(5))` returns 1.

Chapter 18 — The five and four colour theorems

1. $m \leq 3n - 6$ forces a vertex of degree at most 5 (exercise 17.1), and that holds for every subgraph since subgraphs of planar graphs are planar. So planar graphs are 5-degenerate, and Chapter 15's bound gives $\chi \leq d + 1 = 6$.
2. The final step, where a path from v_2 to v_4 is said to be unable to cross the v_1-v_3 path. Crossings do not exist in a plane embedding; that is the only topological input in the argument.
3. The graph K_4 with one vertex duplicated — for instance K_5 minus one edge — has $\chi = 4$ and is planar, as verified above. Any planar triangulation containing K_4 also works.
4. After the first Kempe swap, the colours of v 's neighbours have changed, so the configuration justifying the second swap may no longer hold. The two swaps can interfere: performing one can recreate the obstruction the other was meant to remove. Kempe assumed independence and Heawood found the case where it fails.

Chapter 19 — Perfect and chordal graphs

1. C_5 has no triangle, so $\omega = 2$. It is an odd cycle, so 2 colours are impossible and 3 suffice, giving $\chi = 3$. The book's exhaustive routines return exactly these.
2. An interval graph's vertices are intervals, adjacent when they overlap. Given a cycle of length ≥ 4 , take the interval with the leftmost right endpoint; its two cycle-neighbours both overlap it, so they overlap each other — a chord.
3. Because the class must be **hereditary** to be useful, and hereditary classes are the ones definable by forbidden induced subgraphs. Requiring only $\chi(G) = \omega(G)$ admits C_5 plus a disjoint K_3 , which has $\chi = \omega = 3$ yet contains the canonical imperfect graph.
4. P_4 (the path on 4 vertices) is self-complementary, and it is perfect — chordal, in fact, so Chapter 19's theorem applies. C_5 is the self-complementary graph that is *not* perfect.

Chapter 20 — Hamiltonicity

1. A Hamiltonian cycle in a bipartite graph alternates sides, so it visits equally many from each — forcing $a = b$. Conversely $K_{\{a,a\}}$ with $a \geq 2$ has an obvious alternating cycle. `is_hamiltonian(complete_bipartite(2,3))` is False and `(3,3)` is True.
2. A Hamiltonian cycle gives two internally-disjoint paths between any pair, so the graph is 2-connected by Menger (Chapter 12), and a 2-connected graph has no cut vertex.

3. If every degree is at least $n/2$ then any two vertices have degree sum at least n , so Dirac's hypothesis implies Ore's. For a graph satisfying Ore but not Dirac, exhaustive search gives the smallest: on 5 vertices, edges $\{(0,2), (0,3), (0,4), (1,2), (1,3), (1,4), (2,3)\}$ with degree sequence $[3,3,3,3,2]$. Vertex 4 has degree $2 < 5/2$, so Dirac fails; every non-adjacent pair still sums to at least 5, so Ore holds — and the graph is indeed Hamiltonian.

4. Removing k vertices from a Hamiltonian graph breaks the cycle into at most k arcs, so at most k components remain. The star $K_{1,3}$ fails this: delete the centre and three components remain from one deletion, so it is not Hamiltonian.

Chapter 21 — Cliques, independent sets, covers

1. C_6 : $\alpha = 3$, $\tau = 3$, $\alpha + \tau = 6 = n$. $\omega = 2$, since C_6 is triangle-free.

2. C_5 has $\omega = 2$ and $\alpha = 2$, so $\omega \cdot \alpha = 4 < 5 = n$. This is the smallest counterexample; every graph on at most 4 vertices satisfies the inequality.

3. $\tau(G)$ equals the maximum matching size, by König's theorem (Chapter 14).

4. The complement of a sparse graph is dense: $m = O(n)$ becomes $\Theta(n^2)$. An algorithm whose running time depends on m therefore slows by a factor of n , even though the reduction itself is polynomial. Complexity theory counts the reduction as cheap; your CPU does not.

Chapter 22 — NP-hardness

1. Reduce a known-hard A to B . That shows solving B would solve A , so B inherits A 's difficulty. Reducing B to A merely shows B is no harder than something hard, which is true of every problem in P as well.

2. Without contradiction edges, an independent set could pick x from one clause's triangle and $\neg x$ from another's. The chosen literals would not correspond to any consistent assignment, so the ($\Leftarrow$) direction fails.

3. Without the triangles, an independent set of size k could take several literals from one clause and none from another, leaving that clause unsatisfied. The triangles are what force exactly one literal per clause.

4. No. NP-hardness is a statement about the worst case over *all* instances. Restricting to bipartite inputs is a different problem, and it being easy says nothing about the general one — this is the third response in Chapter 23's list.

Chapter 23 — Living with hardness

1. The algorithm adds both endpoints of each edge it takes, and it takes $|M|$ edges, so it returns exactly $2|M|$ vertices. M being **maximal** means no edge can be added, so every edge of G touches a matched vertex — which is precisely what makes the result a cover.

2. A single edge: $OPT = 1$ and the heuristic returns both endpoints, giving 2. More generally any perfect matching on $2k$ vertices with no other edges.

3. For $n = 10^6$ and $k = 10$: $2^k(n+m) \approx 1024 \cdot 10^6 \approx 10^9$ operations, against $n^k = 10^{60}$. The first finishes in seconds; the second exceeds the number of atoms in the observable universe.

4. Chapter 21's equivalence preserves *exact* answers, not *parameters*. A vertex cover of size k corresponds to an independent set of size $n - k$, so the small parameter on one side becomes a huge one on the other. Parameterised complexity is sensitive to which quantity is called k , and the reduction does not preserve it.

Chapter 24 — The probabilistic method

1. Colour each vertex red or blue independently with probability $1/2$; each edge crosses with probability $1/2$; by linearity of expectation the expected crossing count is $m/2$; some outcome attains at least the mean. Independence is never used — linearity holds regardless, and the edge events are genuinely correlated when edges share a vertex.

2. Orient each edge at random. Each of the $n!$ vertex orderings is a Hamiltonian path with probability $2^{-(n-1)}$, so the expected number is $n!/2^{n-1}$, and some tournament attains at least the mean.

3. Include each vertex independently with probability p . The expected number of surviving vertices is pn and of surviving edges is p^2m . Delete one endpoint per surviving edge: the remainder is independent and has expected size $pn - p^2m$. Optimising at $p = n/(2m)$ gives $n^2/(4m)$.

4. The cut bound derandomises by conditional expectations because the expectation can be computed and compared *locally*, one vertex at a time. For Ramsey there is no known way to evaluate the conditional expectation of "no monochromatic K_k " efficiently, so the greedy step cannot be taken.

Chapter 25 — Random graphs

1. $E[\text{triangles}] = C(n, 3)p^3 \approx n^3p^3/6$. This tends to zero when $p \ll 1/n$ and to infinity when $p \gg 1/n$, so the threshold is $1/n$.

2. A variable can have a large mean while being zero almost always, if it is occasionally enormous — for instance a variable equal to n^2 with probability $1/n$ and zero otherwise has mean n and is usually zero. Ruling this out needs the variance.

3. A given vertex is isolated when none of its $n - 1$ potential edges appears, with probability $(1-p)^{n-1}$; multiply by n . Setting $p = c \ln n / n$ makes this $\approx n^{1-c}$, which tends to a constant at $c = 1$.

4. In $G(n, 1/2)$ every labelled graph on n vertices has the same probability $2^{-C(n, 2)}$, so the distribution is uniform. A statement holding with probability tending to 1 therefore holds for a fraction of all graphs tending to 1.

Chapter 26 — The giant component

1. Iterating $\beta = 1 - e^{-1.5\beta}$ converges to $\beta \approx 0.5828$. The measured largest component at $c = 1.5$, $n = 400$ was 0.581 of the graph — agreement to about three parts in a thousand.

2. The approximation assumes each newly reached vertex has $\approx n$ unexplored potential neighbours. Once a constant fraction has been explored, that count drops materially, the effective offspring mean falls below c , and the growth slows — which is what stops the giant component at βn rather than n .

3. Two components of size ϵn each have $\epsilon^2 n^2$ potential edges between them. At $p = c/n$, the probability that none appears is $(1 - c/n)^{\epsilon^2 n^2} \approx e^{-c\epsilon^2 n}$, which tends to zero. So they would almost surely be joined, contradicting their being distinct components.

4. $c < 1$ means $R_0 < 1$: each case produces fewer than one new case on average, and the outbreak dies out. The second-largest component corresponds to the largest of the small self-limiting clusters — the local outbreaks that never became epidemics.

Chapter 27 — Extremal graph theory

1. $K_{2,3}$ has 6 edges, is triangle-free (bipartite), and $n^2/4 = 6.25$, so it meets the bound as tightly as an integer can at $n = 5$.

2. The edge count is $\sum_{i < j} a_i a_j$, which for a fixed total $\sum a_i = n$ is maximised when the parts are equal as possible. Moving a vertex from a larger part to a smaller one strictly increases the product terms — the standard convexity argument, the same one Cauchy–Schwarz encodes.

3. $\chi(K_3) = 3$, $\chi(K_4) = 4$, $\chi(\text{Petersen}) = 3$ — all three confirmed by `chromatic_number`. So K_3 and the Petersen graph share $r = 2$ and give the same leading term $(1 - 1/2)n^2/2 = n^2/4$; K_4 has $r = 3$ and gives $n^2/3$. The Petersen graph has ten vertices and fifteen edges against the triangle's three and three, and forbidding either costs the same asymptotically — which is the whole point of the theorem.

4. If $\chi(H) = 2$ then $r = 1$ and the leading term $(1 - 1/1)n^2/2$ is zero, so Erdős–Stone reduces to $o(n^2)$ and determines nothing about the true order. Finding it is the Zarankiewicz problem, open in general.

Chapter 28 — Ramsey theory

1. Take any vertex of K_6 . Its 5 edges take 2 colours, so at least 3 share one — say v is red-joined to a, b, c . If any of ab, bc, ac is red, that edge plus v gives a red triangle; if none is, abc is a blue triangle.

2. The witness has red edges $\{(0,3), (0,4), (1,2), (1,4), (2,3)\}$, which is a 5-cycle. Its complement is also a 5-cycle. C_5 is triangle-free, so neither colour class contains a triangle — verified by `canonical` against `cycle(5)` in both directions.

3. $R(2,4) = 4$ (with 2 on one side, any single edge of that colour finishes it, so you need only enough vertices to force 4 mutually non-adjacent), and $R(3,3) = 6$. The recurrence gives $R(3,4) \leq 4 + 6 = 10$. The true value is 9.

4. K_{43} has $C(43,2) = 903$ edges, each independently red or blue, giving 2^{903} colourings. That is far beyond any conceivable computation — the number of atoms in the observable universe is about 2^{266} .

Chapter 29 — Spectral graph theory

1. $A(K_n) = J - I$ where J is all ones. J has eigenvalues n (once) and 0 ($n-1$ times), so A has $n - 1$ once and -1 with multiplicity $n - 1$. For K_4 the solver returns $[-1, -1, -1, 3]$.

2. $\sum \lambda_i^2 = \text{trace}(A^2)$, and $(A^2)_{vv}$ counts closed walks of length 2 from v , which is $\deg(v)$. Summing gives $\sum \deg(v) = 2m$.

3. $n(-\lambda_{\min})/(d - \lambda_{\min}) = 10 \cdot 2/(3 + 2) = 4$, and $\alpha(\text{Petersen}) = 4$ by exhaustive search. The bound is exactly tight.

4. The spectrum is an invariant, so isomorphic graphs share it and a difference disproves isomorphism. But cospectral non-isomorphic graphs exist — $K_{1,4}$ and $C_4 + K_1$ both give $\{-2, 0, 0, 0, 2\}$ — so agreement establishes nothing.

Chapter 30 — The Laplacian

1. $x^T D x = \sum_v \deg(v) x_v^2$ and $x^T A x = 2 \sum_{\{uv \in E\}} x_u x_v$. Subtracting and regrouping by edge gives $\sum_{\{uv \in E\}} (x_u^2 + x_v^2 - 2x_u x_v) = \sum_{\{uv \in E\}} (x_u - x_v)^2$. A sum of squares is non-negative, so L is positive semidefinite.

2. $L(K_3)$ has eigenvalues $\{0, 3, 3\}$. Deleting row and column 0 leaves $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ with determinant 3, and K_3 indeed has 3 spanning trees — one for each edge omitted. Cayley agrees: $3^{3-2} = 3$.

3. Each row of L sums to zero, since the diagonal entry $\deg(v)$ is cancelled by the $\deg(v)$ entries of -1 . So $L \cdot \mathbf{1} = \mathbf{0}$, making the all-ones vector an eigenvector with eigenvalue 0.

4. How hard the graph is to cut in two. P_4 can be split by removing one edge; K_4 cannot be split cheaply at all. Cheeger's inequality makes this precise, bracketing the isoperimetric number $h(G)$ between $\lambda_2/2$ and $\sqrt{2\Delta\lambda_2}$.

Chapter 31 — Minors and treewidth

1. One bag containing all four vertices: width $4 - 1 = 3$. No better exists because every edge must lie inside some bag, and K_4 's edges force any bag system covering them to contain all four vertices together — a clique always ends up in a single bag.

2. So that trees have treewidth 1 rather than 2. A tree's decomposition has bags of size 2 (one per edge), and subtracting one makes the convention match the intuition that trees are the simplest non-trivial case.

3. Process the grid column by column: each bag holds one column's two vertices plus the next column's two, so bags have size at most 3 and the width is 2. `treewidth(grid(2,n))` returns 2 for $n = 2, 3, 4$.

4. The $k \times k$ grid, whose treewidth is k and which is planar. It matters because bounded treewidth gives linear-time algorithms via Courcelle's theorem, and planarity alone does not — so the two restrictions are genuinely independent and planar problems remain hard.

Chapter 32 — Expanders, and where to go next

1. Because expansion is a property of a growing family, and $h(C_n) = 4/n \rightarrow 0$. You can always cut a cycle with two edges however large it gets, so the boundary-to-size ratio vanishes. A single C_6 reporting as "Ramanujan" says nothing about the family.

2. $d = 3$ and the non-trivial eigenvalues are 1 and -2 , so $\lambda = 2$. The Ramanujan bound is $2\sqrt{d-1} = 2\sqrt{2} \approx 2.828$, and $2 \leq 2.828$.

3. A bipartite graph has $-d$ in its spectrum purely because it is bipartite, so keeping it would brand every bipartite graph a poor expander for a reason unrelated to connectivity — hence the Ramanujan condition drops it. The mixing lemma must keep it: in $K_{3,3}$, taking $S = \{0\}$ and $T = \{1\}$ on the same side gives $e(S, T) = 0$ against an expected 0.5 , and with -3 excluded the bound would be 0 .
4. Sample a random d -regular graph and bound, via the union bound, the probability that some set of size at most $n/2$ has a small boundary; the count of bad sets is outweighed by how unlikely each is. This proves the family exists without naming one — the same gap as the Ramsey lower bound in Chapter 28, and it took until 1988 and number theory to close it explicitly.